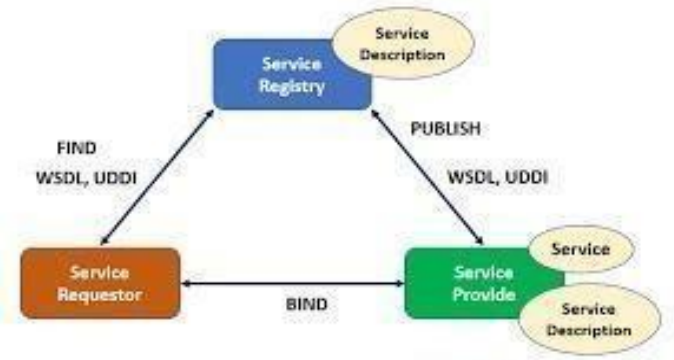
**EXP NO: 1 A** **DEVELOP A NEW WEBSERVICE FOR DESIGNING A**
**DATE:** **CALCULATOR**

**AIM:** To develop a new webservice calculator for designing a calculator using NetBeans IDE.

**DESCRIPTION:**

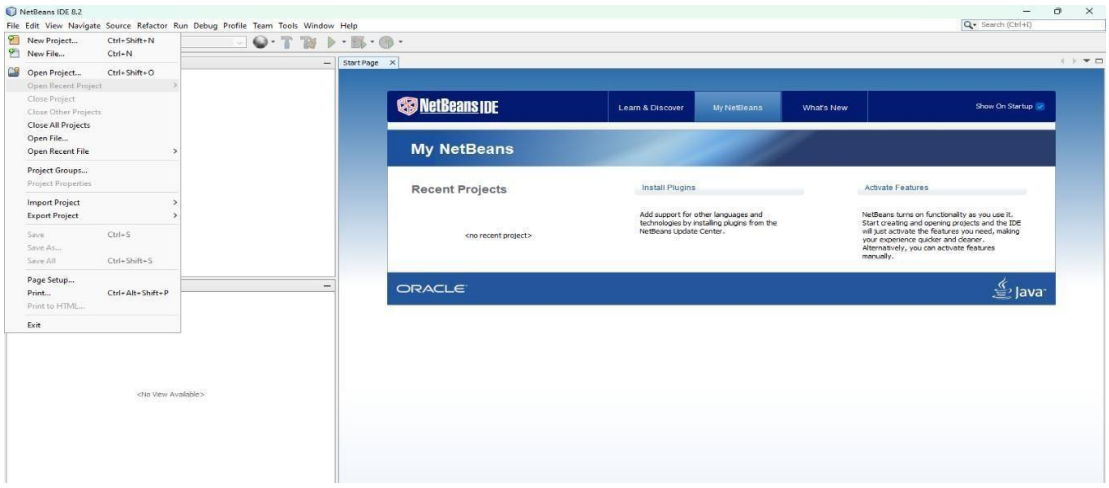



- The Service Provider publishes the WSDL (Web Services Description Language) to a Service Registry (e.g., UDDI), which includes both abstract (interface) and concrete (implementation) definitions. The Service Requestor finds the required service by querying the registry and retrieves the WSDL. Based on the WSDL, the requestor binds to the service using protocols like SOAP/HTTP.
- The four main specifications in WebServices are: XML (Extensible Markup Language), WSDL (Web Service Description Language), SOAP (Simple Object Access Protocol), UDDI (Universal Description Discovery Language).

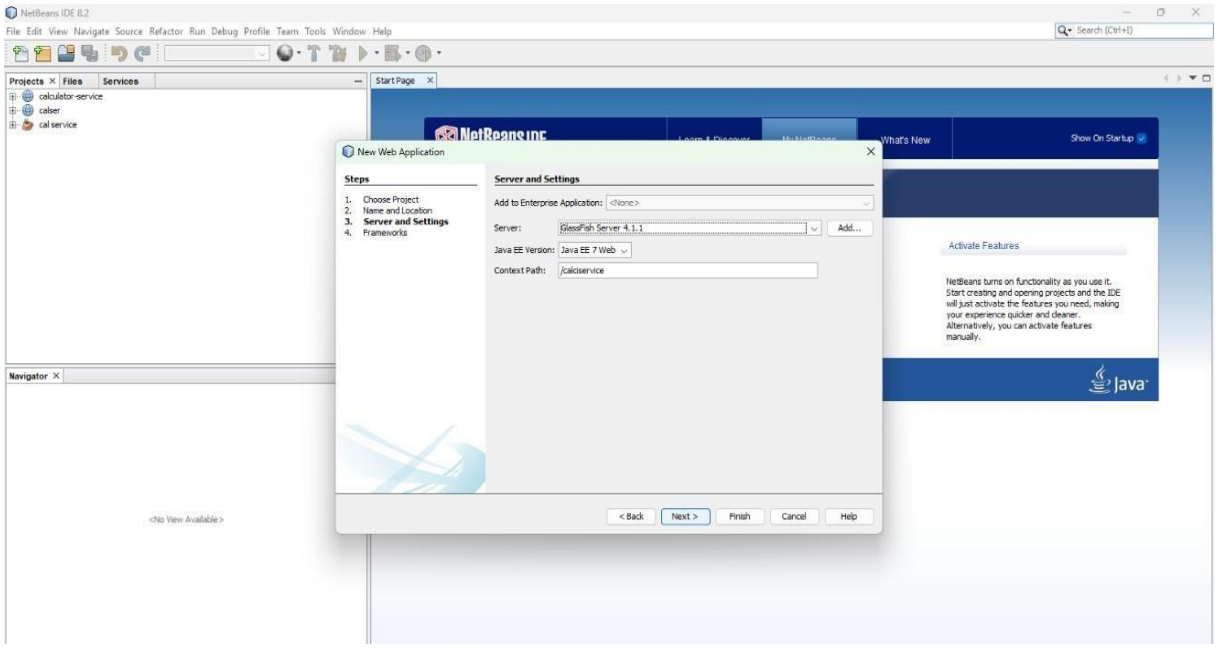
**PROCEDURE:**

**Creating a webservice Calculator**

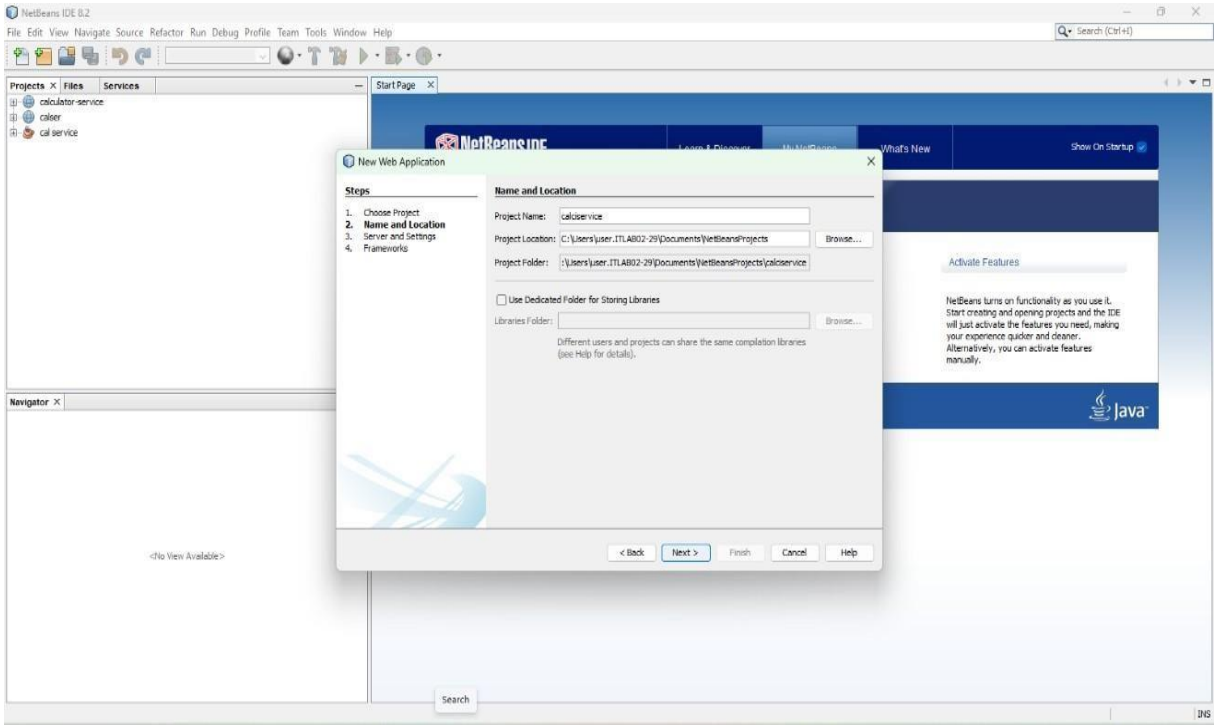
**Step 1:** Install NetBeans IDE , now go to file and click new project.



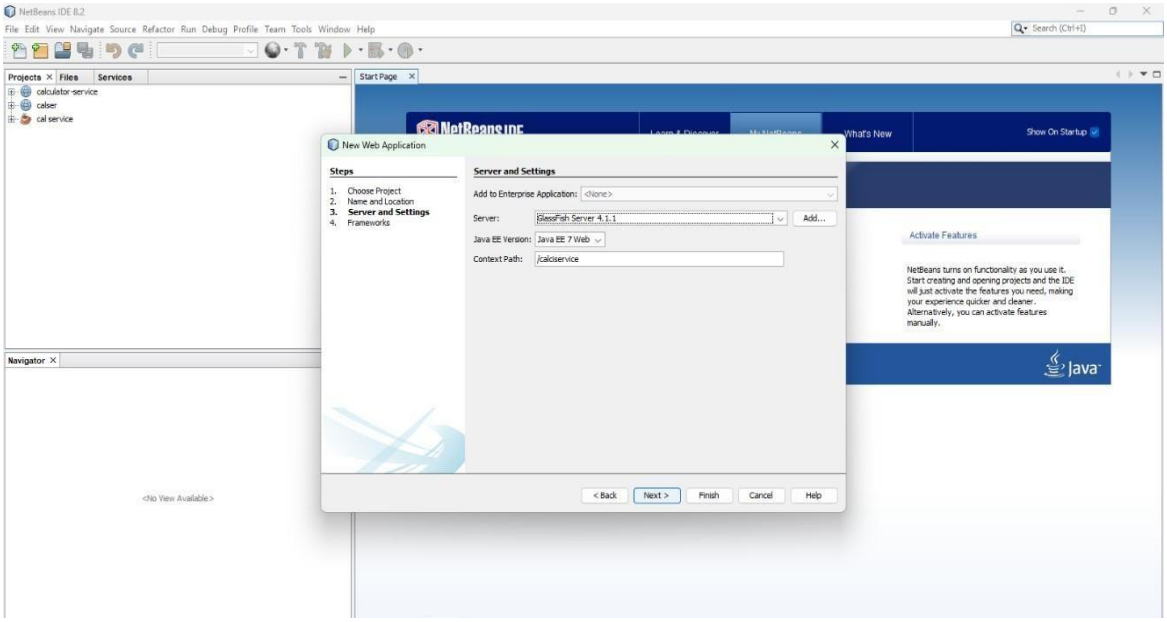

**Step 2:** Select Java Web Application , click finish and next.



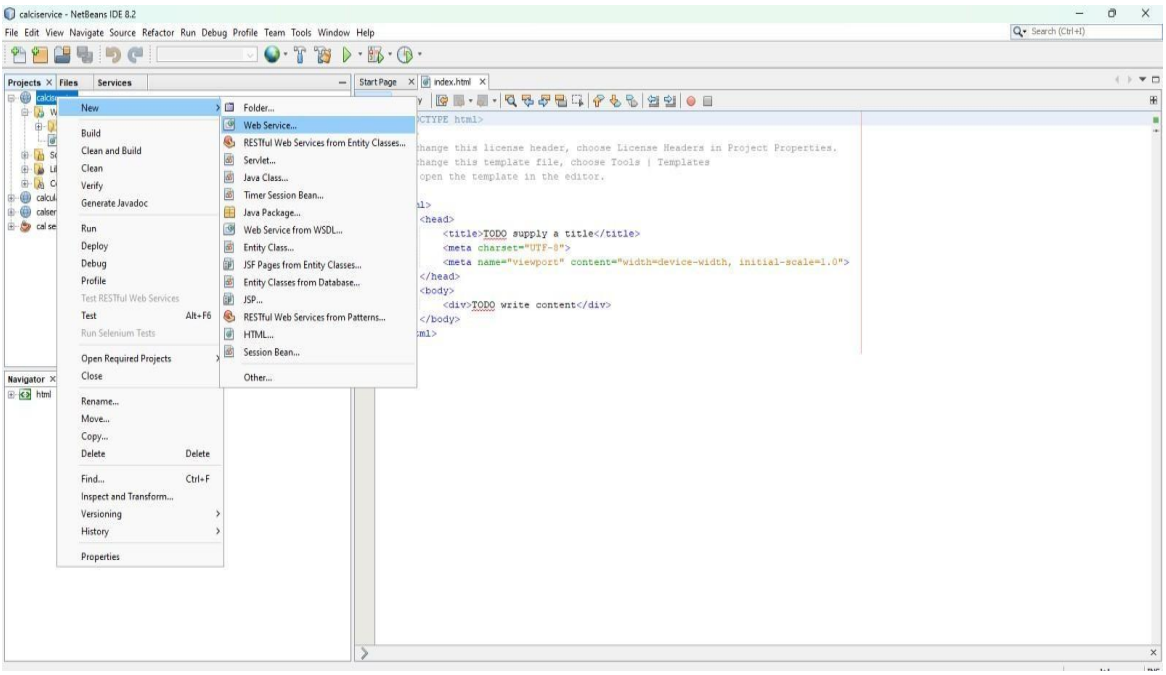

**Step 3:** Give the project name as CalculatorService and click next .Now, right click the CalculatorService , click new and create a webservice.

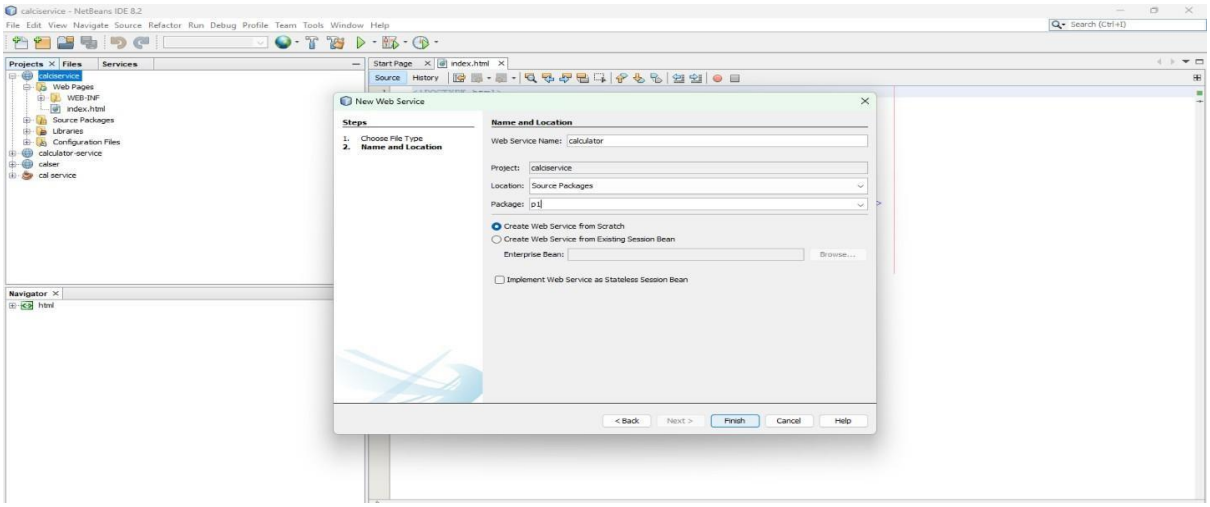



**Step 4:** In the server and settings, ensure that the server name is set to GlassFish Server and the Java EE version is Java EE 7 Web

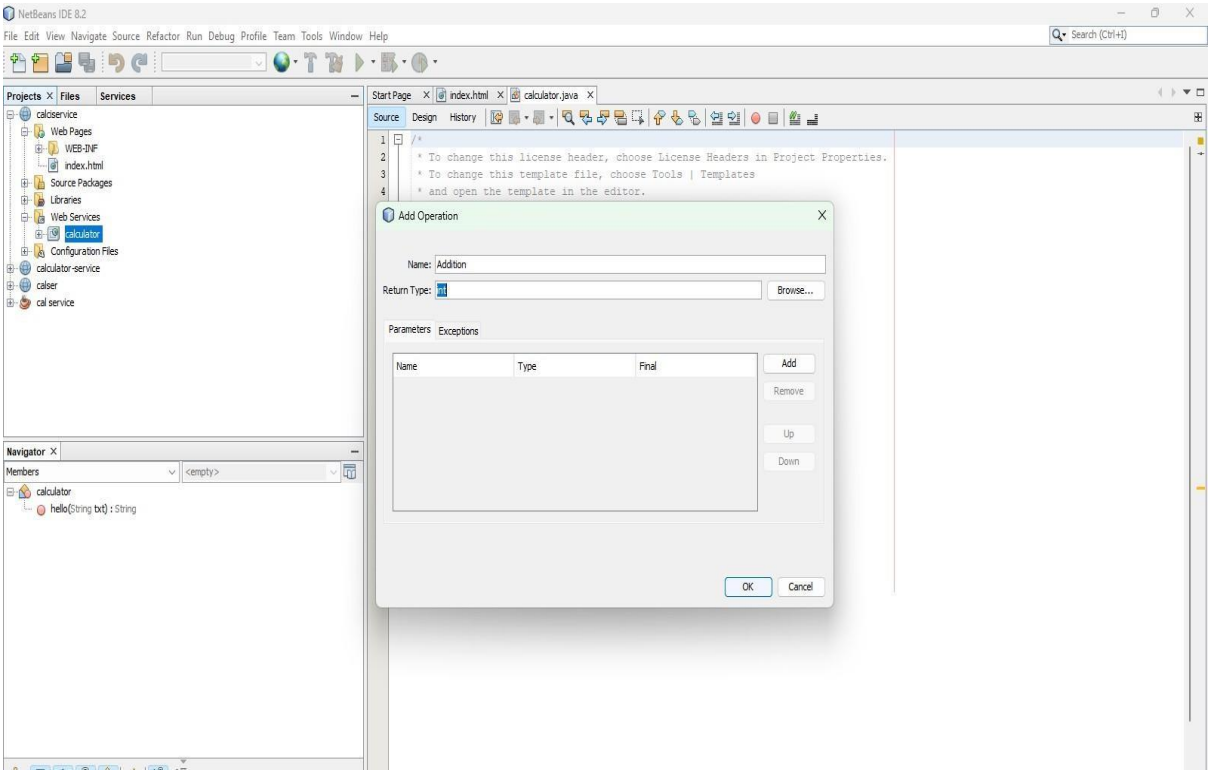



**Step 5:** Right click the service project, Select->new->web service, Enter the web service name, Assign the package name, Click finish.

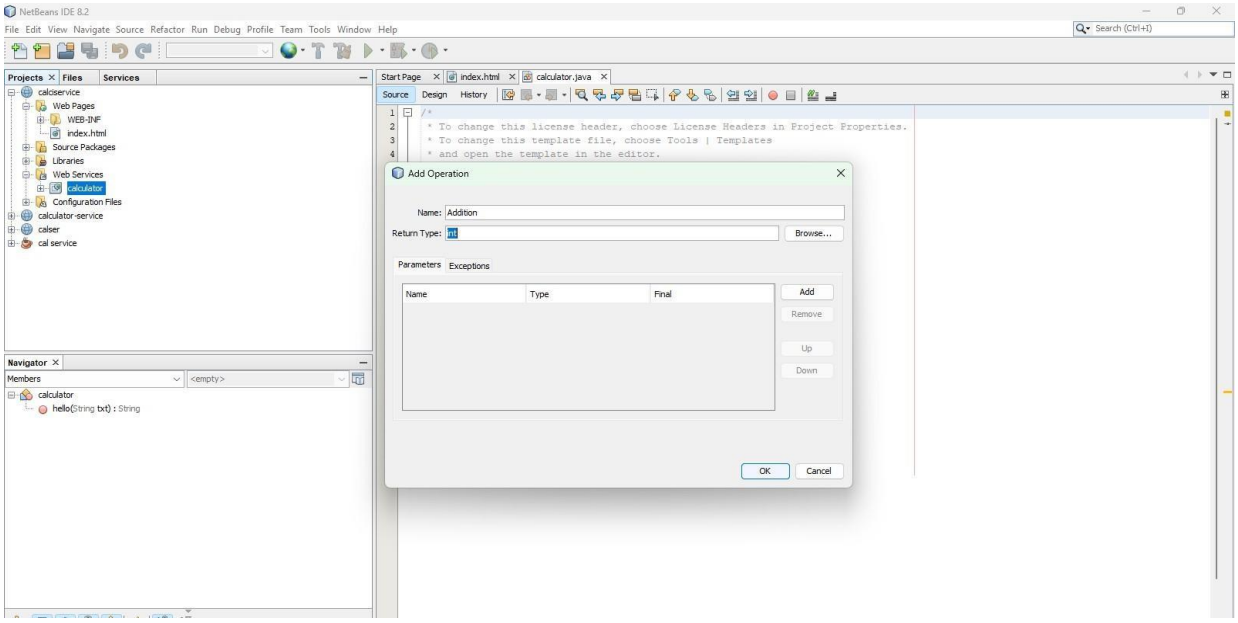




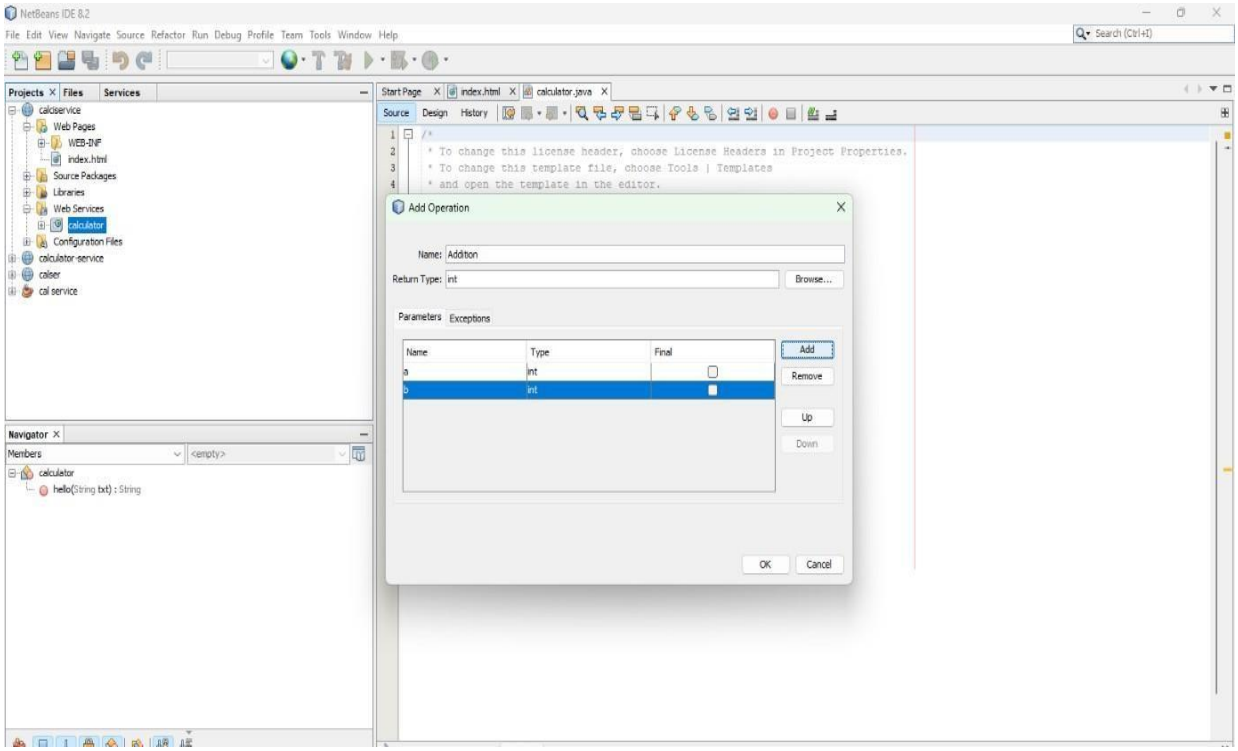

**Step 6:** Expand the webservice folder, right click on the service you have created and select add operation.



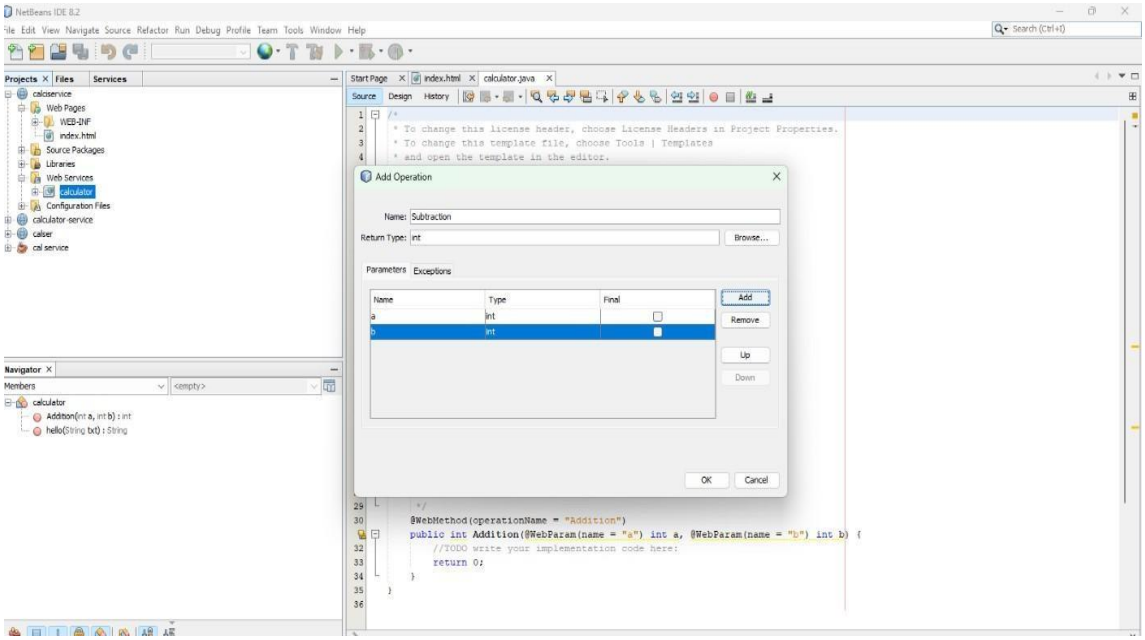

**Step 7:** Give name of the operations as add, Choose the return type as Integer(int).

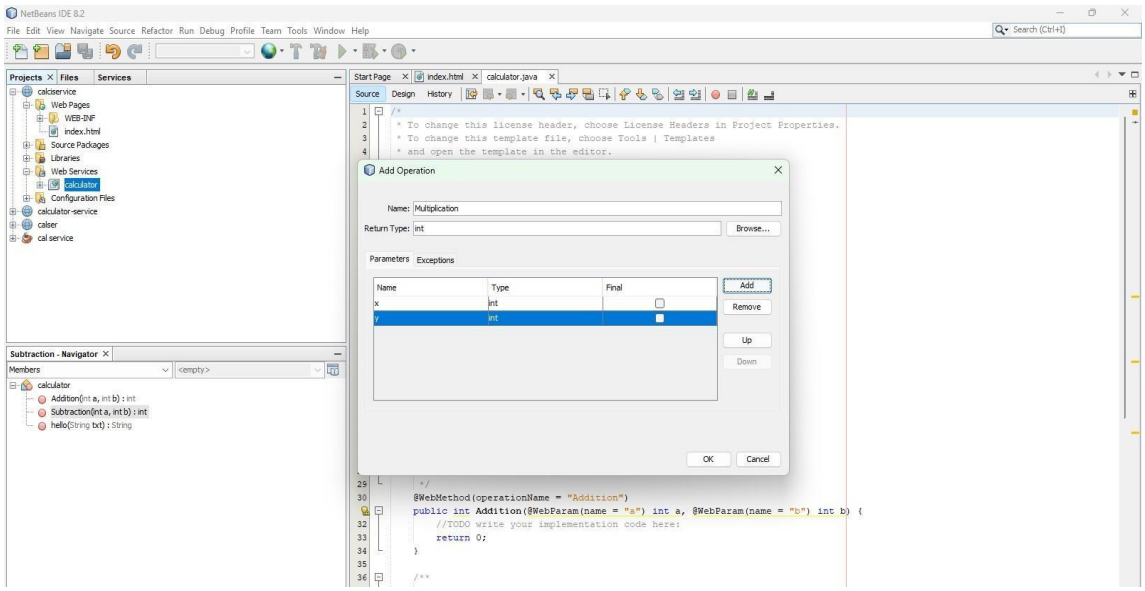



**Step 8:** Click on the add button, enter the parameters, choose the datatype as Integer, click ok.

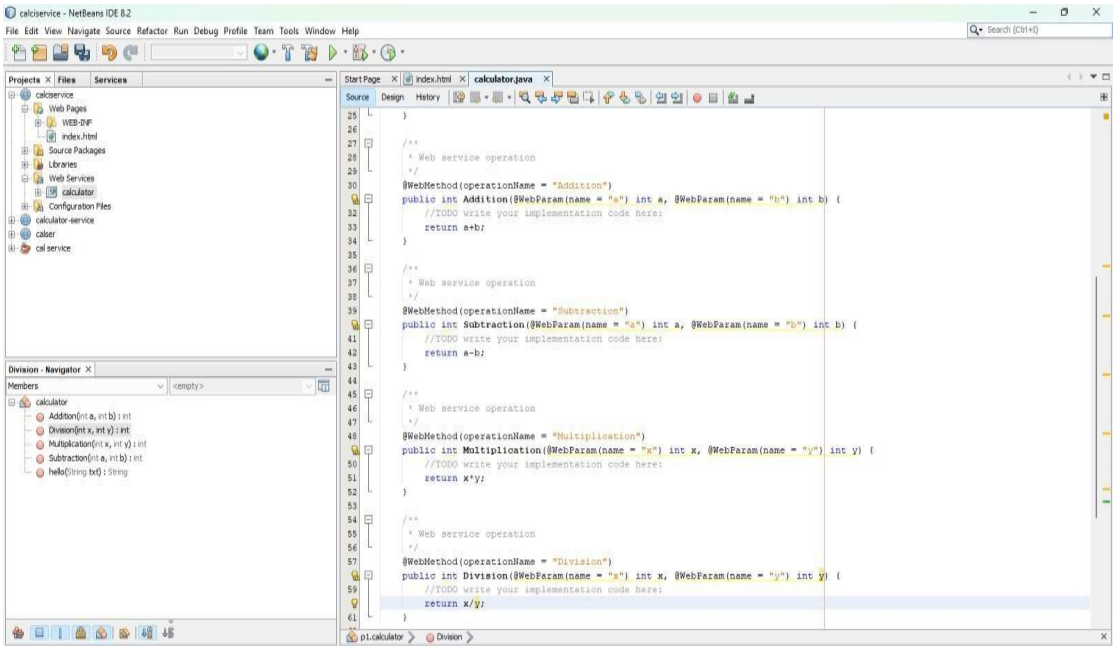



**Step 9:** Similarly create for Subtraction, Multiplication, Division operations, Add the parameters and change the datatype.

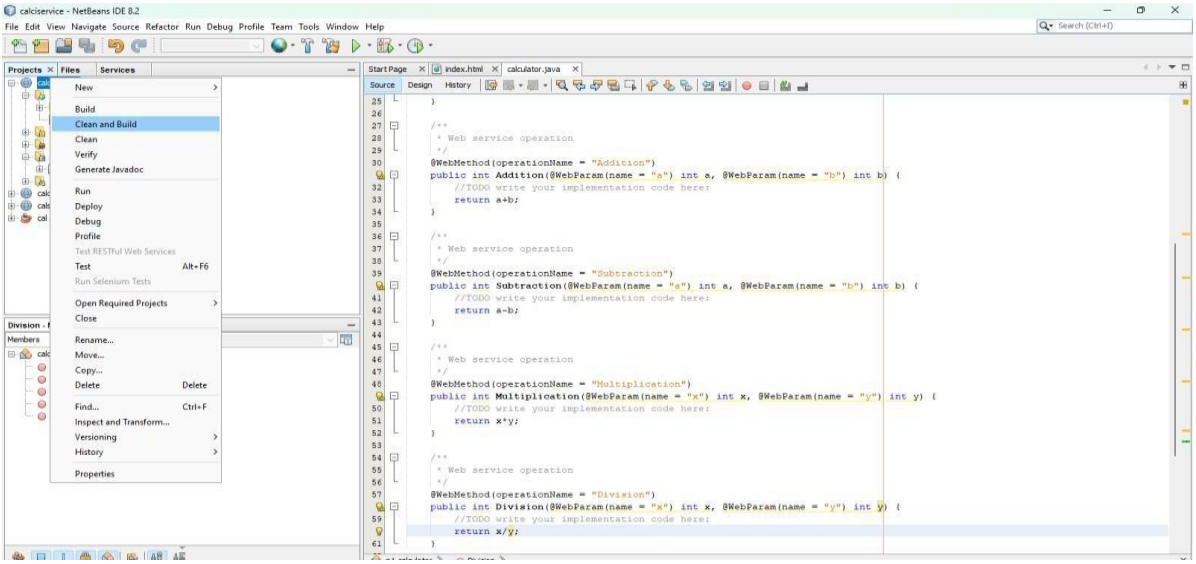


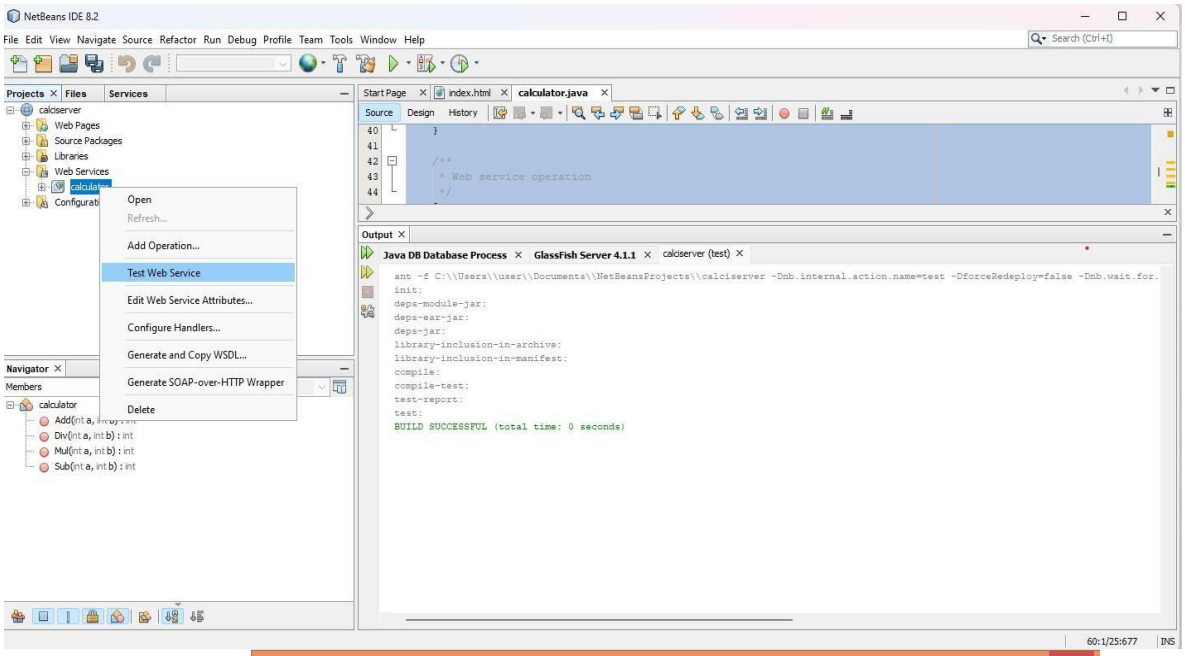

**Step 10:** In web method code, change return ‘0’ as return a+b and follow the same procedure for other operations.

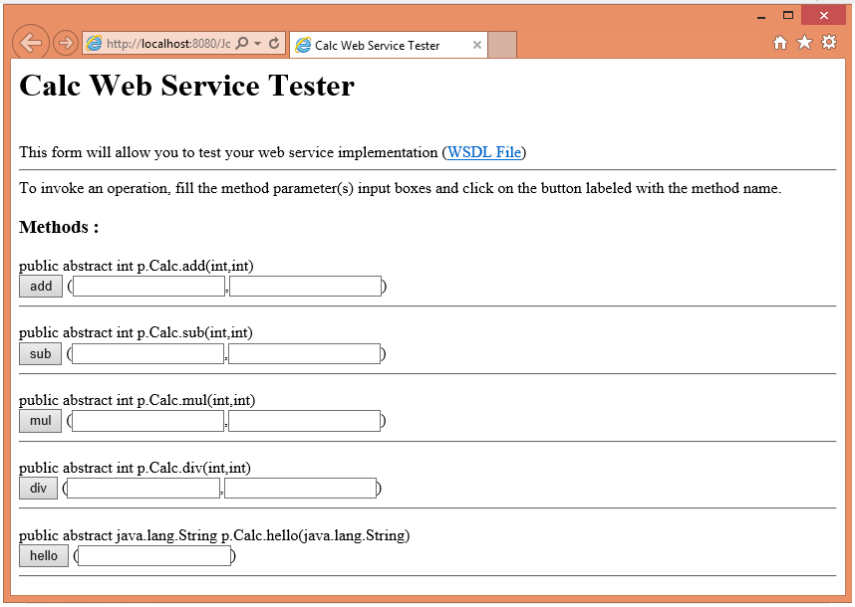



**Step 11:** Right click on the service project, click clean and build and also deploy it.




**Step 12:** Right click the web service page, give Test web service.




**division Method invocation**

**Method parameter(s)**

| Type | Value |
|---|---|
| int | 50 |
| int | 2 |

**Method returned**

int : "25"

**SOAP Request**

```
<?xml version="1.0" encoding="UTF-8"?><S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/" xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
    <SOAP-ENV:Header/>
    <S:Body>
        <ns2:division xmlns:ns2="http://p3/">
            <x>50</x>
            <y>2</y>
        </ns2:division>
    </S:Body>
</S:Envelope>
```

**SOAP Response**

```
<?xml version="1.0" encoding="UTF-8"?><S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/" xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
    <SOAP-ENV:Header/>
    <S:Body>
        <ns2:divisionResponse xmlns:ns2="http://p3/">
            <return>25</return>
        </ns2:divisionResponse>
    </S:Body>
</S:Envelope>
```

## addition Method invocation

**Method parameter(s)**

| Type | Value |
|---|---|
| int | 34 |
| int | 23 |

**Method returned**

int : "**57**"

**SOAP Request**

```
<?xml version="1.0" encoding="UTF-8"?><S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/" xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
    <SOAP-ENV:Header/>
    <S:Body>
        <ns2:addition xmlns:ns2="http://p3/">
            <a>34</a>
            <b>23</b>
        </ns2:addition>
    </S:Body>
</S:Envelope>
```

**SOAP Response**

```
<?xml version="1.0" encoding="UTF-8"?><S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/" xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
    <SOAP-ENV:Header/>
    <S:Body>
        <ns2:additionResponse xmlns:ns2="http://p3/">
            <return>57</return>
        </ns2:additionResponse>
    </S:Body>
</S:Envelope>
```

## subtraction Method invocation

**Method parameter(s)**

| Type | Value |
|---|---|
| int | 55 |
| int | 23 |

**Method returned**

int : "**32**"

**SOAP Request**

```
<?xml version="1.0" encoding="UTF-8"?><S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/" xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
    <SOAP-ENV:Header/>
    <S:Body>
        <ns2:subtraction xmlns:ns2="http://p3/">
            <x>55</x>
            <y>23</y>
        </ns2:subtraction>
    </S:Body>
</S:Envelope>
```

**SOAP Response**

```
<?xml version="1.0" encoding="UTF-8"?><S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/" xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
    <SOAP-ENV:Header/>
    <S:Body>
        <ns2:subtractionResponse xmlns:ns2="http://p3/">
            <return>32</return>
        </ns2:subtractionResponse>
    </S:Body>
</S:Envelope>
```

## multiplication Method invocation

**Method parameter(s)**

| Type | Value |
|---|---|
| int | 34 |
| int | 3 |

**Method returned**

int : "**102**"

**SOAP Request**

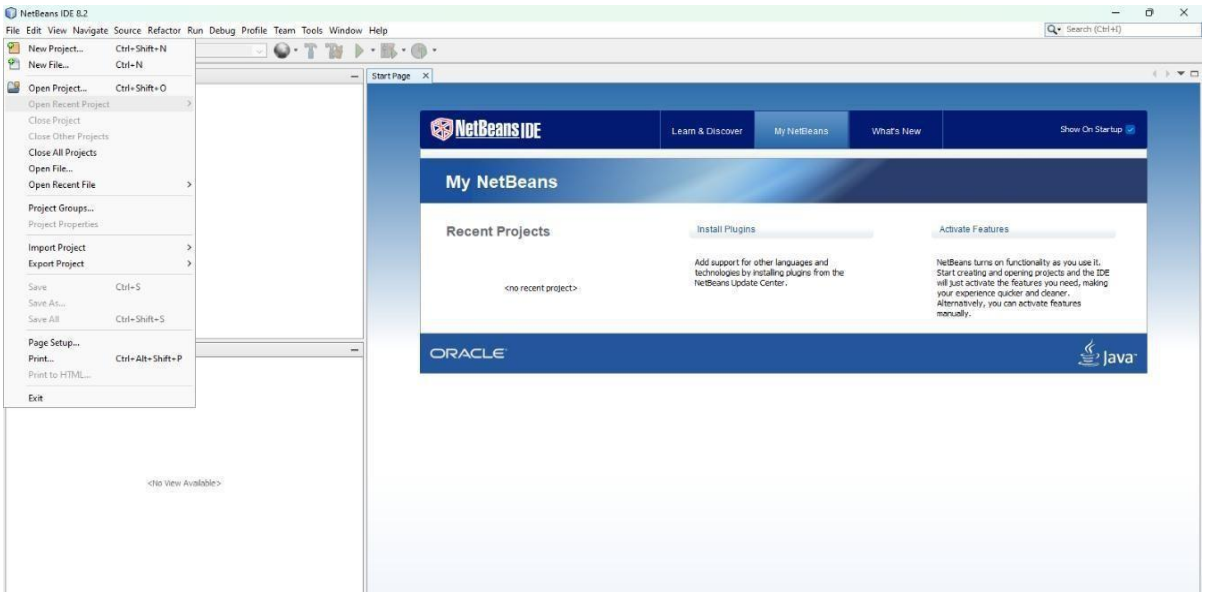
```
<?xml version="1.0" encoding="UTF-8"?><S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/" xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
    <SOAP-ENV:Header/>
    <S:Body>
        <ns2:multiplication xmlns:ns2="http://p3/">
            <a>34</a>
            <b>3</b>
        </ns2:multiplication>
    </S:Body>
</S:Envelope>
```

**SOAP Response**

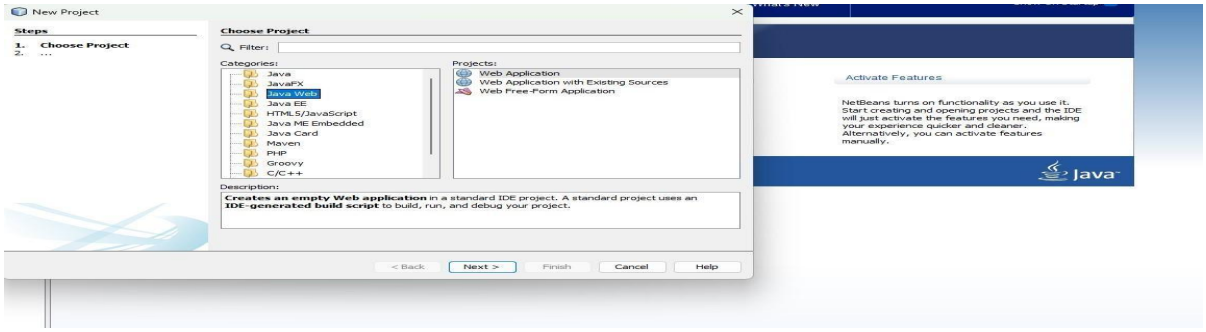
```
<?xml version="1.0" encoding="UTF-8"?><S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/" xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
    <SOAP-ENV:Header/>
    <S:Body>
        <ns2:multiplicationResponse xmlns:ns2="http://p3/">
            <return>102</return>
        </ns2:multiplicationResponse>
    </S:Body>
</S:Envelope>
```

## Creating a Client Application.

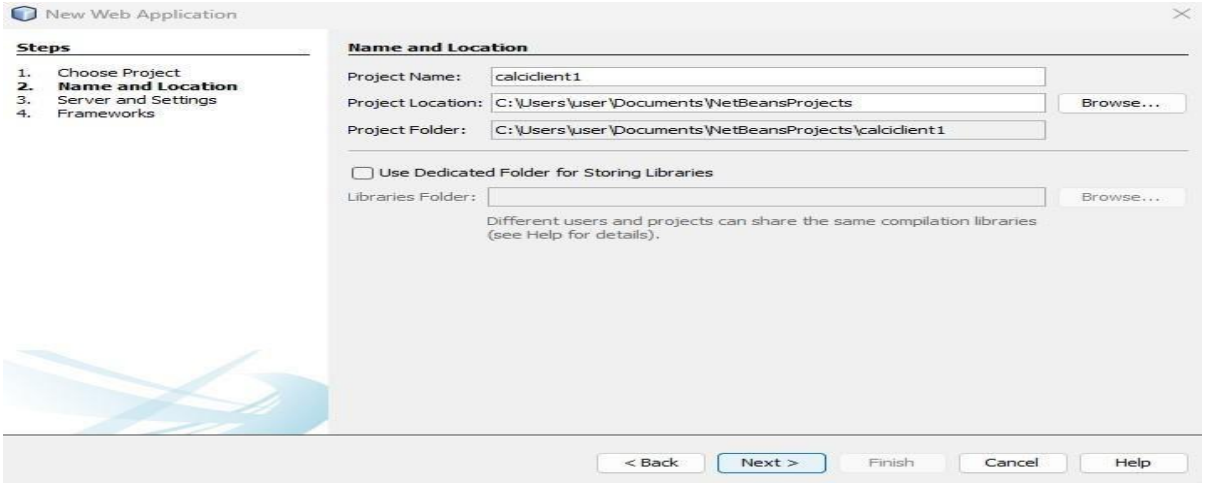
**Step 1:** Install NetBeans IDE , now go to file and click new project.



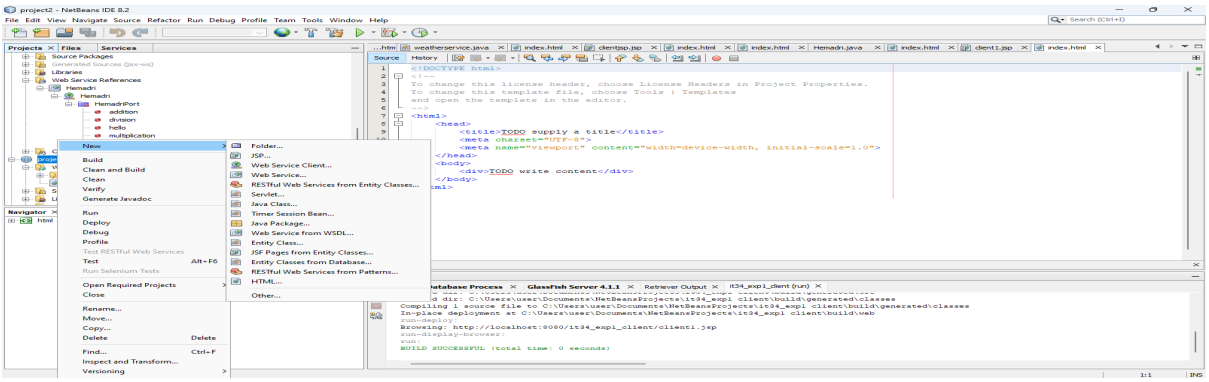

**Step 2:** Select Java web, Choose web application, Click next.



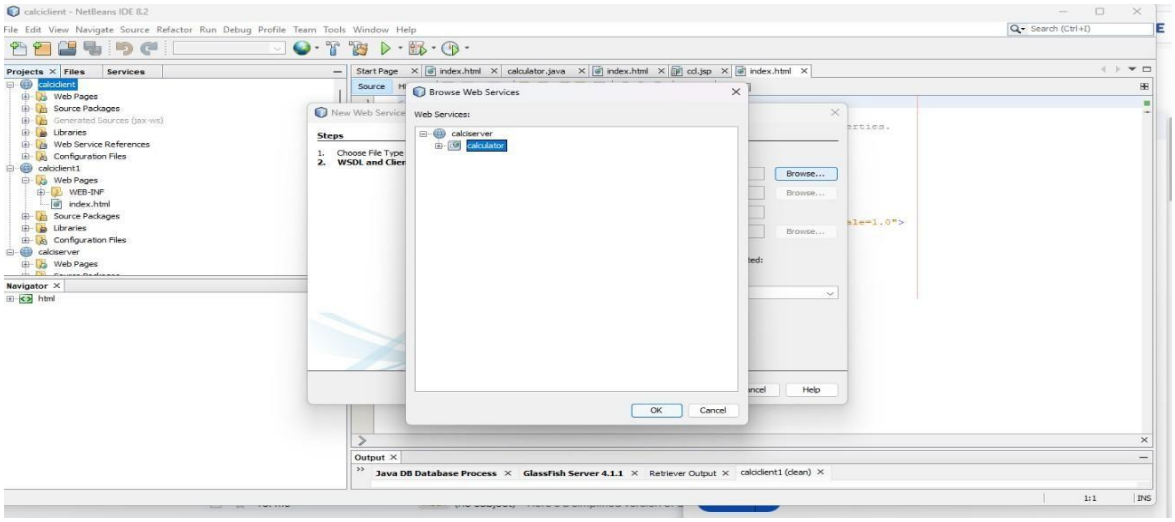

**Step 3 :** Assign a name for your project, Click next.



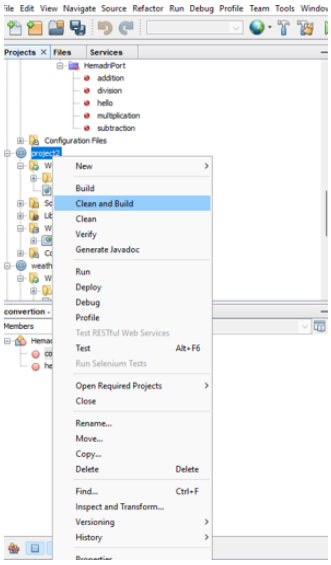
**Step 4:** Right click the client project, select new web service from the client(If webservice is not listed, click other, choose web services from the category, then choose web service client from file type, click next).



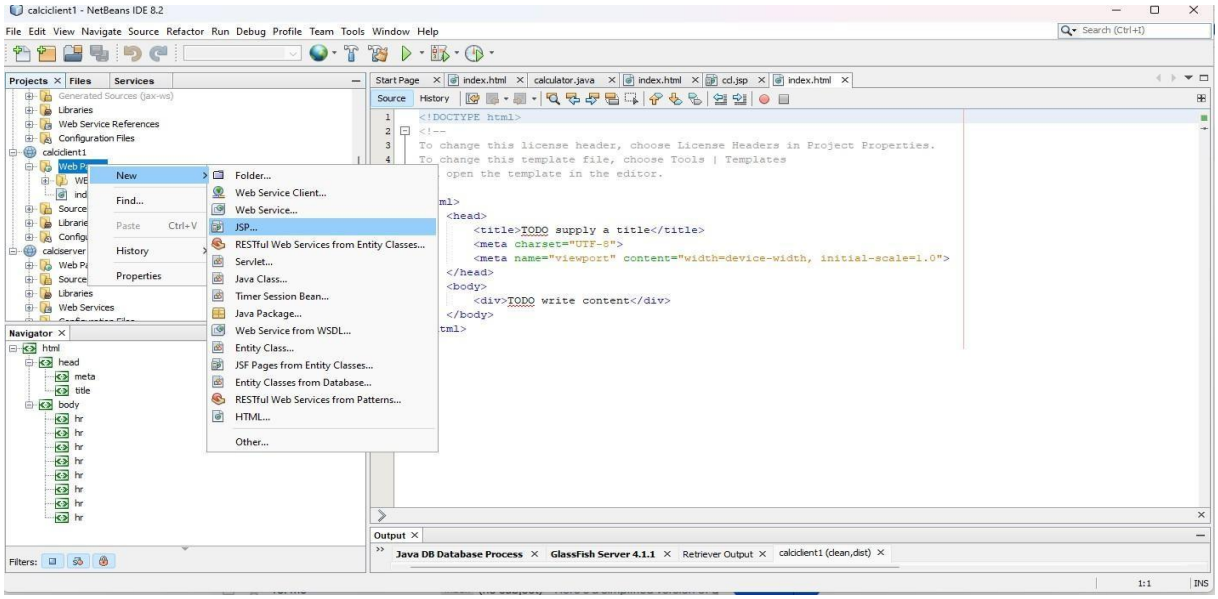
**Step 5:** Next browse for the project you have created and choose the web service that you have created in the service side, click ok and then click finish.



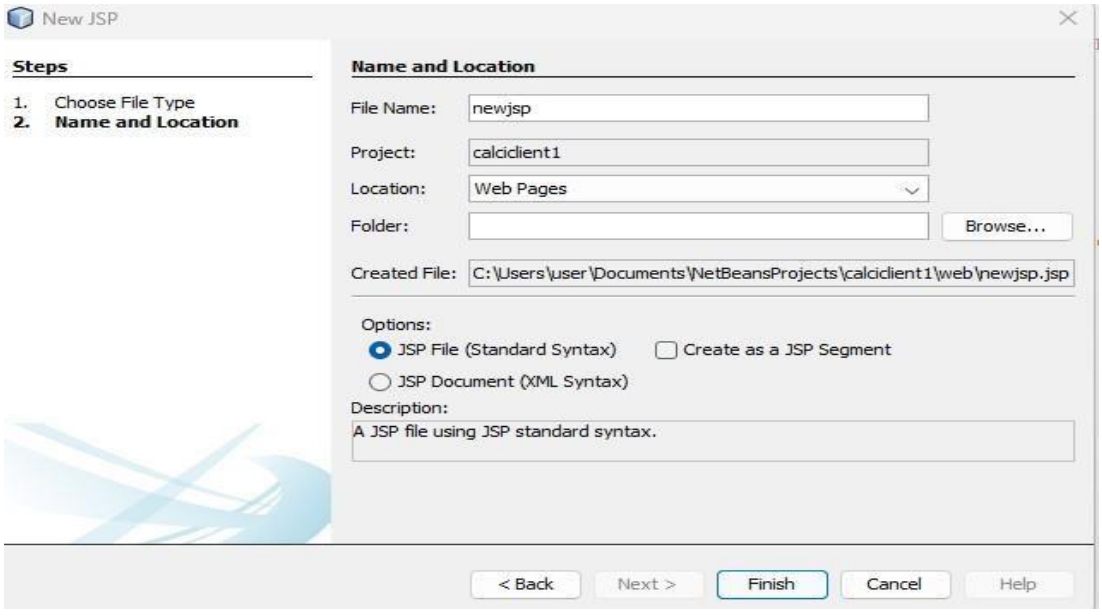
**Step 6:** Right click the project, Clean and build it.



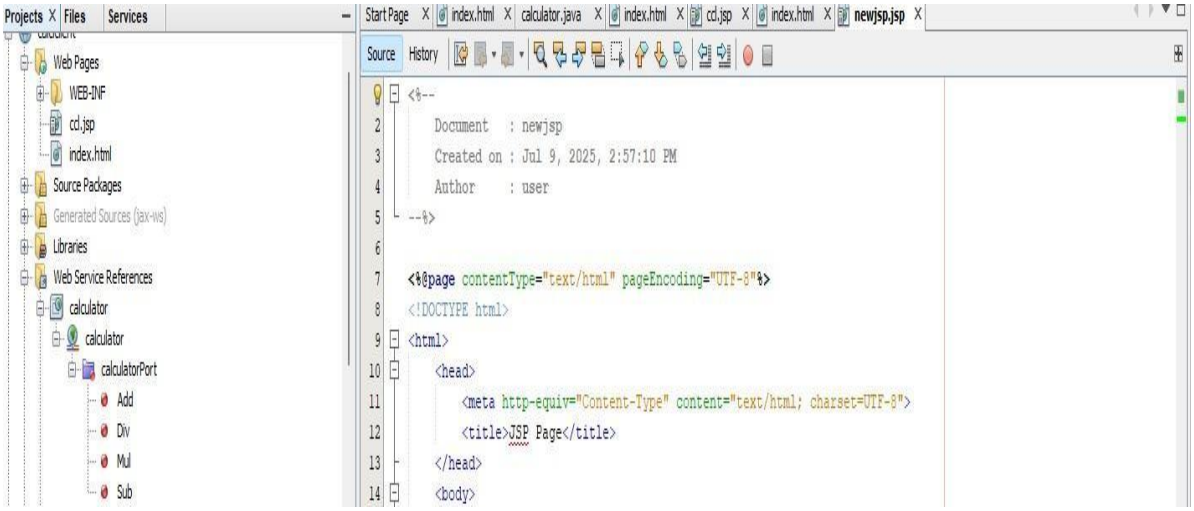
**Step 7:** Expand the client project, choose web pages, right click and select JSP.



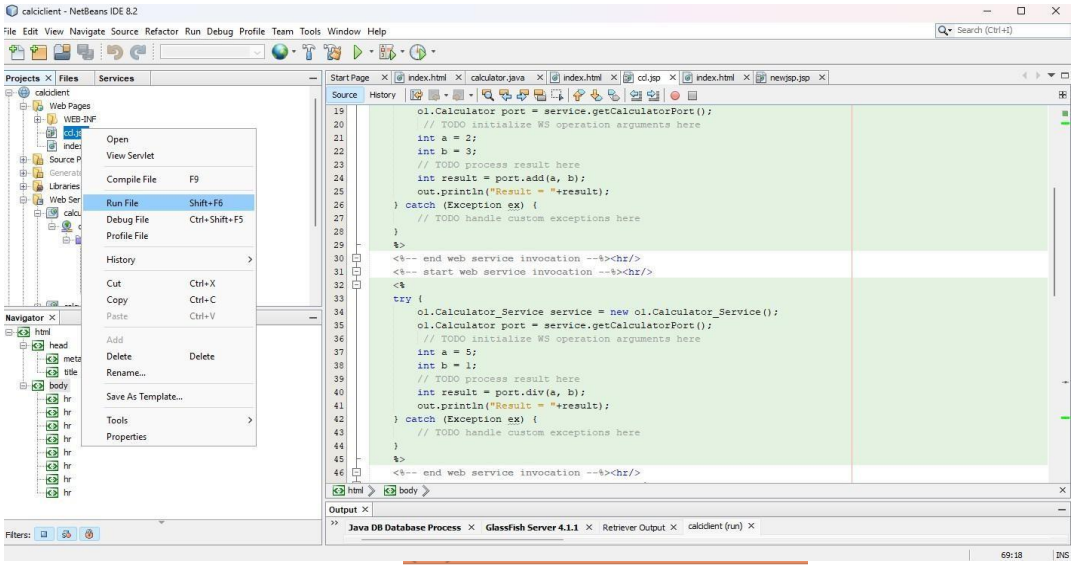

**Step 8:** Assign a name for your JSP file and click finish.




**Step 9:** Delete the content within the body tag in JSP file.




**Step 10 :** Expand the Web Service References, drag and drop the arithmetic operation interfaces inside the <body> tag, change the values of the parameters, then right-click the JSP file and select Run.





**RESULT :** Thus, development of a new webservice calculator for designing a calculator using NetBeans IDE is implemented successfully.

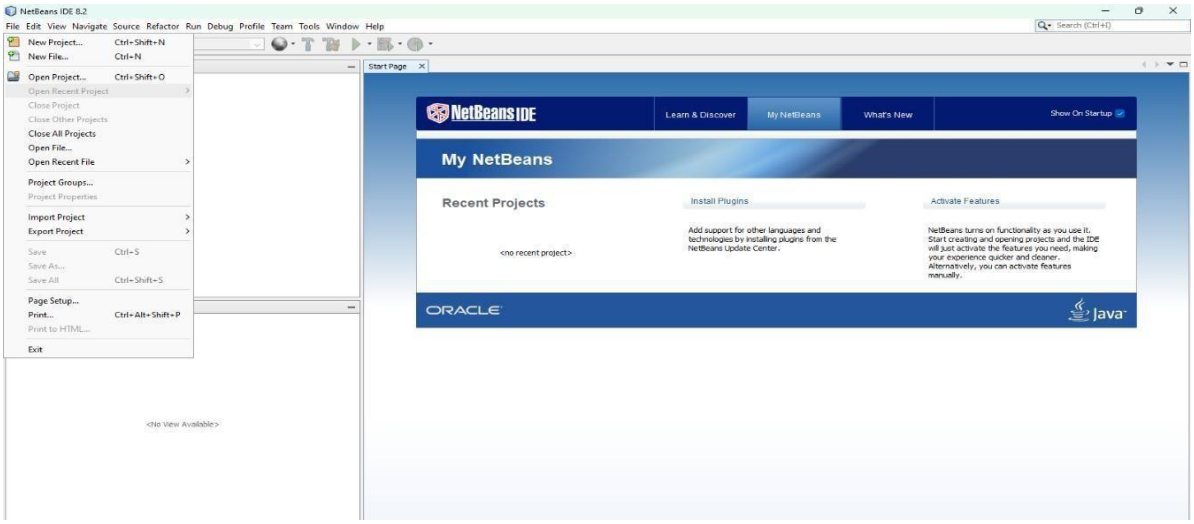
**EXP NO: 1 B** **DEVELOP A NEW WEBSERVICE FOR CONVERTING**
**DATE:** **TEMPERATURE FROM FAHRENHEIT TO CELSIUS**

**AIM:** To develop a new webservice for converting temperature from Fahrenheit to Celsius using NetBeans IDE.

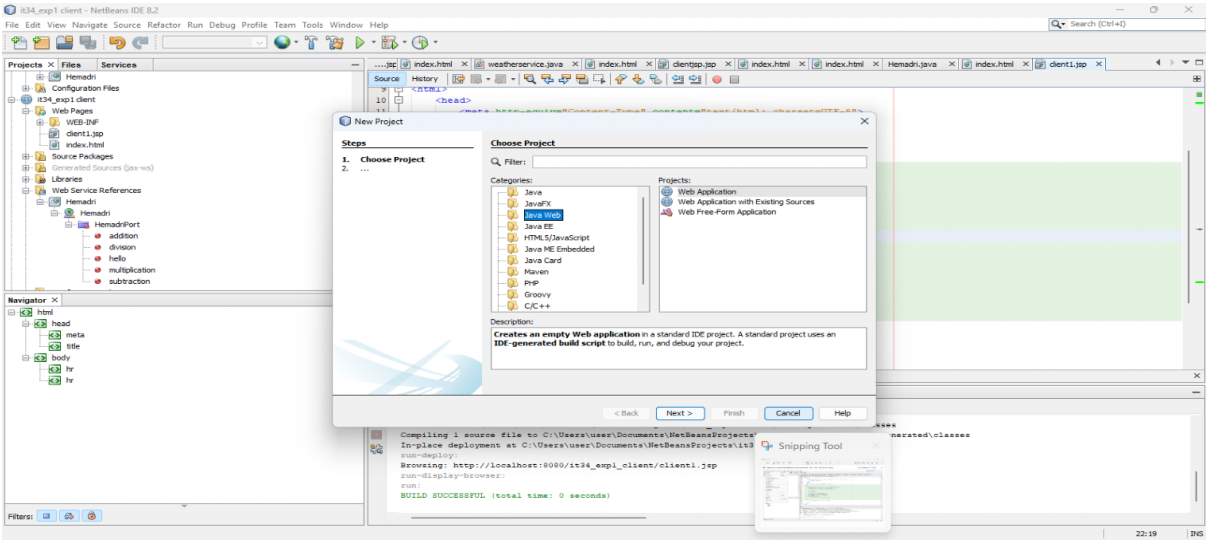
**PROCEDURE:**

**Creating Web Service – (Fahrenheit to Celsius).**

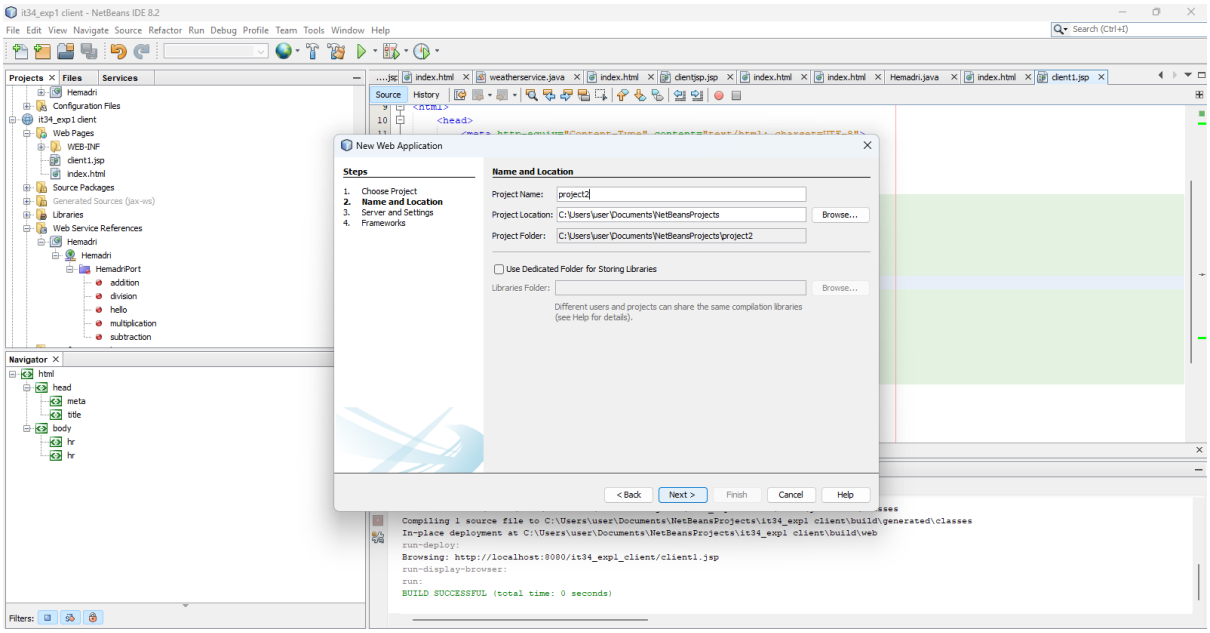
**Step 1:** Install NetBeans IDE , now go to file and click new project.



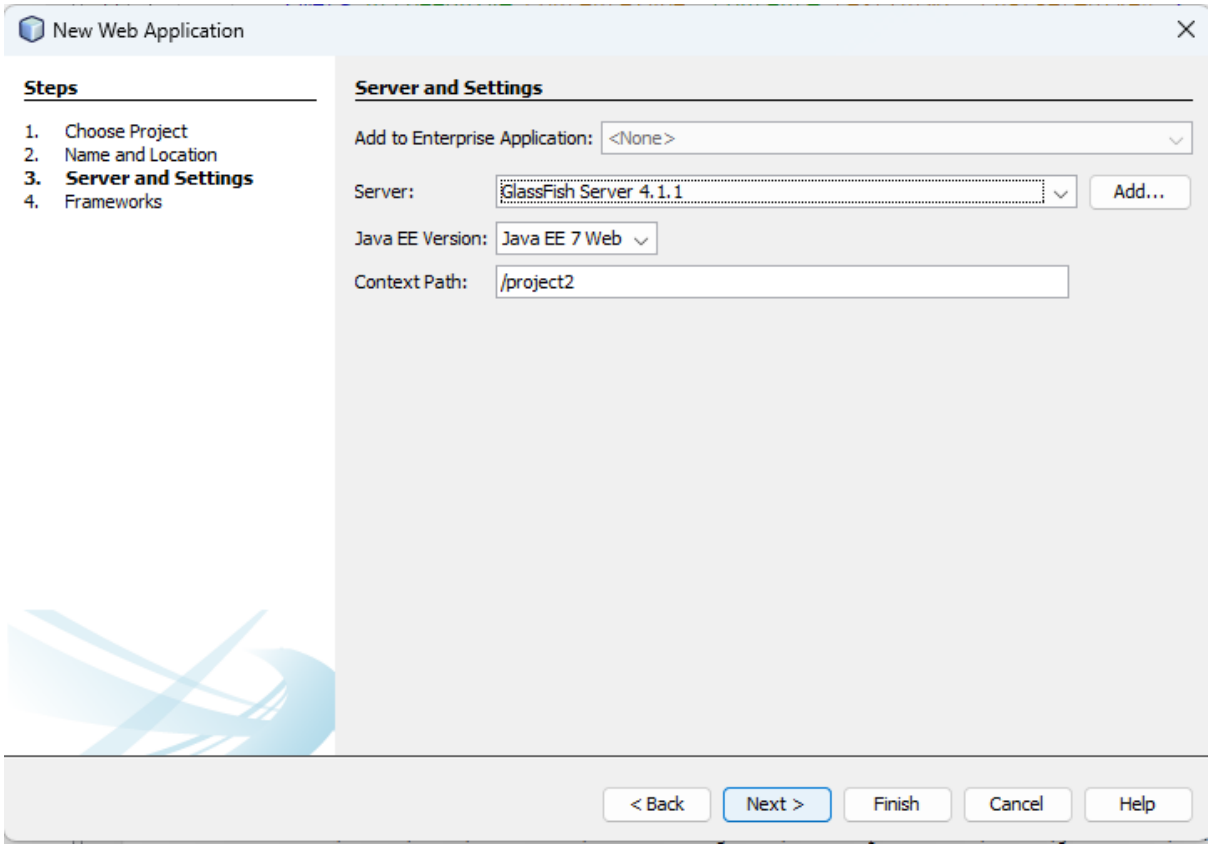

**Step 2:** Select Java Web Application , click finish and next.

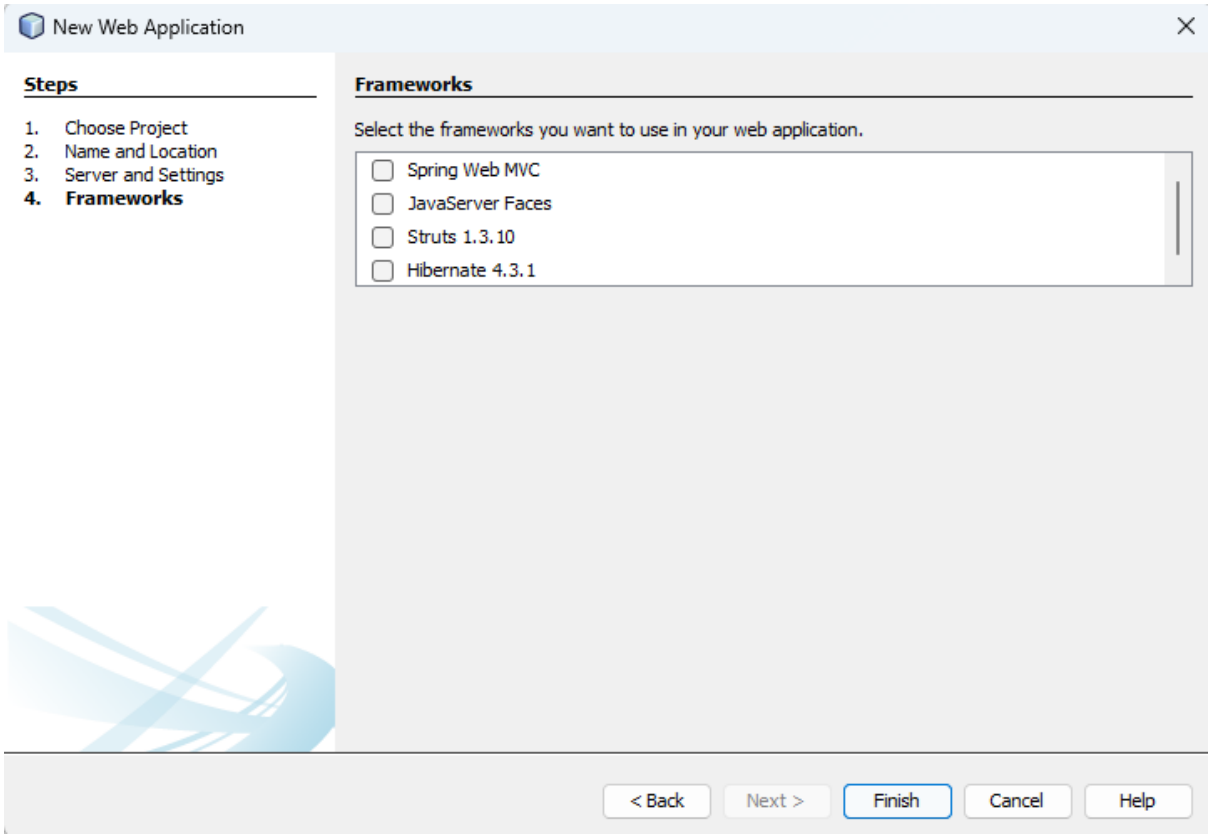



**Step 3:** Give the project name as TemperatureConverterService and click Next.Then, right-click the TemperatureConverterService project, select New → Web Service, and create a web service.

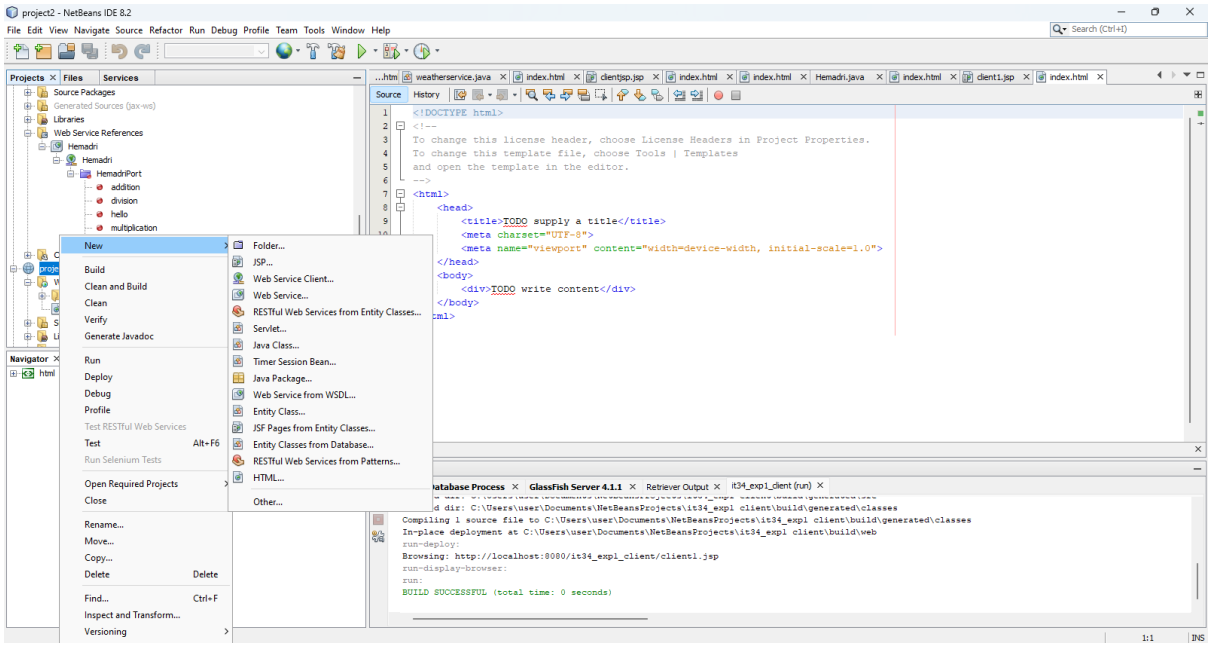


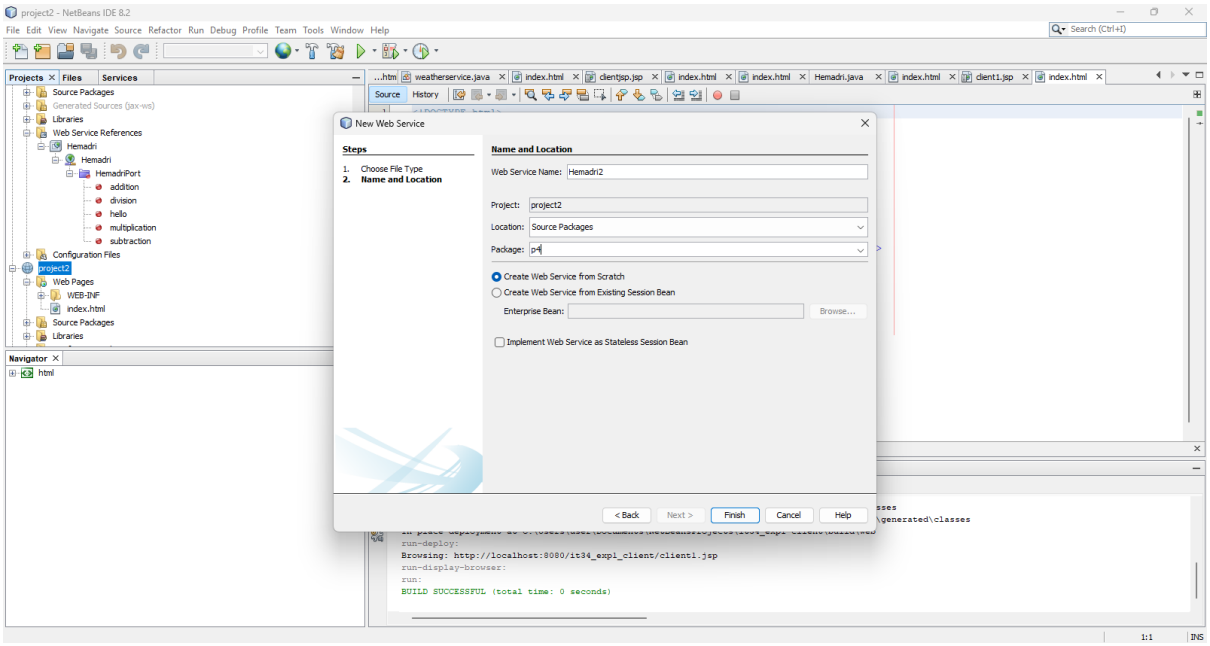

**Step 4:** In the Server and Settings window, ensure that the Server Name is set to GlassFish Server and the Java EE Version is Java EE 7 Web. Then click Finish.



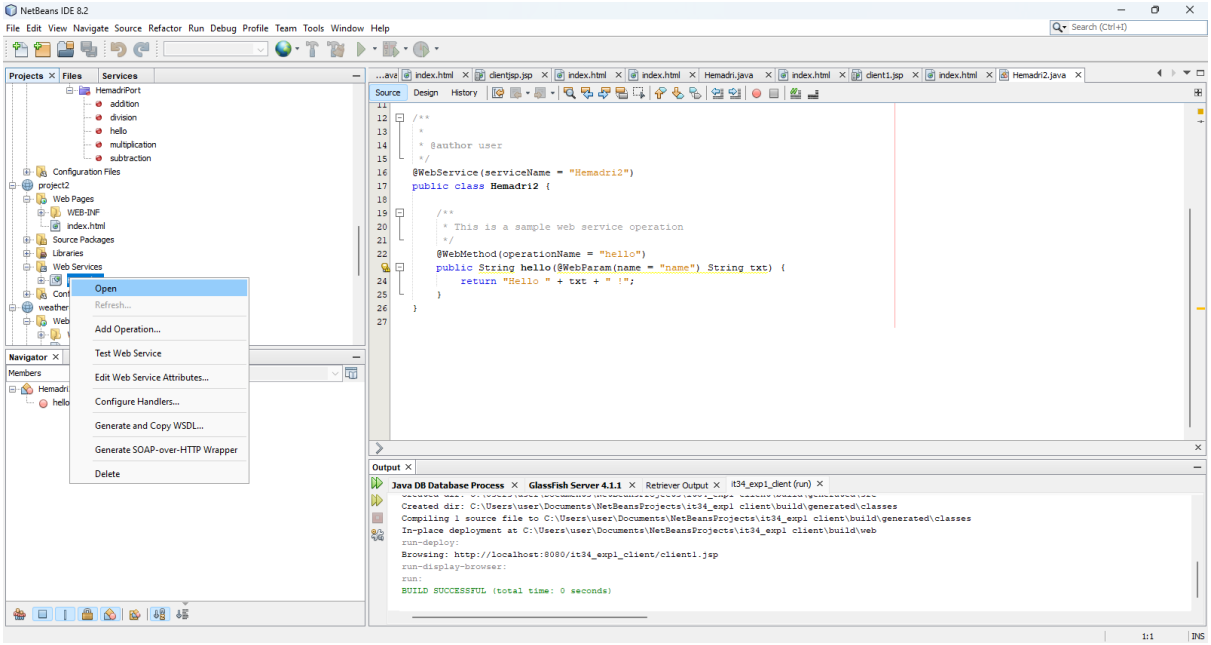



**Step 5 :** Right-click the TemperatureConverterService project, select New → Web Service. Enter the Web Service Name as TemperatureService, assign the package name (e.g., com.temperature), and click Finish.

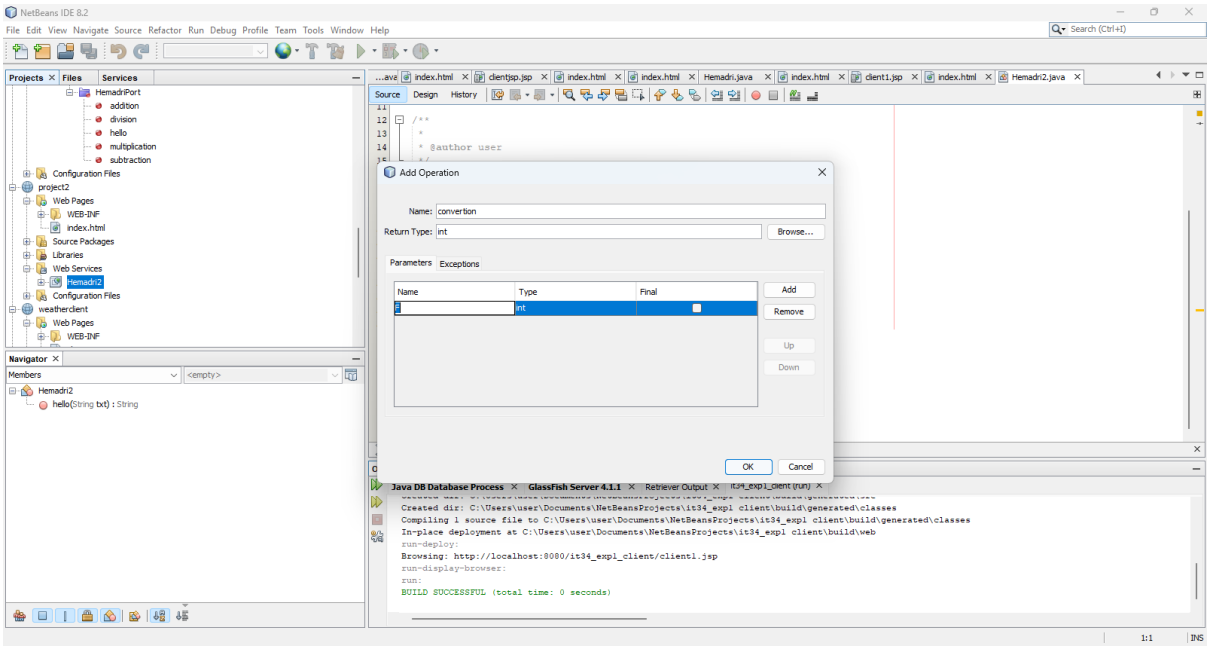




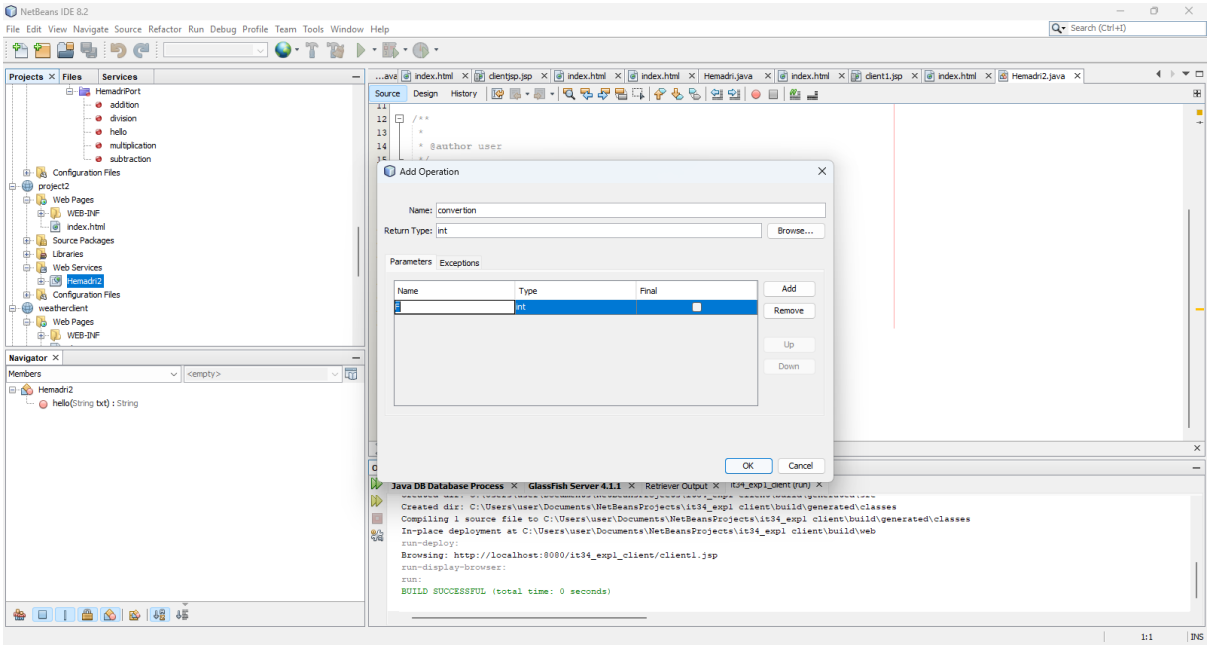

**Step 6:** Expand the Web Service folder, right-click the TemperatureService web service, and select Add Operation.



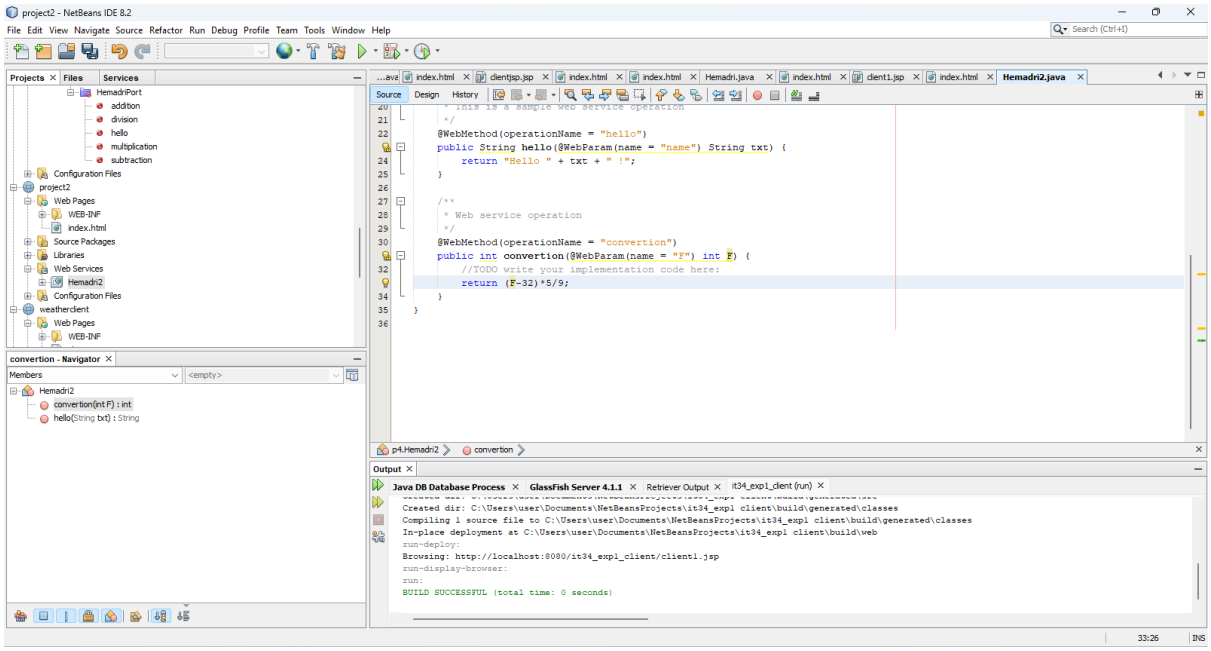

**Step 7:** Give the operation name as fahrenheitToCelsius and choose the Return Type as Double (double).



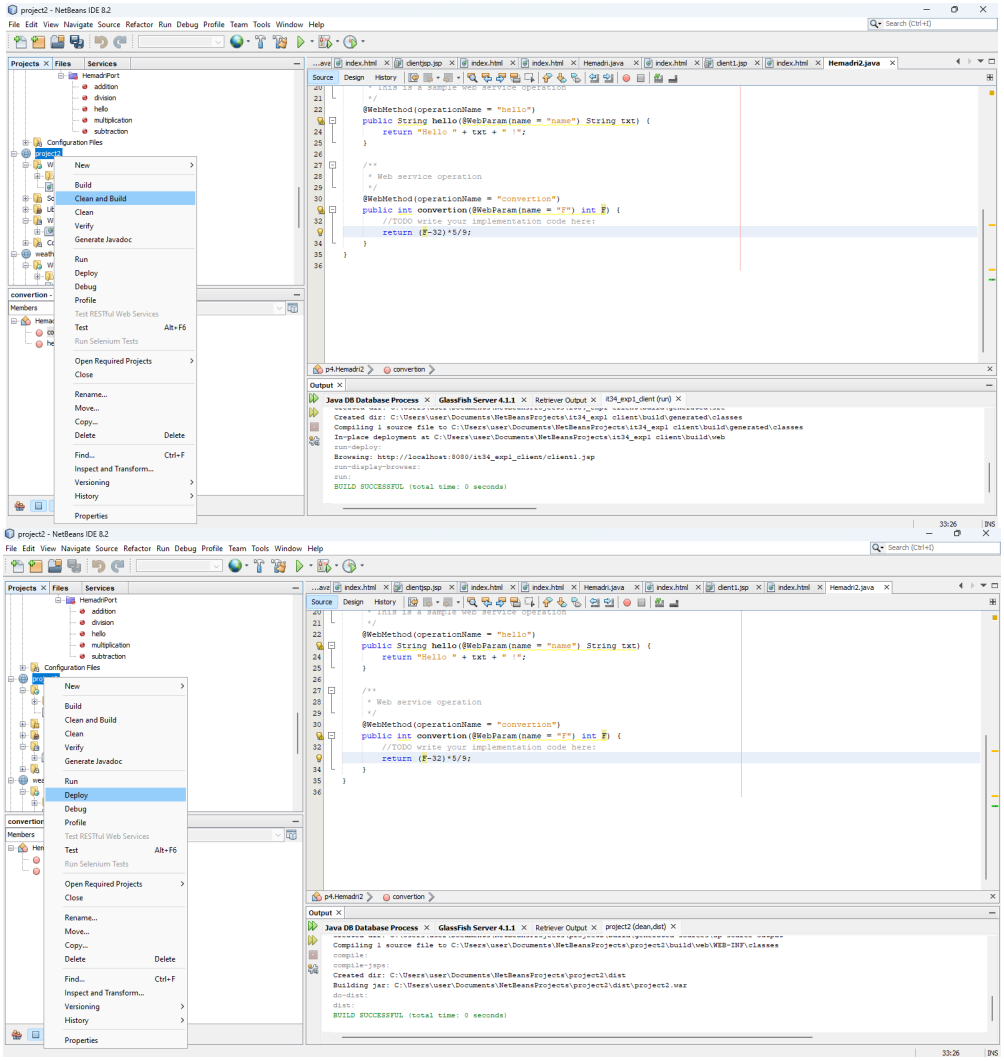

**Step 8 :** Click on the Add button, enter the parameter name as fahrenheit, choose the Data Type as Double (double), and click OK.



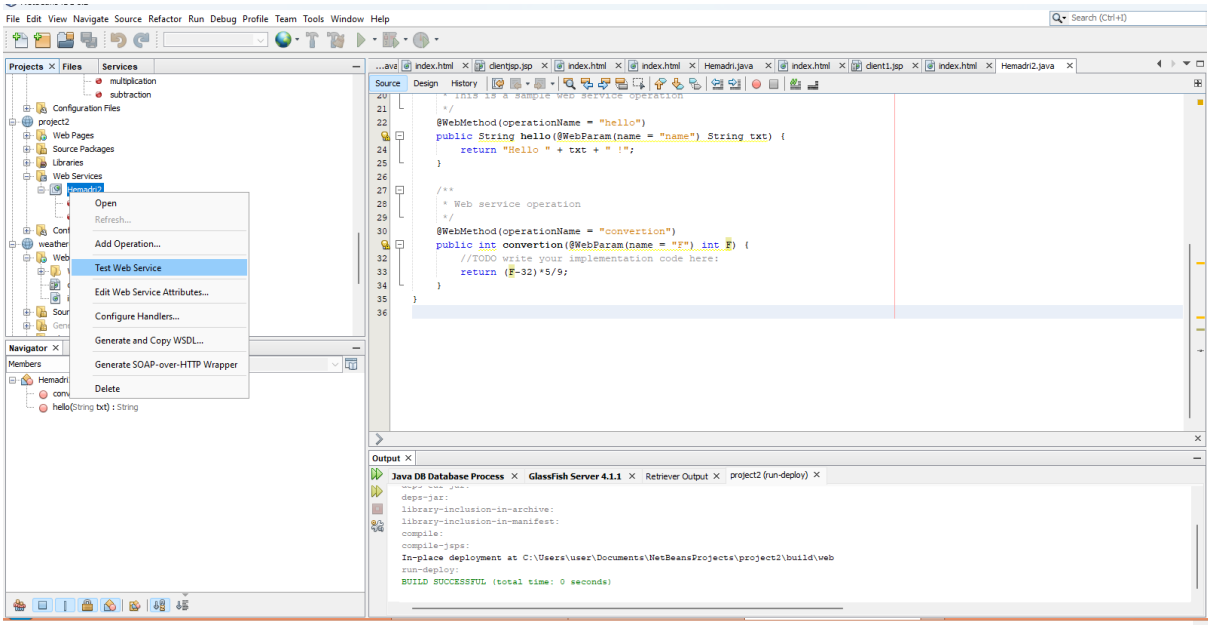

**Step 9 :** In the web method code, replace return 0; with the Fahrenheit to Celsius conversion formula: return (fahrenheit - 32) * 5 / 9;



**Step 10:** Right-click the TemperatureConverterService project and select Clean and Build. After the build is completed successfully, right-click the project again and select Deploy.



**Step 11 :** Right-click the TemperatureService web service and select Test Web Service to test the Fahrenheit to Celsius conversion.




**ftoC Method invocation**

---

**Method parameter(s)**

| Type | Value |
|---|---|
| int | 65 |

---

**Method returned**

float : "**18.0**"

---

**SOAP Request**

---

```
<?xml version="1.0" encoding="UTF-8"?><S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/" xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
    <SOAP-ENV:Header/>
    <S:Body>
        <ns2:FtoC xmlns:ns2="http://p/">
            <a>65</a>
        </ns2:FtoC>
    </S:Body>
</S:Envelope>
```

---

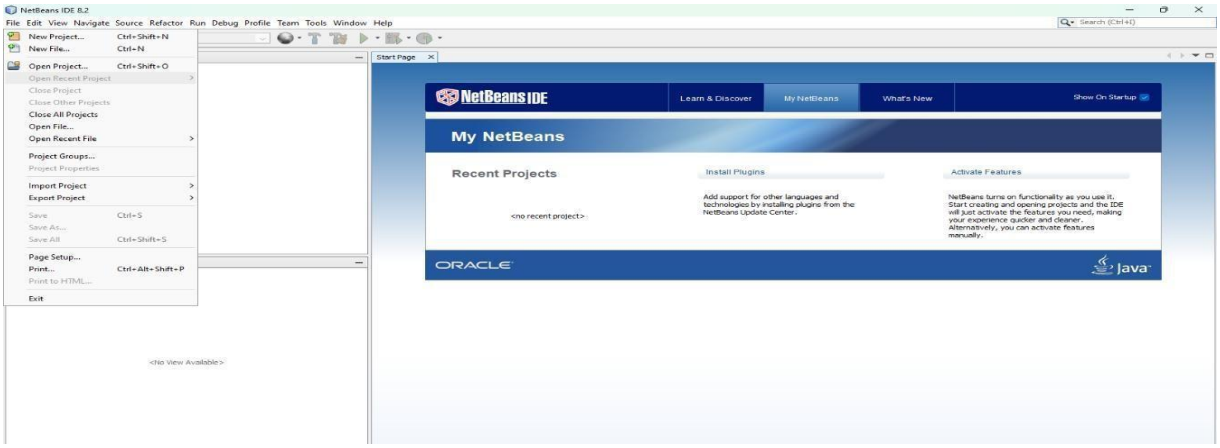
**SOAP Response**

---

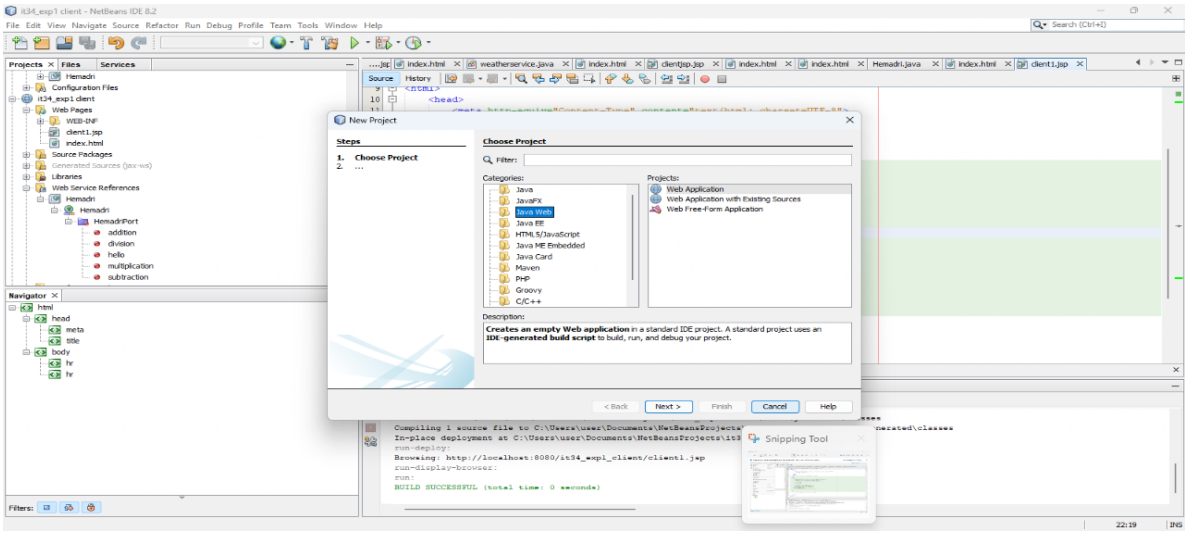
```
<?xml version="1.0" encoding="UTF-8"?><S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/" xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
    <SOAP-ENV:Header/>
```

## Creating a Client Application  Fahrenheit to Celsius

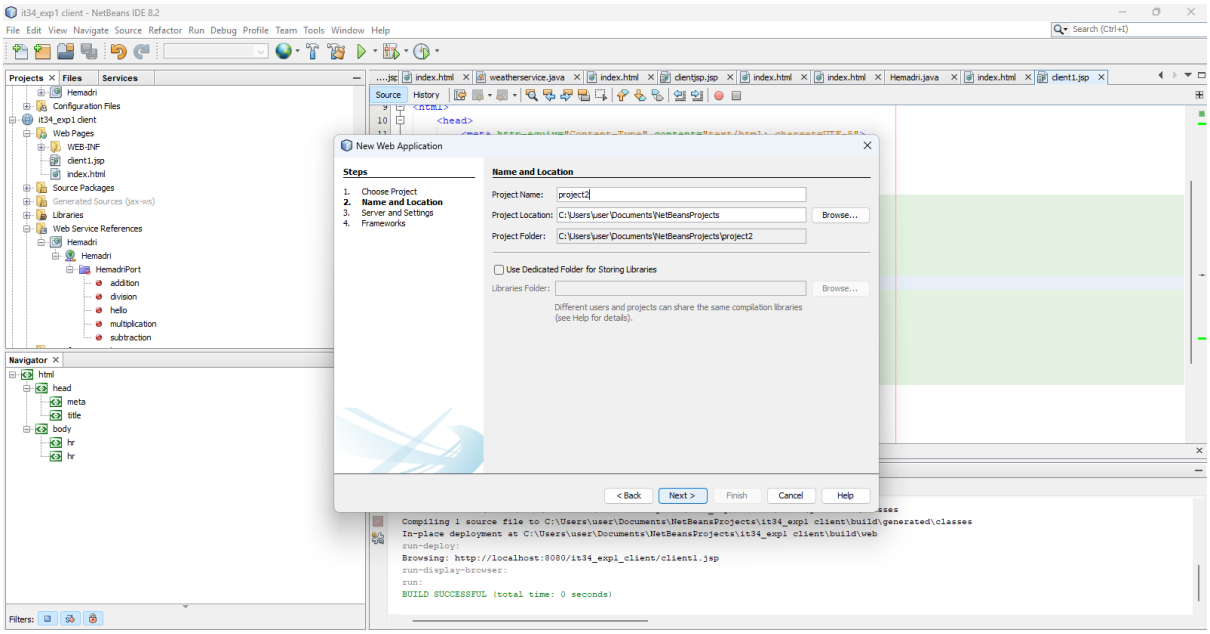
**Step 1 :** Install NetBeans IDE , now go to file and click new project.



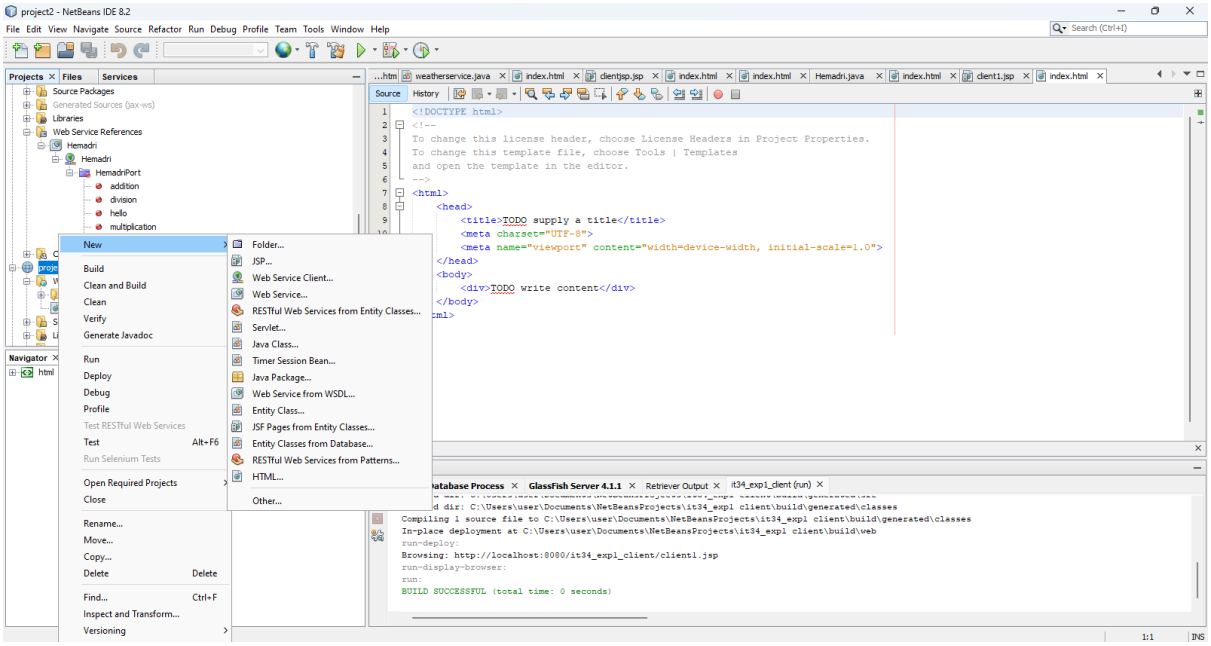

**Step 2 :** Select Java Web Application , click finish and next.



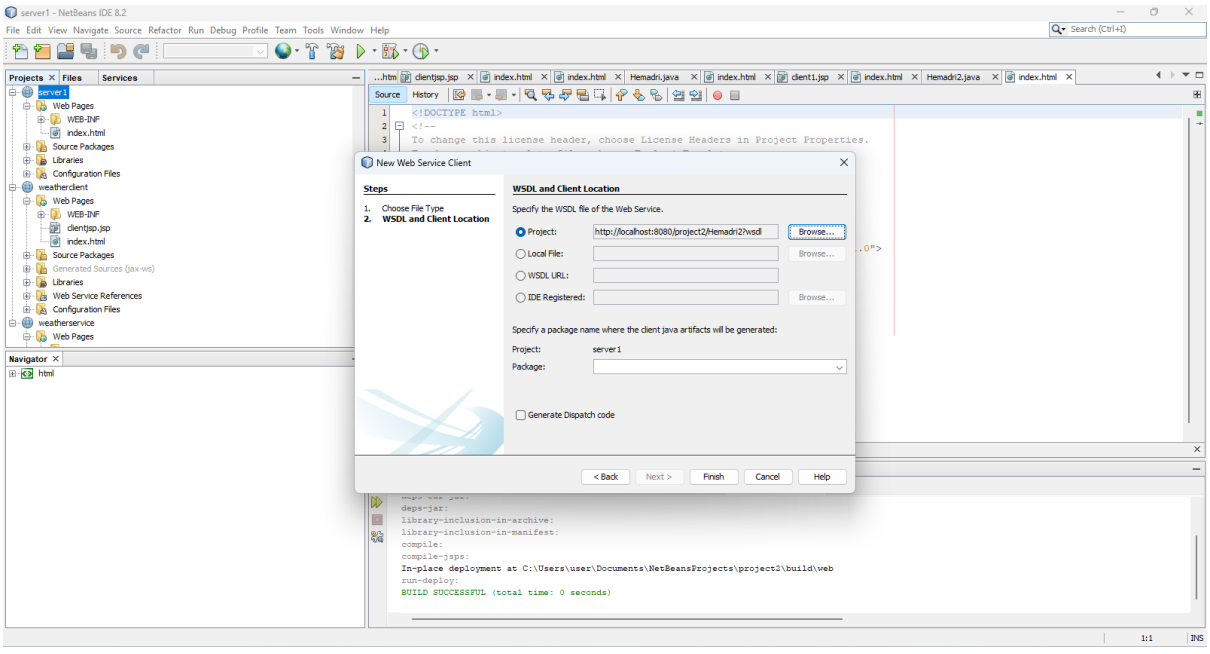

**Step 3 :** Assign a name for your client project as TemperatureConverterClient and click Next.



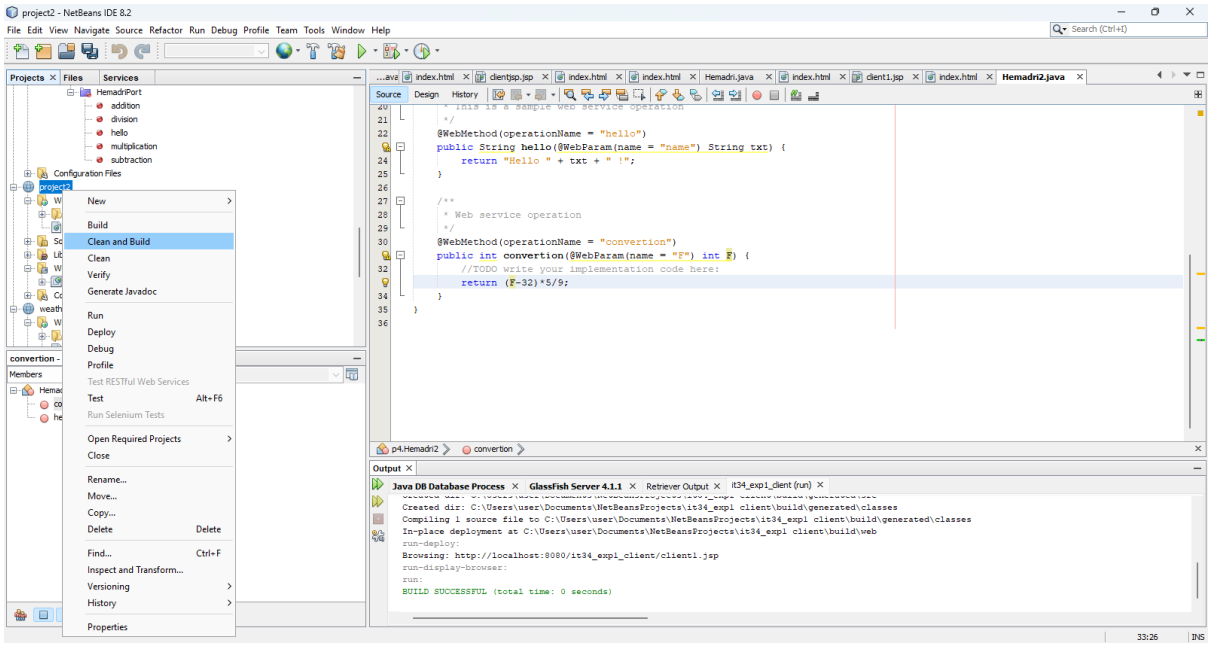
**Step 4 :** Right-click the TemperatureConverterClient project, select New → Web Service Client. If Web Service Client is not listed, click Other → Web Services → Web Service Client, then click Next.



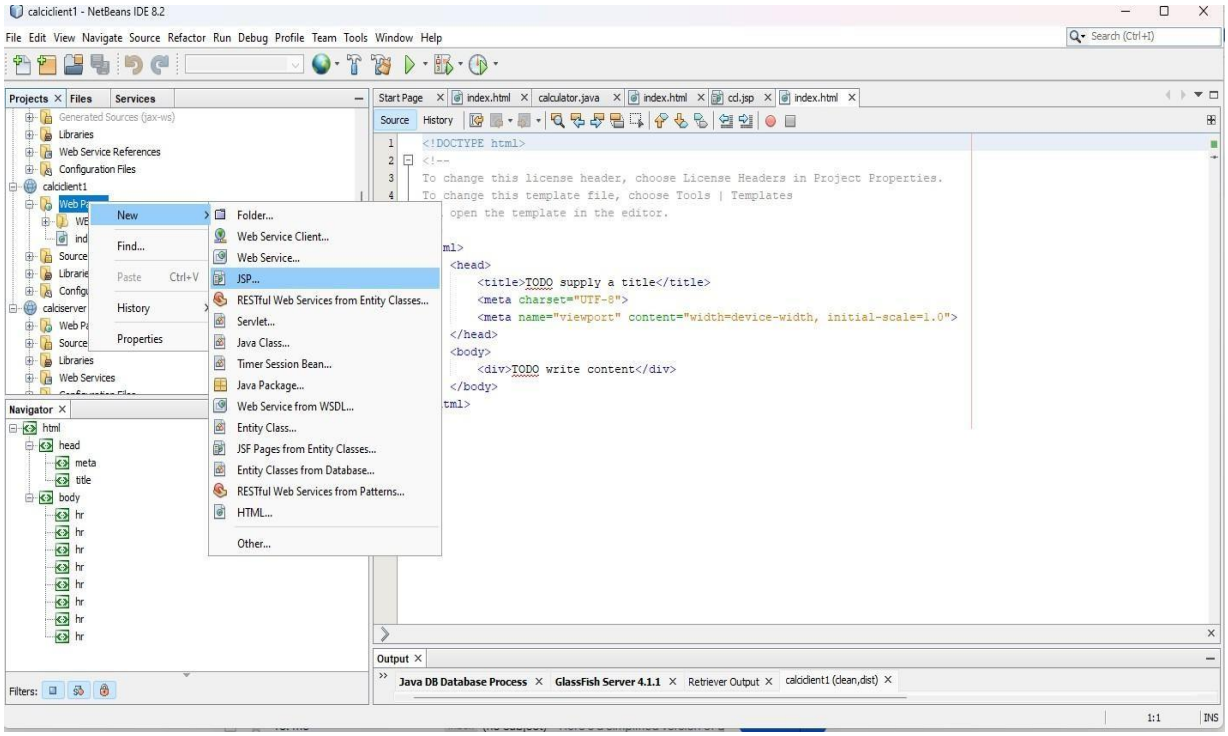

**Step 5 :** Browse and select the TemperatureService web service (WSDL) created in the service project. Click OK, then click Finish to create the Web Service Client.



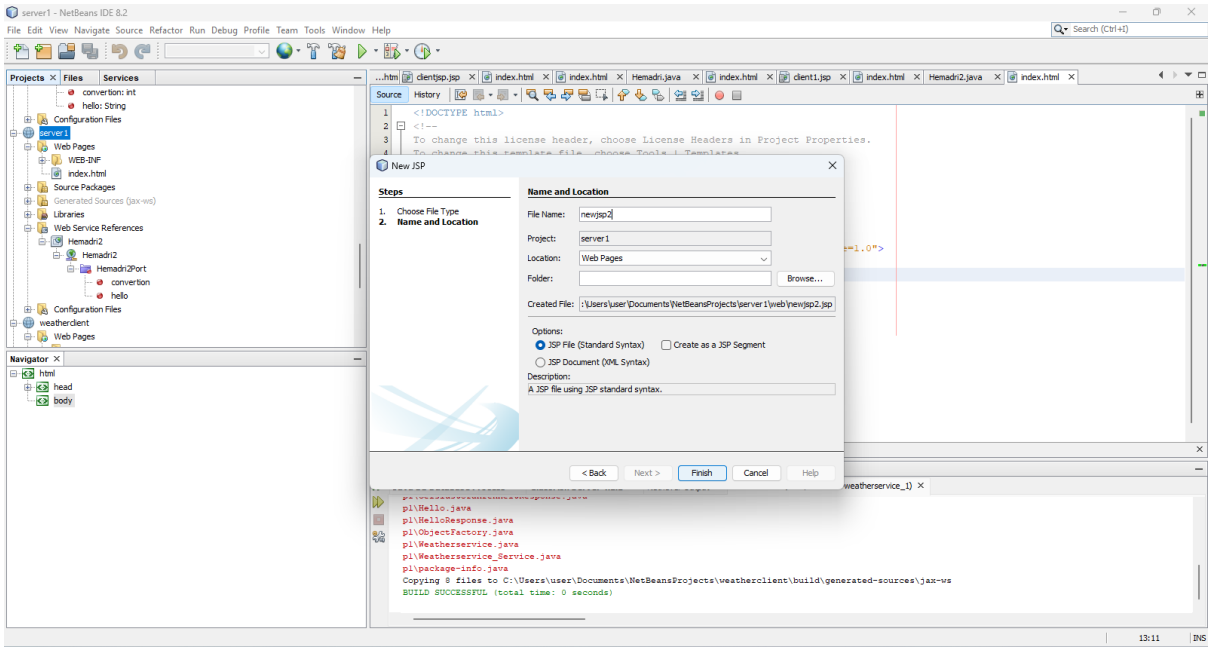

**Step 6 :** Right-click the TemperatureConverterClient project and select Clean and Build. After the build is completed successfully, run the client project.



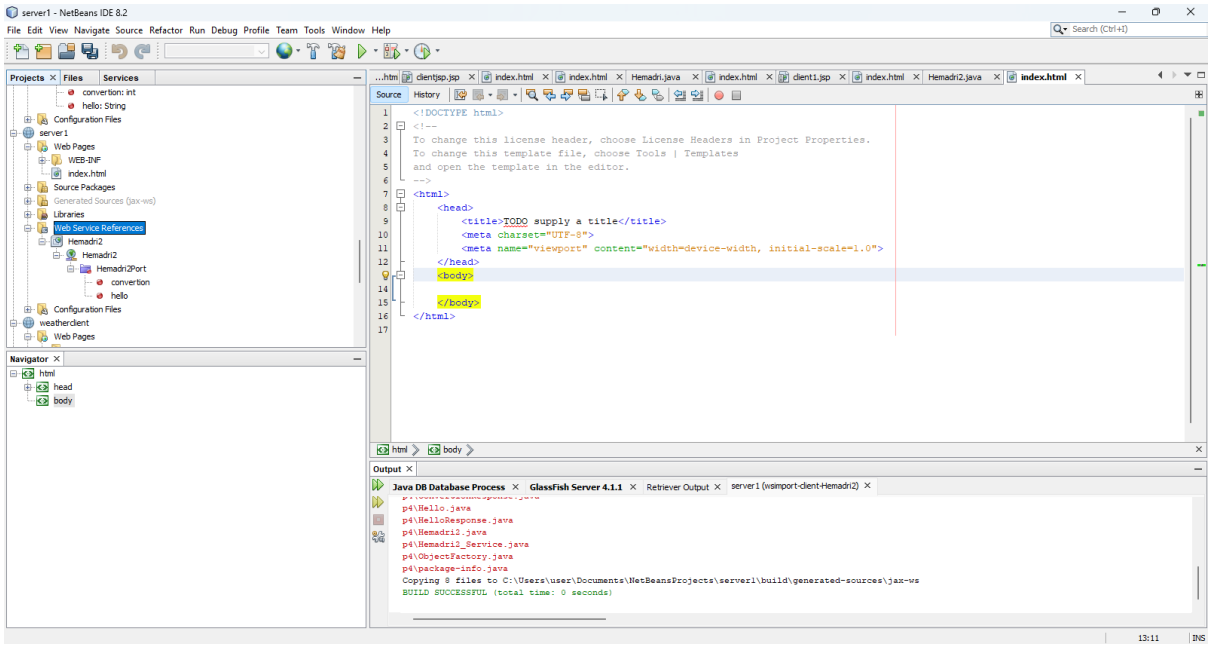

**Step 7 :** Expand the TemperatureConverterClient project, right-click Web Pages, select New → JSP, and create a new JSP file.



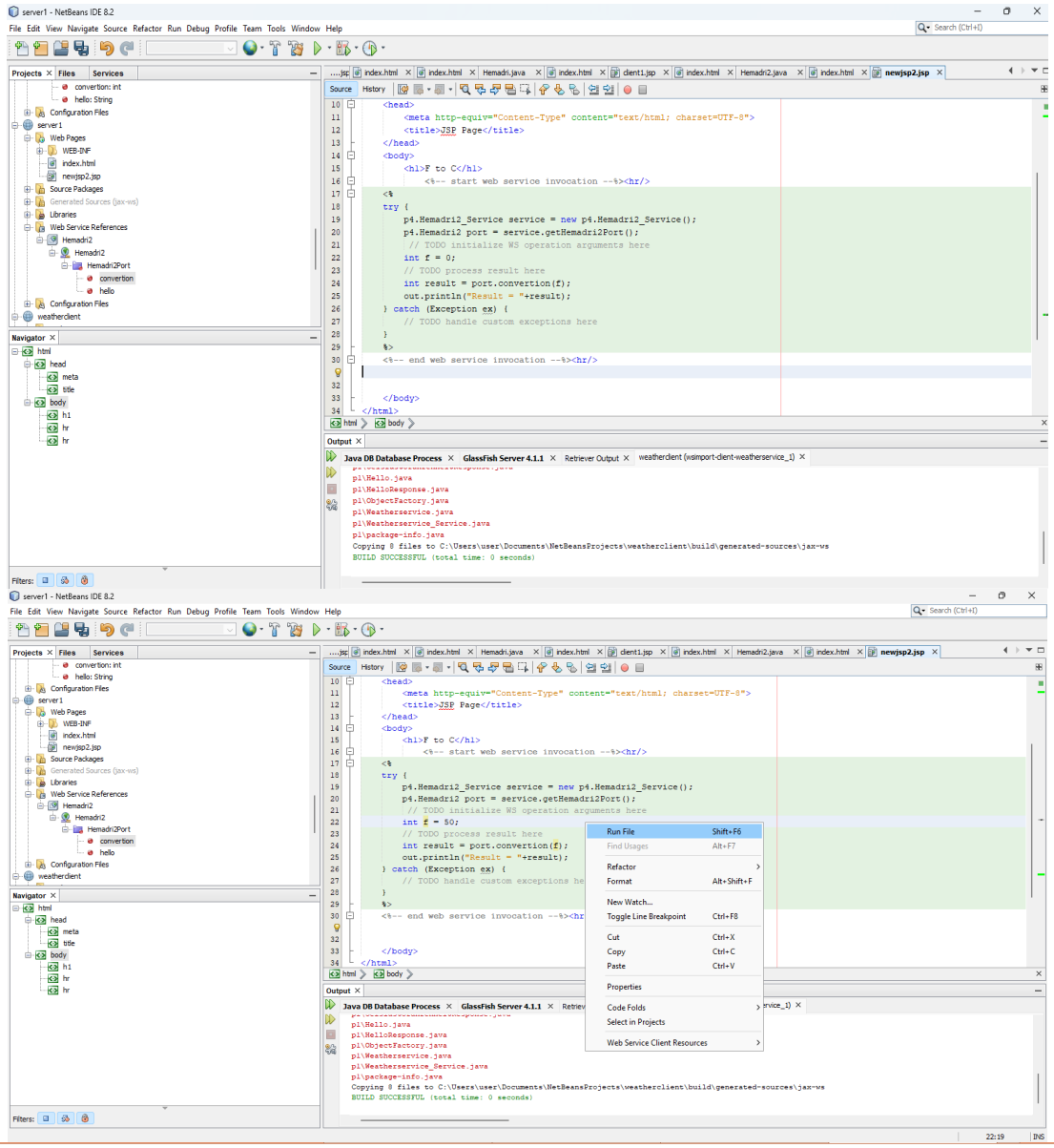

**Step 8 :** Assign a name for the JSP file (for example, index.jsp) and click Finish.




**Step 9 :** Open the JSP file and delete the default content inside the <body> tag.




**Step 10 :** Expand Web Service References, drag and drop the fahrenheitToCelsius operation into the <body> tag of the JSP file. Enter the Fahrenheit value, then right-click the JSP file and select Run to display the Celsius result.



**Hello World!**

Result = 11

Result = 3

Result = Hello John !

Result = 6

Result = 1

Result = 18.0

**RESULT :** Thus, the web service for converting temperature from Fahrenheit to Celsius using NetBeans IDE was designed, implemented, tested, and executed successfully.

**EXP NO: 2 SIMULATION OF CLOUD ENVIRONMENT USING CLOUDSIM**
**DATE:**

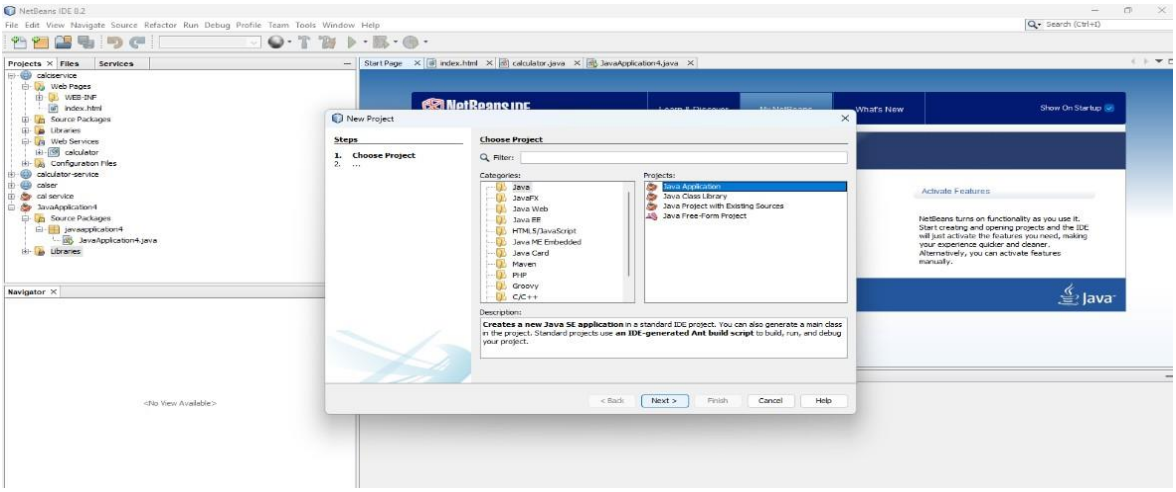
**AIM:** To simulate various cloud computing scenarios using the CloudSim toolkit by creating datacentres, virtual machines (VMs), and cloudlets, and to observe their execution in a cloud environment.

**DESCRIPTION:**

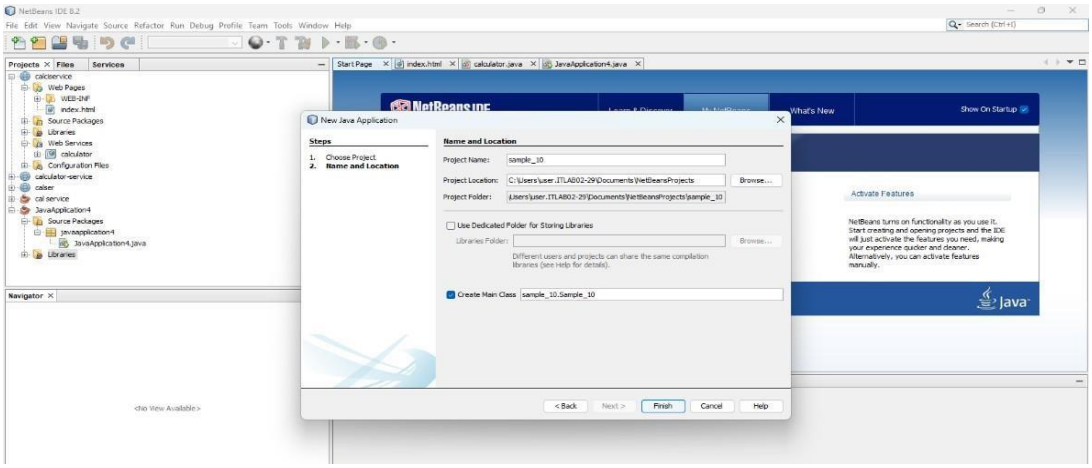
CloudSim is an open-source simulation toolkit used for modelling and simulating cloud computing infrastructures and services. It enables users to create and test cloud environments consisting of datacentres, hosts, virtual machines (VMs), brokers, and cloudlets without requiring actual cloud hardware. CloudSim is widely used for evaluating resource allocation policies, scheduling algorithms, VM provisioning, and cloud performance in a cost-effective and flexible manner.

**PROCEDURE:**

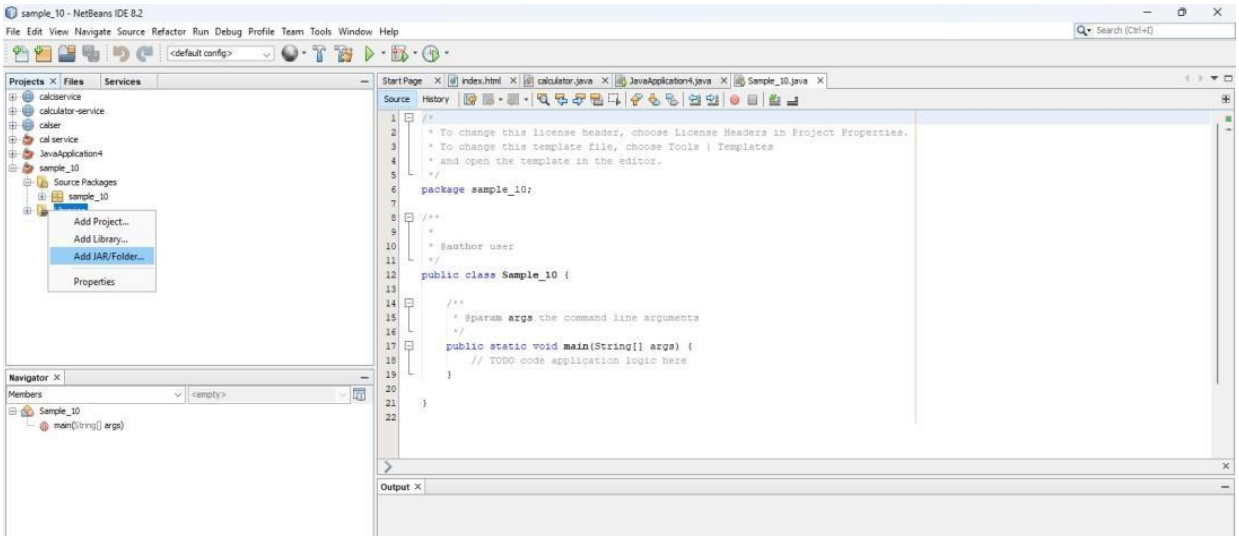
1. Open NetBeans IDE.
2. Select File → New Project → Java → Java Application.



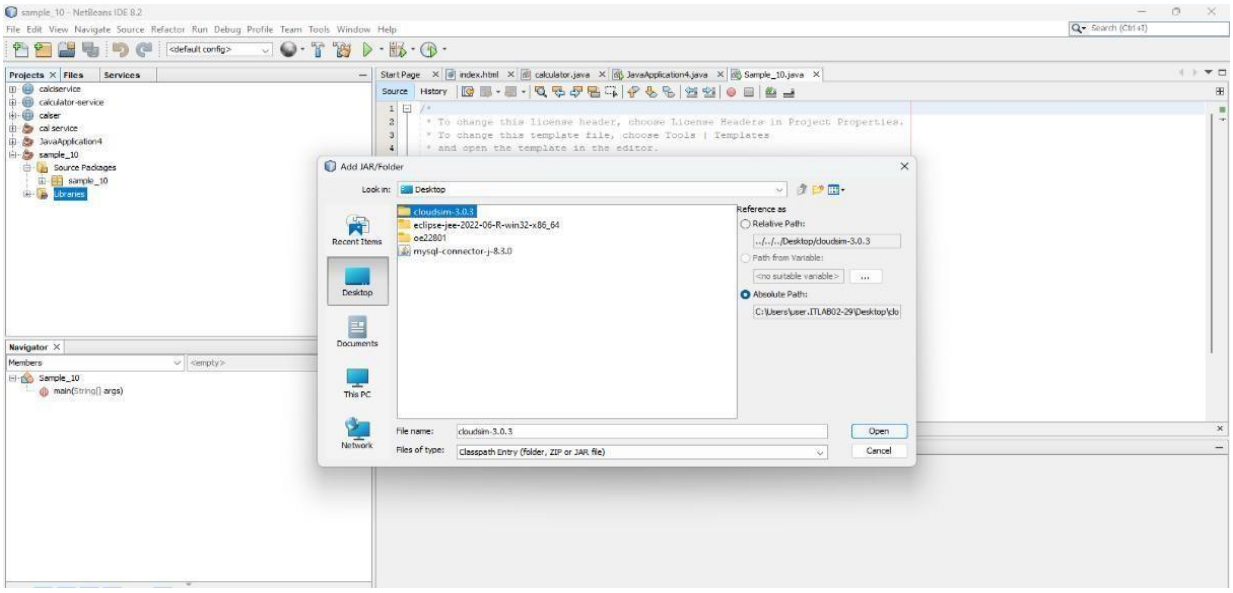

3. Enter the project name and click Finish.



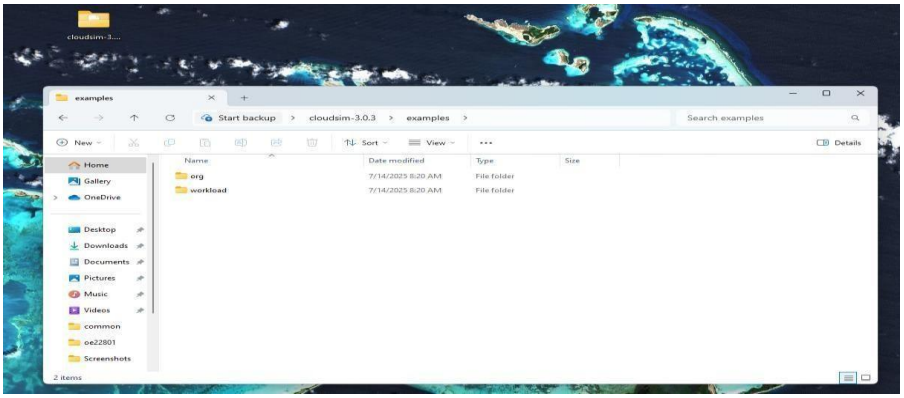
4. Expand the created project, right-click Libraries, and select Add JAR/Folder.

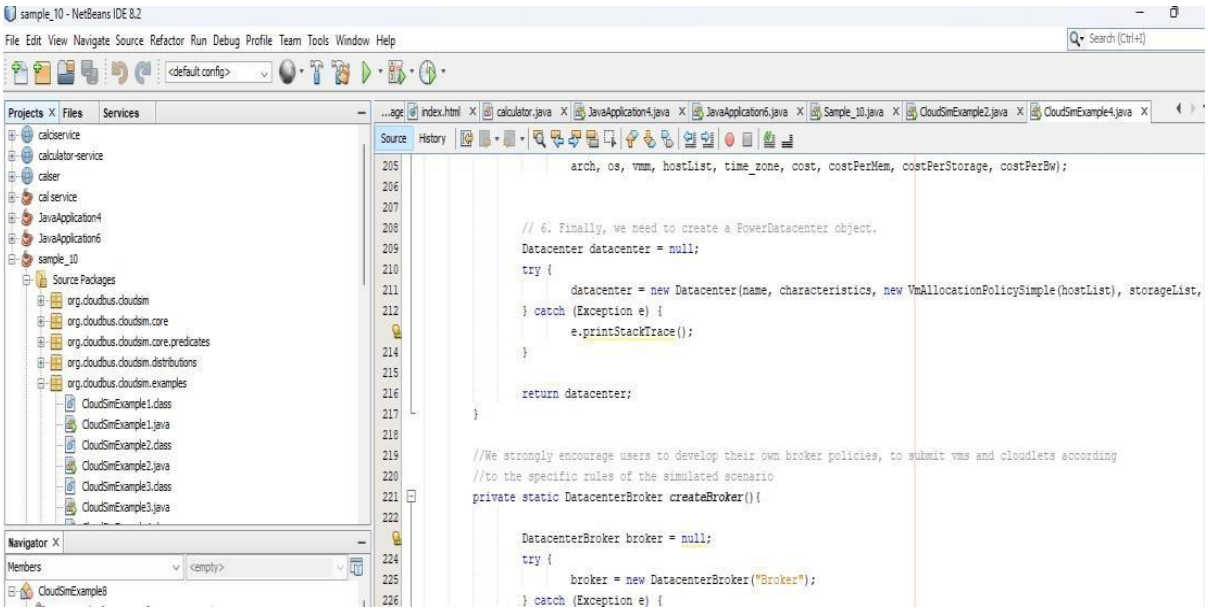



5. Browse to Desktop → CloudSim 3.0 → jars, select cloudsim-3.0.jar, and click Open.

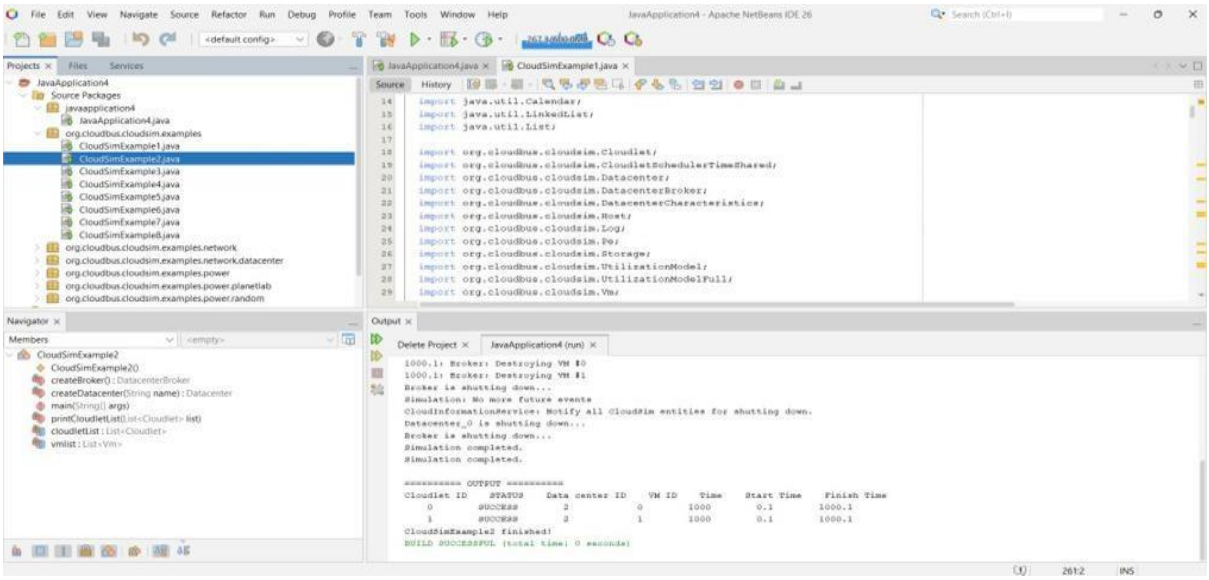



6. Open Desktop → CloudSim 3.0 → examples → org → cloudbus → cloudsim → examples.
7. Copy the required example program.






8. Return to NetBeans, right-click Source Packages, and paste the example package, or create a new Java class using New → Java Class.
9. Copy and paste the required program into the Java class.
10. Save the program.
11. Right-click the Java class and select Run File.
12. Observe the simulation output in the Output window.




13. Repeat the above steps for each of the following programs.

**QUESTIONS:**

**1. Create a basic simulation using CloudSim.**

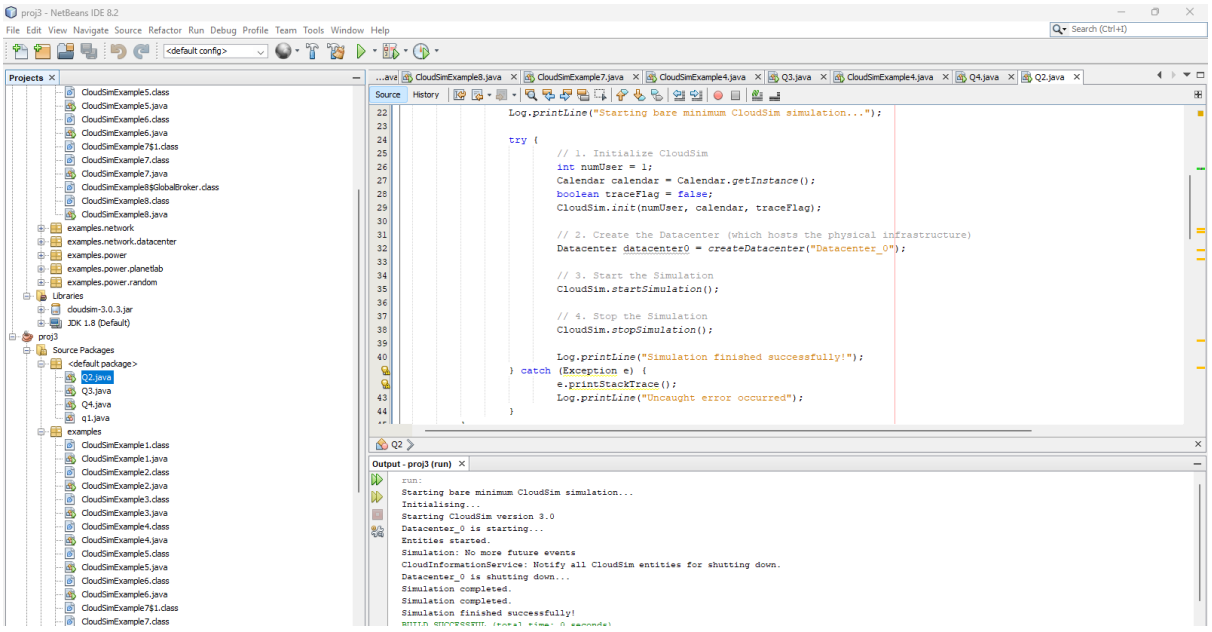
**CODE:**

```
import java.util.ArrayList;
import java.util.Calendar;
import java.util.LinkedList;
import java.util.List;
import org.cloudbus.cloudsim.Datacenter;
import org.cloudbus.cloudsim.DatacenterCharacteristics;
import org.cloudbus.cloudsim.Host;
import org.cloudbus.cloudsim.Log;
import org.cloudbus.cloudsim.Pe;
import org.cloudbus.cloudsim.Storage;
import org.cloudbus.cloudsim.VmAllocationPolicySimple;
import org.cloudbus.cloudsim.VmSchedulerTimeShared;
import org.cloudbus.cloudsim.core.CloudSim;
import org.cloudbus.cloudsim.provisioners.BwProvisionerSimple;
import org.cloudbus.cloudsim.provisioners.PeProvisionerSimple;
import org.cloudbus.cloudsim.provisioners.RamProvisionerSimple;

public class Q1{

public static void main(String[] args) {
Log.printLine("Starting bare minimum CloudSim simulation...");

try {
// 1. Initialize CloudSim
int numUser = 1;
Calendar calendar = Calendar.getInstance();
boolean traceFlag = false;
CloudSim.init(numUser, calendar, traceFlag);

// 2. Create the Datacenter (which hosts the physical infrastructure)
Datacenter datacenter0 = createDatacenter("Datacenter_0");

// 3. Start the Simulation
CloudSim.startSimulation();

// 4. Stop the Simulation
CloudSim.stopSimulation();

Log.printLine("Simulation finished successfully!");
} catch (Exception e) {
e.printStackTrace();
Log.printLine("Uncaught error occurred");
}
}
```

```
private static Datacenter createDatacenter(String name) {
// 1. Create a list to hold our physical machines (Hosts)
List<Host> hostList = new ArrayList<Host>();

// 2. Create Processing Elements (CPUs/Cores)
List<Pe> peList = new ArrayList<Pe>();
int mips = 1000;
peList.add(new Pe(0, new PeProvisionerSimple(mips)));

// 3. Create a Host with its configurations
int hostId = 0;
int ram = 2048; // host memory (MB)
long storage = 1000000; // host storage
int bw = 10000;

hostList.add(
new Host(
hostId,
new RamProvisionerSimple(ram),
new BwProvisionerSimple(bw),
storage,
peList,
new VmSchedulerTimeShared(peList)
)
);

// 4. Set Datacenter characteristics
String arch = "x86";
String os = "Linux";
String vmm = "Xen";
double time_zone = 10.0;
double cost = 3.0;
double costPerMem = 0.05;
double costPerStorage = 0.001;
double costPerBw = 0.0;

LinkedList<Storage> storageList = new LinkedList<Storage>();

DatacenterCharacteristics characteristics = new DatacenterCharacteristics(
arch, os, vmm, hostList, time_zone, cost, costPerMem, costPerStorage, costPerBw);

// 5. Create and return the Datacenter object
Datacenter datacenter = null;
try {
datacenter = new Datacenter(name, characteristics, new VmAllocationPolicySimple(hostList),
storageList, 0);
} catch (Exception e) {
e.printStackTrace();
}
```

```
return datacenter;
}
}
```

**OUTPUT:**




**2. Create a simple datacenter using CloudSim.**

**CODE:**

```
/*
 * Title:        CloudSim Toolkit
 * Description:  CloudSim (Cloud Simulation) Toolkit for Modeling and Simulation
 *               of Clouds
 * Licence:      GPL - http://www.gnu.org/copyleft/gpl.html
 *
 * Copyright (c) 2009, The University of Melbourne, Australia
 */

import java.text.DecimalFormat;
import java.util.ArrayList;
import java.util.Calendar;
import java.util.LinkedList;
import java.util.List;

import org.cloudbus.cloudsim.Cloudlet;
import org.cloudbus.cloudsim.CloudletSchedulerTimeShared;
import org.cloudbus.cloudsim.Datacenter;
import org.cloudbus.cloudsim.DatacenterBroker;
import org.cloudbus.cloudsim.DatacenterCharacteristics;
import org.cloudbus.cloudsim.Host;
import org.cloudbus.cloudsim.Log;
```

```
import org.cloudbus.cloudsim.Pe;
import org.cloudbus.cloudsim.Storage;
import org.cloudbus.cloudsim.UtilizationModel;
import org.cloudbus.cloudsim.UtilizationModelFull;
import org.cloudbus.cloudsim.Vm;
import org.cloudbus.cloudsim.VmAllocationPolicySimple;
import org.cloudbus.cloudsim.VmSchedulerTimeShared;
import org.cloudbus.cloudsim.core.CloudSim;
import org.cloudbus.cloudsim.provisioners.BwProvisionerSimple;
import org.cloudbus.cloudsim.provisioners.PeProvisionerSimple;
import org.cloudbus.cloudsim.provisioners.RamProvisionerSimple;

/**
 * A simple example showing how to create a datacenter with one host and run one
 * cloudlet on it.
 */
public class q2{

/** The cloudlet list. */
private static List<Cloudlet> cloudletList;

/** The vmlist. */
private static List<Vm> vmlist;

/**
* Creates main() to run this example.
*
* @param args the args
*/
@SuppressWarnings("unused")
public static void main(String[] args) {

Log.printLine("Starting CloudSimExample1...");

try {
// First step: Initialize the CloudSim package. It should be called
// before creating any entities.
int num_user = 1; // number of cloud users
Calendar calendar = Calendar.getInstance();
boolean trace_flag = false; // mean trace events

// Initialize the CloudSim library
CloudSim.init(num_user, calendar, trace_flag);

// Second step: Create Datacenters
// Datacenters are the resource providers in CloudSim. We need at
// list one of them to run a CloudSim simulation
Datacenter datacenter0 = createDatacenter("Datacenter_0");

// Third step: Create Broker
```

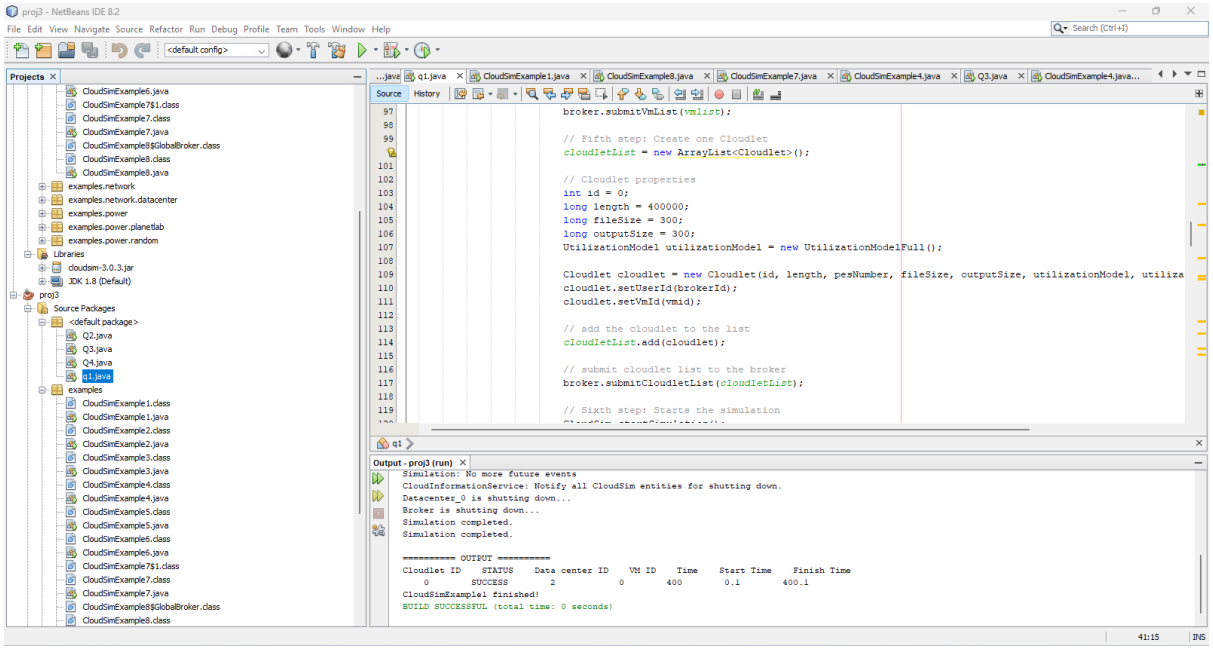
```
DatacenterBroker broker = createBroker();
int brokerId = broker.getId();

// Fourth step: Create one virtual machine
vmlist = new ArrayList<Vm>();

// VM description
int vmid = 0;
int mips = 1000;
long size = 10000; // image size (MB)
int ram = 512; // vm memory (MB)
long bw = 1000;
int pesNumber = 1; // number of cpus
String vmm = "Xen"; // VMM name

// create VM
Vm vm = new Vm(vmid, brokerId, mips, pesNumber, ram, bw, size, vmm, new
CloudletSchedulerTimeShared());

// add the VM to the vmList
vmlist.add(vm);

// submit vm list to the broker
broker.submitVmList(vmlist);

// Fifth step: Create one Cloudlet
cloudletList = new ArrayList<Cloudlet>();

// Cloudlet properties
int id = 0;
long length = 400000;
long fileSize = 300;
long outputSize = 300;
UtilizationModel utilizationModel = new UtilizationModelFull();

Cloudlet cloudlet = new Cloudlet(id, length, pesNumber, fileSize, outputSize,
utilizationModel, utilizationModel, utilizationModel);
cloudlet.setUserId(brokerId);
cloudlet.setVmId(vmid);

// add the cloudlet to the list
cloudletList.add(cloudlet);

// submit cloudlet list to the broker
broker.submitCloudletList(cloudletList);

// Sixth step: Starts the simulation
CloudSim.startSimulation();

CloudSim.stopSimulation();
```

```
//Final step: Print results when simulation is over
List<Cloudlet> newList = broker.getCloudletReceivedList();
printCloudletList(newList);

Log.printLine("CloudSimExample1 finished!");
} catch (Exception e) {
e.printStackTrace();
Log.printLine("Unwanted errors happen");
}
}

/**
* Creates the datacenter.
*
* @param name the name
*
* @return the datacenter
*/
private static Datacenter createDatacenter(String name) {

// Here are the steps needed to create a PowerDatacenter:
// 1. We need to create a list to store
// our machine
List<Host> hostList = new ArrayList<Host>();

// 2. A Machine contains one or more PEs or CPUs/Cores.
// In this example, it will have only one core.
List<Pe> peList = new ArrayList<Pe>();

int mips = 1000;

// 3. Create PEs and add these into a list.
peList.add(new Pe(0, new PeProvisionerSimple(mips))); // need to store Pe id and MIPS Rating

// 4. Create Host with its id and list of PEs and add them to the list
// of machines
int hostId = 0;
int ram = 2048; // host memory (MB)
long storage = 1000000; // host storage
int bw = 10000;

hostList.add(
new Host(
hostId,
new RamProvisionerSimple(ram),
new BwProvisionerSimple(bw),
storage,
peList,
new VmSchedulerTimeShared(peList)
```

```
)
); // This is our machine

// 5. Create a DatacenterCharacteristics object that stores the
// properties of a data center: architecture, OS, list of
// Machines, allocation policy: time- or space-shared, time zone
// and its price (G$/Pe time unit).
String arch = "x86"; // system architecture
String os = "Linux"; // operating system
String vmm = "Xen";
double time_zone = 10.0; // time zone this resource located
double cost = 3.0; // the cost of using processing in this resource
double costPerMem = 0.05; // the cost of using memory in this resource
double costPerStorage = 0.001; // the cost of using storage in this
// resource
double costPerBw = 0.0; // the cost of using bw in this resource
LinkedList<Storage> storageList = new LinkedList<Storage>(); // we are not adding SAN
// devices by now

DatacenterCharacteristics characteristics = new DatacenterCharacteristics(
arch, os, vmm, hostList, time_zone, cost, costPerMem,
costPerStorage, costPerBw);

// 6. Finally, we need to create a PowerDatacenter object.
Datacenter datacenter = null;
try {
datacenter = new Datacenter(name, characteristics, new VmAllocationPolicySimple(hostList),
storageList, 0);
} catch (Exception e) {
e.printStackTrace();
}

return datacenter;
}

// We strongly encourage users to develop their own broker policies, to
// submit vms and cloudlets according
// to the specific rules of the simulated scenario
/**
* Creates the broker.
*
* @return the datacenter broker
*/
private static DatacenterBroker createBroker() {
DatacenterBroker broker = null;
try {
broker = new DatacenterBroker("Broker");
} catch (Exception e) {
e.printStackTrace();
return null;
```

```
}
return broker;
}

/**
* Prints the Cloudlet objects.
*
* @param list list of Cloudlets
*/
private static void printCloudletList(List<Cloudlet> list) {
int size = list.size();
Cloudlet cloudlet;

String indent = "    ";
Log.printLine();
Log.printLine("========== OUTPUT ==========");
Log.printLine("Cloudlet ID" + indent + "STATUS" + indent
+ "Data center ID" + indent + "VM ID" + indent + "Time" + indent
+ "Start Time" + indent + "Finish Time");

DecimalFormat dft = new DecimalFormat("###.##");
for (int i = 0; i < size; i++) {
cloudlet = list.get(i);
Log.print(indent + cloudlet.getCloudletId() + indent + indent);

if (cloudlet.getCloudletStatus() == Cloudlet.SUCCESS) {
Log.print("SUCCESS");

Log.printLine(indent + indent + cloudlet.getResourceId()
+ indent + indent + indent + cloudlet.getVmId()
+ indent + indent
+ dft.format(cloudlet.getActualCPUTime()) + indent
+ indent + dft.format(cloudlet.getExecStartTime())
+ indent + indent
+ dft.format(cloudlet.getFinishTime()));
}
}
}
}
```

**OUTPUT:**




**3. Create a single virtual machine and cloudlet using CloudSim.**

**CODE:**

```
/*
 * Title:        CloudSim Toolkit
 * Description:  CloudSim (Cloud Simulation) Toolkit for Modeling and Simulation
 *               of Clouds
 * Licence:      GPL - http://www.gnu.org/copyleft/gpl.html
 *
 * Copyright (c) 2009, The University of Melbourne, Australia
 */

package org.cloudbus.cloudsim.examples;

import java.text.DecimalFormat;
import java.util.ArrayList;
import java.util.Calendar;
import java.util.LinkedList;
import java.util.List;
import org.cloudbus.cloudsim.Cloudlet;
import org.cloudbus.cloudsim.CloudletSchedulerTimeShared;
import org.cloudbus.cloudsim.Datacenter;
import org.cloudbus.cloudsim.DatacenterBroker;
import org.cloudbus.cloudsim.DatacenterCharacteristics;
import org.cloudbus.cloudsim.Host;
import org.cloudbus.cloudsim.Log;
import org.cloudbus.cloudsim.Pe;
import org.cloudbus.cloudsim.Storage;
import org.cloudbus.cloudsim.UtilizationModel;
import org.cloudbus.cloudsim.UtilizationModelFull;
```

```
import org.cloudbus.cloudsim.Vm;
import org.cloudbus.cloudsim.VmAllocationPolicySimple;
import org.cloudbus.cloudsim.VmSchedulerSpaceShared;
import org.cloudbus.cloudsim.core.CloudSim;
import org.cloudbus.cloudsim.provisioners.BwProvisionerSimple;
import org.cloudbus.cloudsim.provisioners.PeProvisionerSimple;
import org.cloudbus.cloudsim.provisioners.RamProvisionerSimple;


/**
 * A simple example showing how to create
 * two datacenters with one host each and
 * run two cloudlets on them.
 */
public class Q3{

/** The cloudlet list. */
private static List<Cloudlet> cloudletList;

/** The vmlist. */
private static List<Vm> vmlist;

/**
* Creates main() to run this example
*/
public static void main(String[] args) {

Log.printLine("Starting CloudSimExample4...");

try {
// First step: Initialize the CloudSim package. It should be called
// before creating any entities.
int num_user = 1;   // number of cloud users
Calendar calendar = Calendar.getInstance();
boolean trace_flag = false;  // mean trace events

// Initialize the GridSim library
CloudSim.init(num_user, calendar, trace_flag);

// Second step: Create Datacenters
//Datacenters are the resource providers in CloudSim. We need at list one of them to run a
CloudSim simulation
@SuppressWarnings("unused")
Datacenter datacenter0 = createDatacenter("Datacenter_0");
@SuppressWarnings("unused")
Datacenter datacenter1 = createDatacenter("Datacenter_1");

//Third step: Create Broker
DatacenterBroker broker = createBroker();
int brokerId = broker.getId();
```

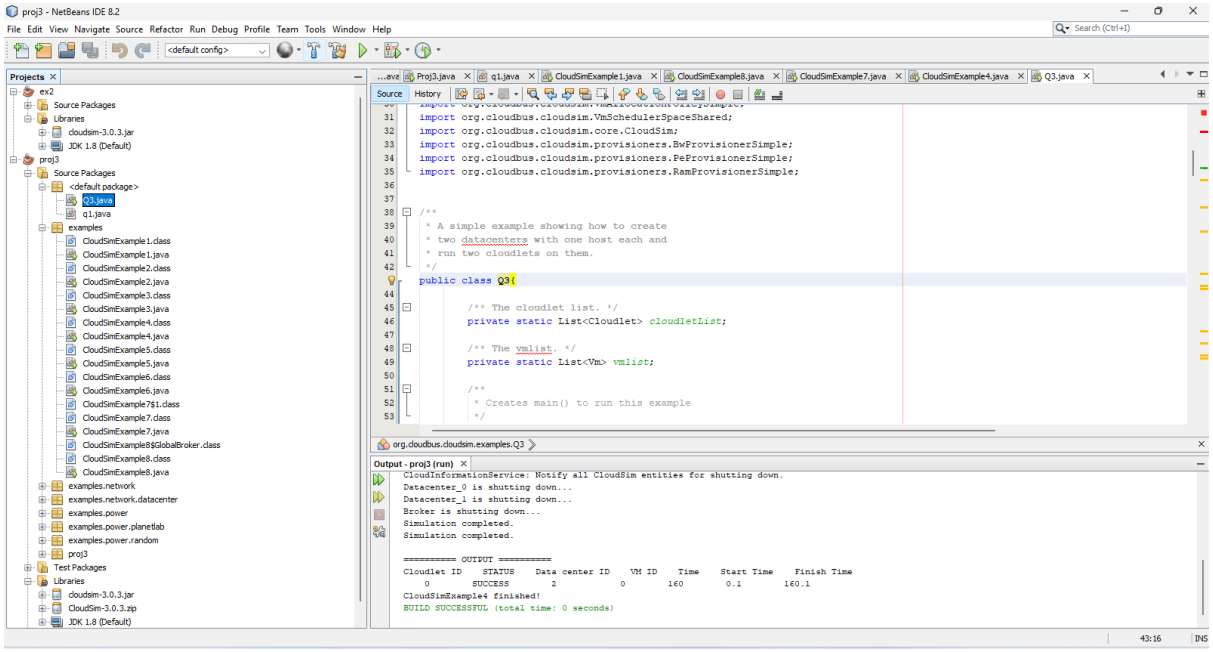
```
//Fourth step: Create one virtual machine
vmlist = new ArrayList<Vm>();

//VM description
int vmid = 0;
int mips = 250;
long size = 10000; //image size (MB)
int ram = 512; //vm memory (MB)
long bw = 1000;
int pesNumber = 1; //number of cpus
String vmm = "Xen"; //VMM name

//create two VMs
Vm vm1 = new Vm(vmid, brokerId, mips, pesNumber, ram, bw, size, vmm, new
CloudletSchedulerTimeShared());


//add the VMs to the vmList
vmlist.add(vm1);

//submit vm list to the broker
broker.submitVmList(vmlist);


//Fifth step: Create two Cloudlets
cloudletList = new ArrayList<Cloudlet>();

//Cloudlet properties
int id = 0;
long length = 40000;
long fileSize = 300;
long outputSize = 300;
UtilizationModel utilizationModel = new UtilizationModelFull();

Cloudlet cloudlet1 = new Cloudlet(id, length, pesNumber, fileSize, outputSize,
utilizationModel, utilizationModel, utilizationModel);
cloudlet1.setUserId(brokerId);


//add the cloudlets to the list
cloudletList.add(cloudlet1);

//submit cloudlet list to the broker
broker.submitCloudletList(cloudletList);


//bind the cloudlets to the vms. This way, the broker
// will submit the bound cloudlets only to the specific VM
broker.bindCloudletToVm(cloudlet1.getCloudletId(),vm1.getId());
```

```
// Sixth step: Starts the simulation
CloudSim.startSimulation();


// Final step: Print results when simulation is over
List<Cloudlet> newList = broker.getCloudletReceivedList();

CloudSim.stopSimulation();

      printCloudletList(newList);

Log.printLine("CloudSimExample4 finished!");
}
catch (Exception e) {
e.printStackTrace();
Log.printLine("The simulation has been terminated due to an unexpected error");
}
}

private static Datacenter createDatacenter(String name){

// Here are the steps needed to create a PowerDatacenter:
// 1. We need to create a list to store
//    our machine
List<Host> hostList = new ArrayList<Host>();

// 2. A Machine contains one or more PEs or CPUs/Cores.
// In this example, it will have only one core.
List<Pe> peList = new ArrayList<Pe>();

int mips = 1000;

// 3. Create PEs and add these into a list.
peList.add(new Pe(0, new PeProvisionerSimple(mips))); // need to store Pe id and MIPS
Rating

//4. Create Host with its id and list of PEs and add them to the list of machines
int hostId=0;
int ram = 2048; //host memory (MB)
long storage = 1000000; //host storage
int bw = 10000;


//in this example, the VMAllocatonPolicy in use is SpaceShared. It means that only one VM
//is allowed to run on each Pe. As each Host has only one Pe, only one VM can run on each
Host.
hostList.add(
   new Host(
   hostId,
```

```
    new RamProvisionerSimple(ram),
    new BwProvisionerSimple(bw),
    storage,
    peList,
    new VmSchedulerSpaceShared(peList)
    )
    ); // This is our first machine

// 5. Create a DatacenterCharacteristics object that stores the
//    properties of a data center: architecture, OS, list of
//    Machines, allocation policy: time- or space-shared, time zone
//    and its price (G$/Pe time unit).
String arch = "x86";      // system architecture
String os = "Linux";          // operating system
String vmm = "Xen";
double time_zone = 10.0;         // time zone this resource located
double cost = 3.0;              // the cost of using processing in this resource
double costPerMem = 0.05; // the cost of using memory in this resource
double costPerStorage = 0.001; // the cost of using storage in this resource
double costPerBw = 0.0; // the cost of using bw in this resource
LinkedList<Storage> storageList = new LinkedList<Storage>(); //we are not adding SAN
devices by now

     DatacenterCharacteristics characteristics = new DatacenterCharacteristics(
             arch, os, vmm, hostList, time_zone, cost, costPerMem, costPerStorage, costPerBw);


// 6. Finally, we need to create a PowerDatacenter object.
Datacenter datacenter = null;
try {
datacenter = new Datacenter(name, characteristics, new
VmAllocationPolicySimple(hostList), storageList, 0);
} catch (Exception e) {
e.printStackTrace();
}

return datacenter;
}

//We strongly encourage users to develop their own broker policies, to submit vms and
cloudlets according
//to the specific rules of the simulated scenario
private static DatacenterBroker createBroker(){

DatacenterBroker broker = null;
try {
broker = new DatacenterBroker("Broker");
} catch (Exception e) {
e.printStackTrace();
return null;
```

```
}
return broker;
}

/**
* Prints the Cloudlet objects
* @param list  list of Cloudlets
*/
private static void printCloudletList(List<Cloudlet> list) {
int size = list.size();
Cloudlet cloudlet;

String indent = "    ";
Log.printLine();
Log.printLine("========== OUTPUT ==========");
Log.printLine("Cloudlet ID" + indent + "STATUS" + indent +
"Data center ID" + indent + "VM ID" + indent + "Time" + indent + "Start Time" + indent +
"Finish Time");

DecimalFormat dft = new DecimalFormat("###.##");
for (int i = 0; i < size; i++) {
cloudlet = list.get(i);
Log.print(indent + cloudlet.getCloudletId() + indent + indent);

if (cloudlet.getCloudletStatus() == Cloudlet.SUCCESS){
Log.print("SUCCESS");

Log.printLine( indent + indent + cloudlet.getResourceId() + indent + indent + indent +
cloudlet.getVmId() +
indent + indent + dft.format(cloudlet.getActualCPUTime()) + indent + indent +
dft.format(cloudlet.getExecStartTime())+
indent + indent + dft.format(cloudlet.getFinishTime()));
}
}
}
}
```

**OUTPUT:**




**4. Create multiple virtual machines and multiple cloudlets using CloudSim.**

**CODE:**

```
/*
 * Title:        CloudSim Toolkit
 * Description:  CloudSim (Cloud Simulation) Toolkit for Modeling and Simulation
 *               of Clouds
 * Licence:      GPL - http://www.gnu.org/copyleft/gpl.html
 *
 * Copyright (c) 2009, The University of Melbourne, Australia
 */

package org.cloudbus.cloudsim.examples;

import java.text.DecimalFormat;
import java.util.ArrayList;
import java.util.Calendar;
import java.util.LinkedList;
import java.util.List;
import org.cloudbus.cloudsim.Cloudlet;
import org.cloudbus.cloudsim.CloudletSchedulerTimeShared;
import org.cloudbus.cloudsim.Datacenter;
import org.cloudbus.cloudsim.DatacenterBroker;
import org.cloudbus.cloudsim.DatacenterCharacteristics;
import org.cloudbus.cloudsim.Host;
import org.cloudbus.cloudsim.Log;
import org.cloudbus.cloudsim.Pe;
import org.cloudbus.cloudsim.Storage;
import org.cloudbus.cloudsim.UtilizationModel;
```

```
import org.cloudbus.cloudsim.UtilizationModelFull;
import org.cloudbus.cloudsim.Vm;
import org.cloudbus.cloudsim.VmAllocationPolicySimple;
import org.cloudbus.cloudsim.VmSchedulerSpaceShared;
import org.cloudbus.cloudsim.core.CloudSim;
import org.cloudbus.cloudsim.provisioners.BwProvisionerSimple;
import org.cloudbus.cloudsim.provisioners.PeProvisionerSimple;
import org.cloudbus.cloudsim.provisioners.RamProvisionerSimple;


/**
 * A simple example showing how to create
 * two datacenters with one host each and
 * run two cloudlets on them.
 */
public class Q4{

/** The cloudlet list. */
private static List<Cloudlet> cloudletList;

/** The vmlist. */
private static List<Vm> vmlist;

/**
* Creates main() to run this example
*/
public static void main(String[] args) {

Log.printLine("Starting CloudSimExample4...");

try {
// First step: Initialize the CloudSim package. It should be called
// before creating any entities.
int num_user = 1;   // number of cloud users
Calendar calendar = Calendar.getInstance();
boolean trace_flag = false;  // mean trace events

// Initialize the GridSim library
CloudSim.init(num_user, calendar, trace_flag);

// Second step: Create Datacenters
//Datacenters are the resource providers in CloudSim. We need at list one of them to run a
CloudSim simulation
@SuppressWarnings("unused")
Datacenter datacenter0 = createDatacenter("Datacenter_0");
@SuppressWarnings("unused")
Datacenter datacenter1 = createDatacenter("Datacenter_1");

//Third step: Create Broker
DatacenterBroker broker = createBroker();
```

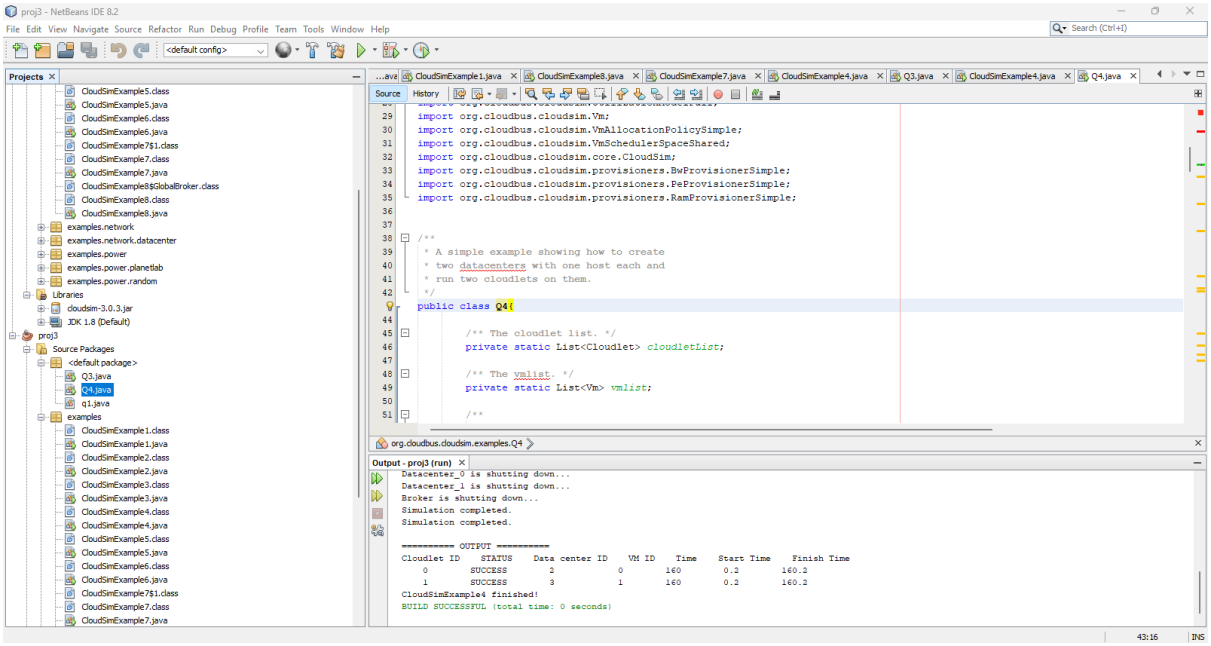
```
int brokerId = broker.getId();

//Fourth step: Create one virtual machine
vmlist = new ArrayList<Vm>();

//VM description
int vmid = 0;
int mips = 250;
long size = 10000; //image size (MB)
int ram = 512; //vm memory (MB)
long bw = 1000;
int pesNumber = 1; //number of cpus
String vmm = "Xen"; //VMM name

//create two VMs
Vm vm1 = new Vm(vmid, brokerId, mips, pesNumber, ram, bw, size, vmm, new
CloudletSchedulerTimeShared());

vmid++;
Vm vm2 = new Vm(vmid, brokerId, mips, pesNumber, ram, bw, size, vmm, new
CloudletSchedulerTimeShared());

//add the VMs to the vmList
vmlist.add(vm1);
vmlist.add(vm2);

//submit vm list to the broker
broker.submitVmList(vmlist);


//Fifth step: Create two Cloudlets
cloudletList = new ArrayList<Cloudlet>();

//Cloudlet properties
int id = 0;
long length = 40000;
long fileSize = 300;
long outputSize = 300;
UtilizationModel utilizationModel = new UtilizationModelFull();

Cloudlet cloudlet1 = new Cloudlet(id, length, pesNumber, fileSize, outputSize,
utilizationModel, utilizationModel, utilizationModel);
cloudlet1.setUserId(brokerId);

id++;
Cloudlet cloudlet2 = new Cloudlet(id, length, pesNumber, fileSize, outputSize,
utilizationModel, utilizationModel, utilizationModel);
cloudlet2.setUserId(brokerId);

//add the cloudlets to the list
```

```
cloudletList.add(cloudlet1);
cloudletList.add(cloudlet2);

//submit cloudlet list to the broker
broker.submitCloudletList(cloudletList);


//bind the cloudlets to the vms. This way, the broker
// will submit the bound cloudlets only to the specific VM
broker.bindCloudletToVm(cloudlet1.getCloudletId(),vm1.getId());
broker.bindCloudletToVm(cloudlet2.getCloudletId(),vm2.getId());

// Sixth step: Starts the simulation
CloudSim.startSimulation();


// Final step: Print results when simulation is over
List<Cloudlet> newList = broker.getCloudletReceivedList();

CloudSim.stopSimulation();

      printCloudletList(newList);

Log.printLine("CloudSimExample4 finished!");
}
catch (Exception e) {
e.printStackTrace();
Log.printLine("The simulation has been terminated due to an unexpected error");
}
}

private static Datacenter createDatacenter(String name){

// Here are the steps needed to create a PowerDatacenter:
// 1. We need to create a list to store
//    our machine
List<Host> hostList = new ArrayList<Host>();

// 2. A Machine contains one or more PEs or CPUs/Cores.
// In this example, it will have only one core.
List<Pe> peList = new ArrayList<Pe>();

int mips = 1000;

// 3. Create PEs and add these into a list.
peList.add(new Pe(0, new PeProvisionerSimple(mips))); // need to store Pe id and MIPS
Rating

//4. Create Host with its id and list of PEs and add them to the list of machines
int hostId=0;
```

```
int ram = 2048; //host memory (MB)
long storage = 1000000; //host storage
int bw = 10000;


//in this example, the VMAllocatonPolicy in use is SpaceShared. It means that only one VM
//is allowed to run on each Pe. As each Host has only one Pe, only one VM can run on each
Host.
hostList.add(
   new Host(
   hostId,
   new RamProvisionerSimple(ram),
   new BwProvisionerSimple(bw),
   storage,
   peList,
   new VmSchedulerSpaceShared(peList)
   )
   ); // This is our first machine

// 5. Create a DatacenterCharacteristics object that stores the
//    properties of a data center: architecture, OS, list of
//    Machines, allocation policy: time- or space-shared, time zone
//    and its price (G$/Pe time unit).
String arch = "x86";      // system architecture
String os = "Linux";          // operating system
String vmm = "Xen";
double time_zone = 10.0;         // time zone this resource located
double cost = 3.0;              // the cost of using processing in this resource
double costPerMem = 0.05; // the cost of using memory in this resource
double costPerStorage = 0.001; // the cost of using storage in this resource
double costPerBw = 0.0; // the cost of using bw in this resource
LinkedList<Storage> storageList = new LinkedList<Storage>(); //we are not adding SAN
devices by now

     DatacenterCharacteristics characteristics = new DatacenterCharacteristics(
             arch, os, vmm, hostList, time_zone, cost, costPerMem, costPerStorage, costPerBw);


// 6. Finally, we need to create a PowerDatacenter object.
Datacenter datacenter = null;
try {
datacenter = new Datacenter(name, characteristics, new
VmAllocationPolicySimple(hostList), storageList, 0);
} catch (Exception e) {
e.printStackTrace();
}

return datacenter;
}
```

```
//We strongly encourage users to develop their own broker policies, to submit vms and
cloudlets according
//to the specific rules of the simulated scenario
private static DatacenterBroker createBroker(){

DatacenterBroker broker = null;
try {
broker = new DatacenterBroker("Broker");
} catch (Exception e) {
e.printStackTrace();
return null;
}
return broker;
}

/**
* Prints the Cloudlet objects
* @param list  list of Cloudlets
*/
private static void printCloudletList(List<Cloudlet> list) {
int size = list.size();
Cloudlet cloudlet;

String indent = "    ";
Log.printLine();
Log.printLine("========== OUTPUT ==========");
Log.printLine("Cloudlet ID" + indent + "STATUS" + indent +
"Data center ID" + indent + "VM ID" + indent + "Time" + indent + "Start Time" + indent +
"Finish Time");

DecimalFormat dft = new DecimalFormat("###.##");
for (int i = 0; i < size; i++) {
cloudlet = list.get(i);
Log.print(indent + cloudlet.getCloudletId() + indent + indent);

if (cloudlet.getCloudletStatus() == Cloudlet.SUCCESS){
Log.print("SUCCESS");

Log.printLine( indent + indent + cloudlet.getResourceId() + indent + indent + indent +
cloudlet.getVmId() +
indent + indent + dft.format(cloudlet.getActualCPUTime()) + indent + indent +
dft.format(cloudlet.getExecStartTime())+
indent + indent + dft.format(cloudlet.getFinishTime()));
}
}

}
}
```

## OUTPUT:




**RESULT:** The CloudSim toolkit was successfully configured in NetBeans, and various cloud computing scenarios were simulated.

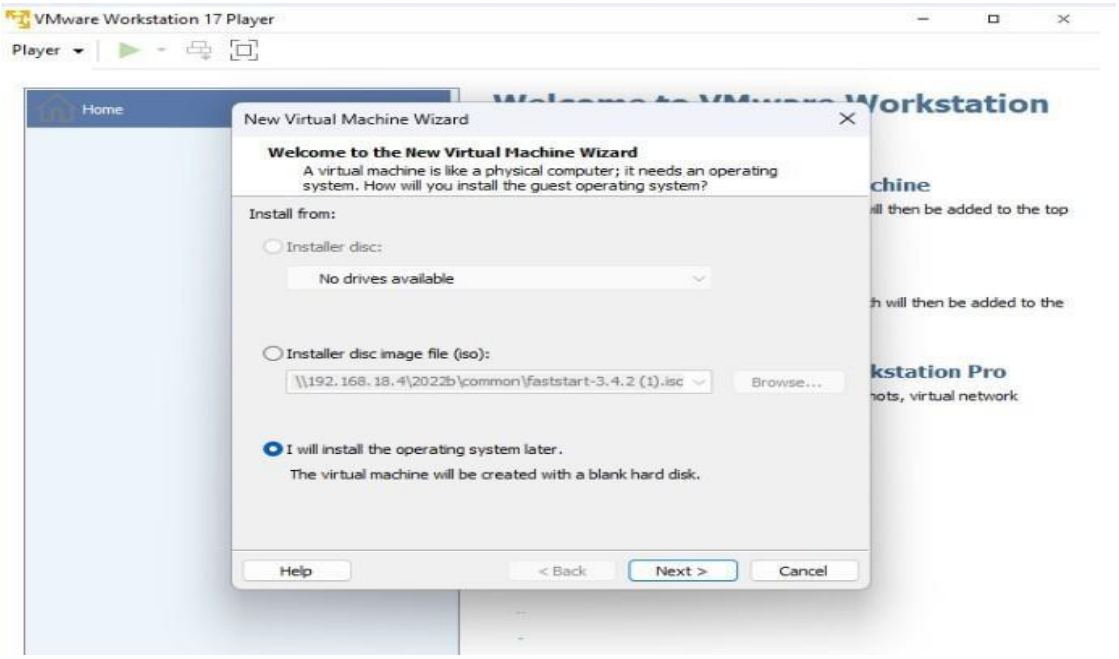
**EXP NO: 3** **INSTALLATION OF EUCALYPTUS**
**DATE:**

**AIM:** To preform installation of eucalyptus.

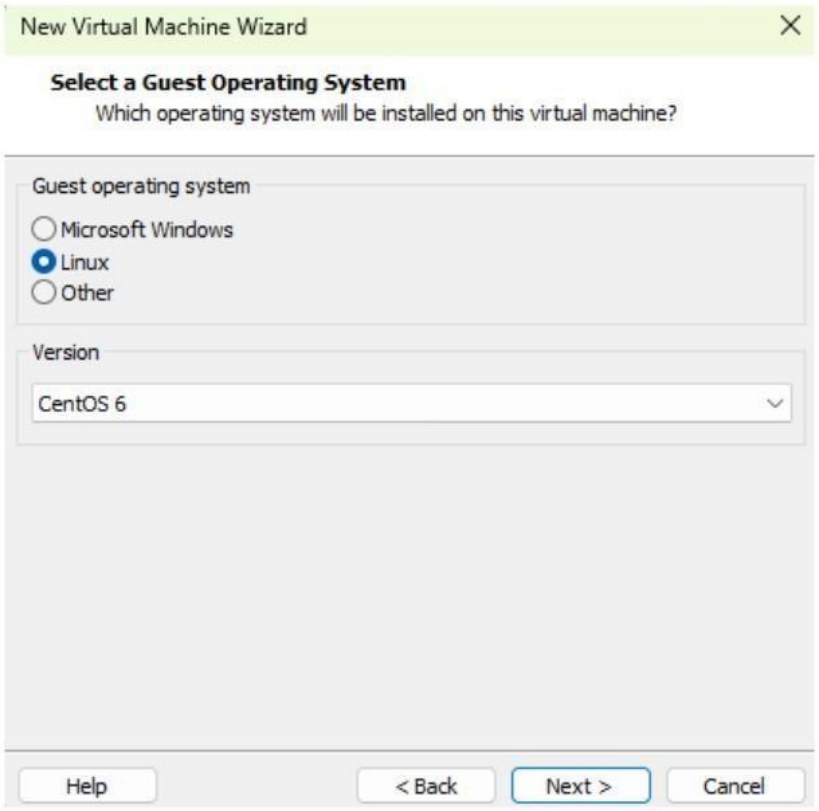
**PROCEDURE:**

1. Click on VMWare workstation in your desktop and click on Create a New Virtual Machine.

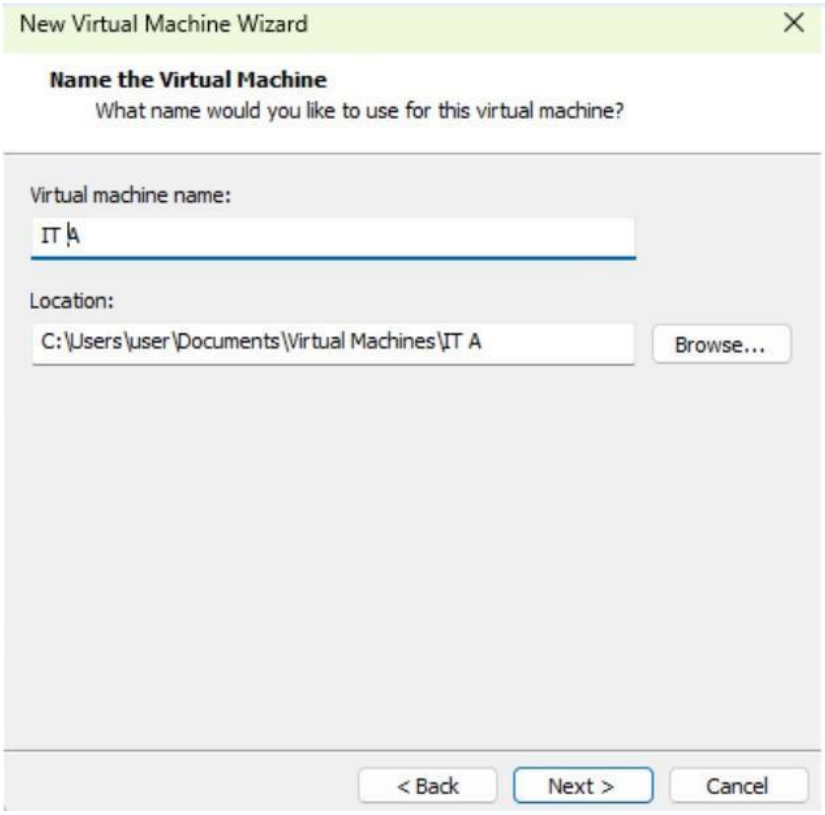

2. Choose the 'I will install the operating system later' option and click Next.



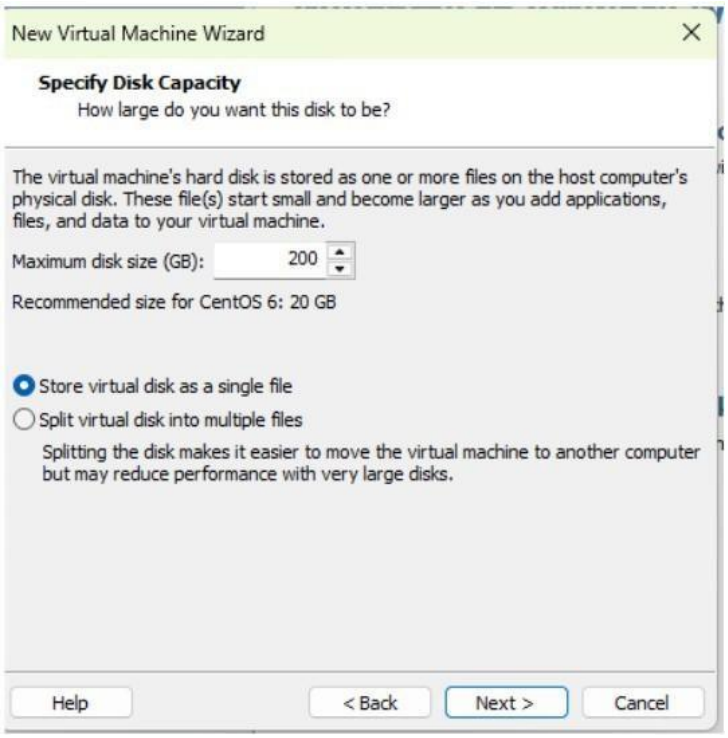
3. Choose Linux as the Guest Operating system, select CentOS 6 as version and click Next.



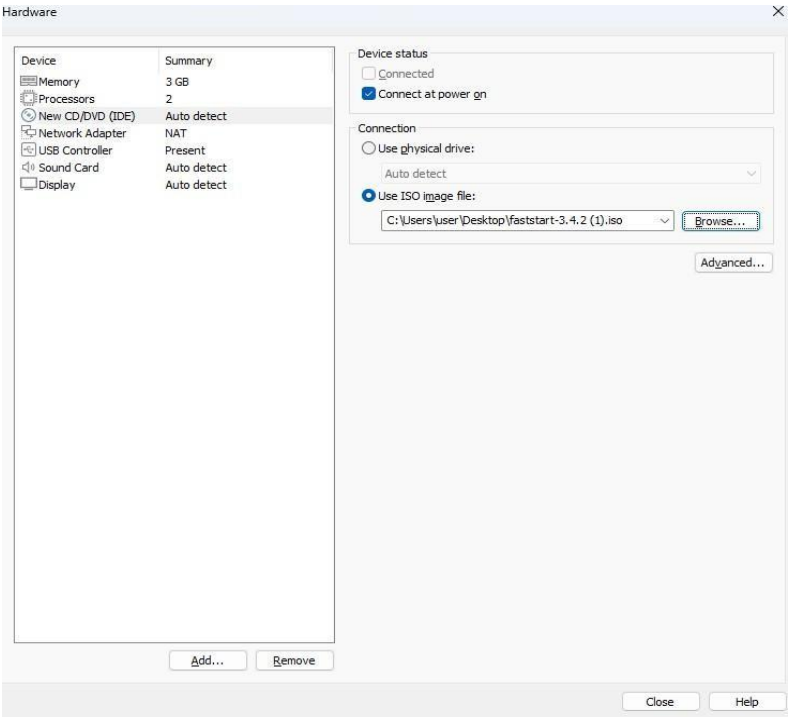

4. Name the Virtual Machine as John, click Next.



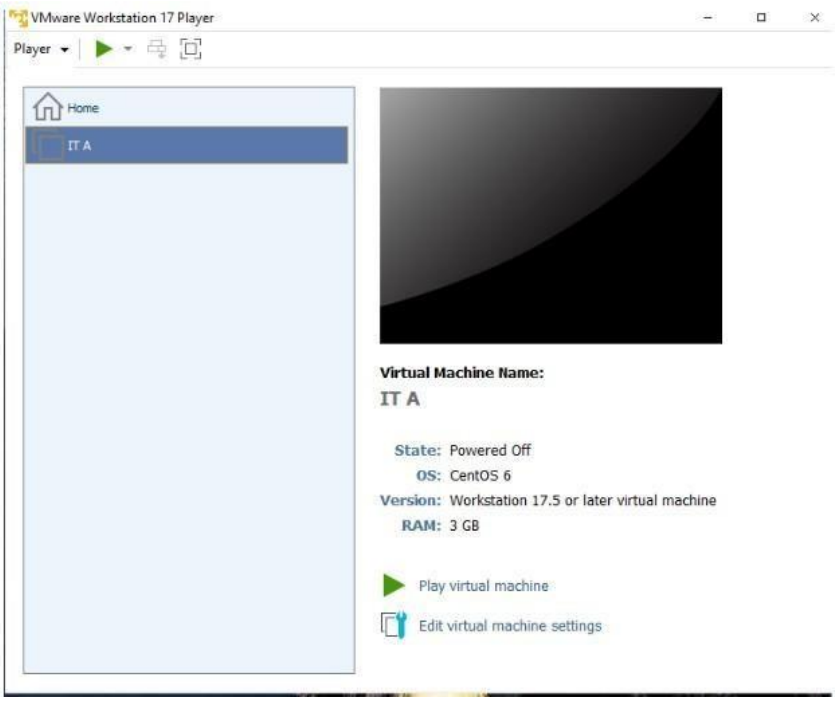
5. Enter the maximum disk size as 200GB and choose the option. Store visual disk as a single file.



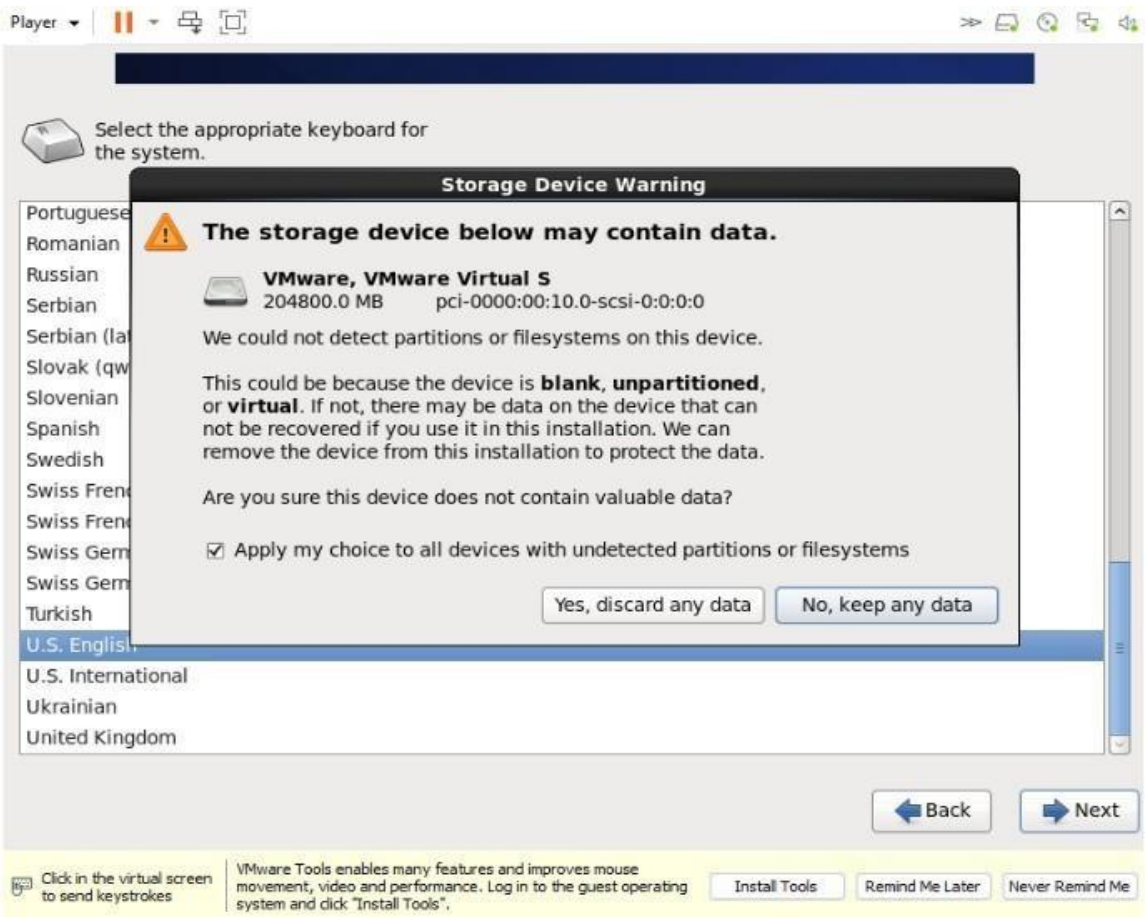

6. Click on Customize Hardware and configure the Virtual Machine as below:
   1. Choose the memory as 3GB(3072 MB)
   2. Enter the number of processors as 2.
   3. Choose ISO image file, click Browse, select faststart-3.4.2.iso, click close and then finish.



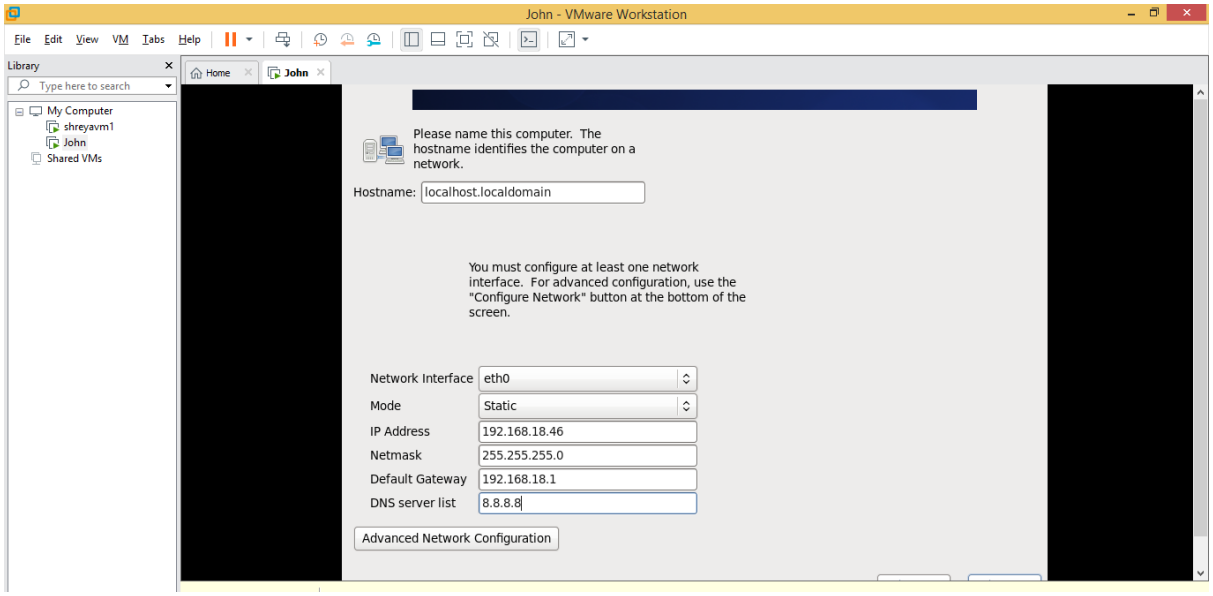
7. Click Play Virtual Machine, choose Install CentOS 6 single system cloud in 60x.,then press skip and then ok.



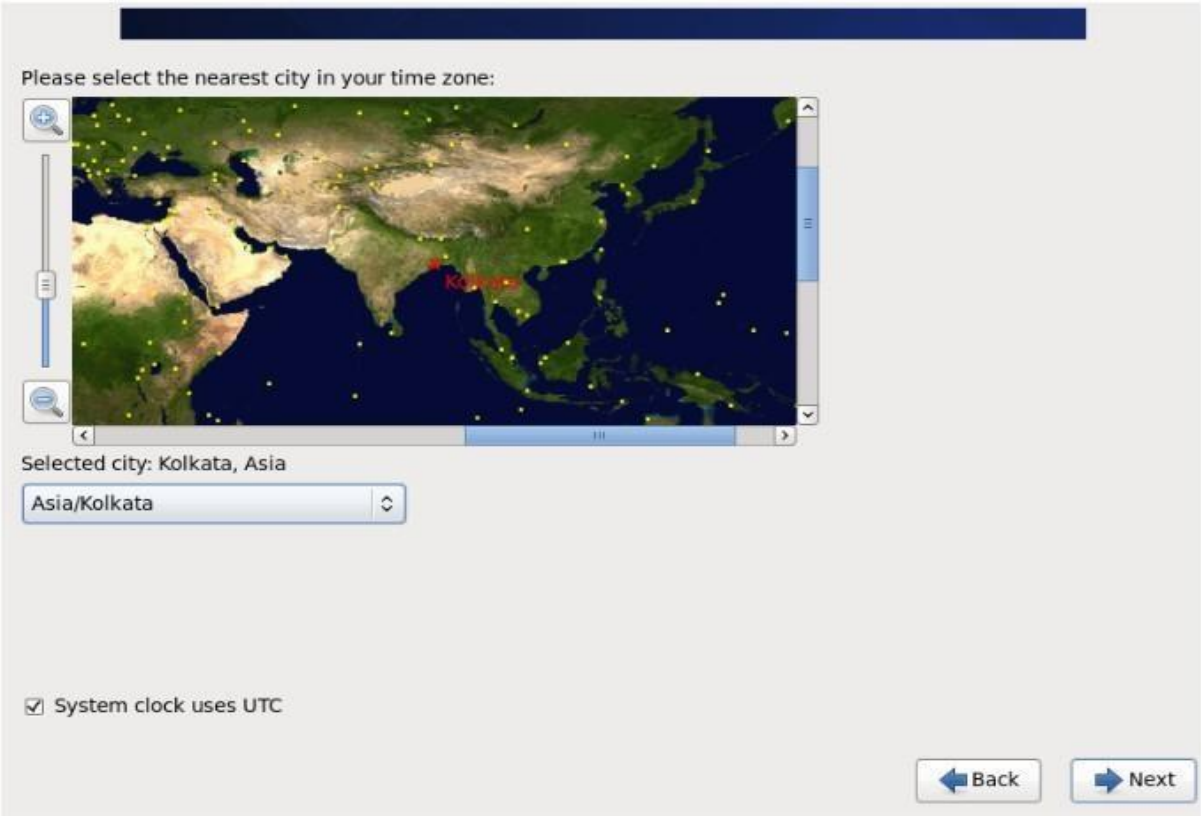

8. Select language as English and click Next. And Click Yes, discard my data.



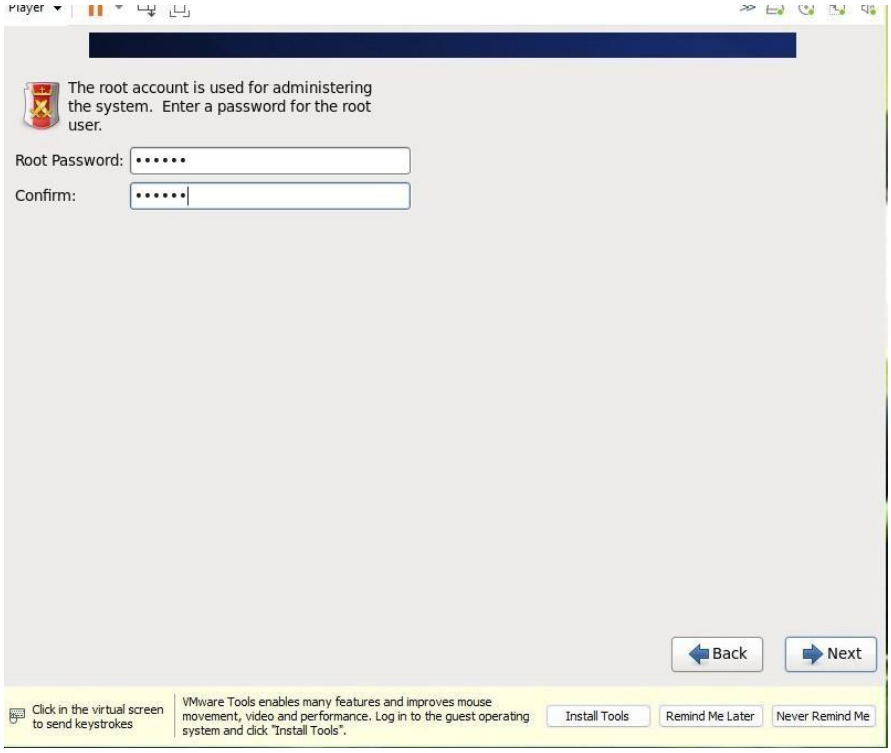
**9.** Set hostname as “root”, set ip= 192.168.18.35, netmask= 255.255.254.0, gateway= 192.168.18.1, mask=8.8.8.8.



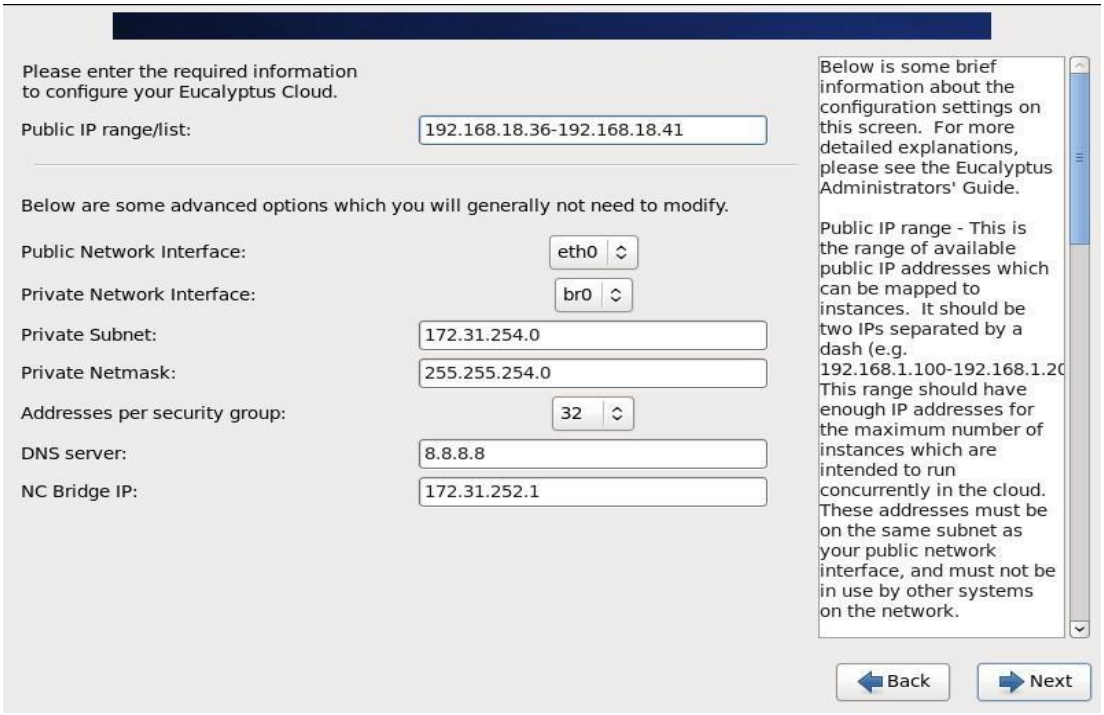

**10.** Locate Asia/Kolkata and click Next.



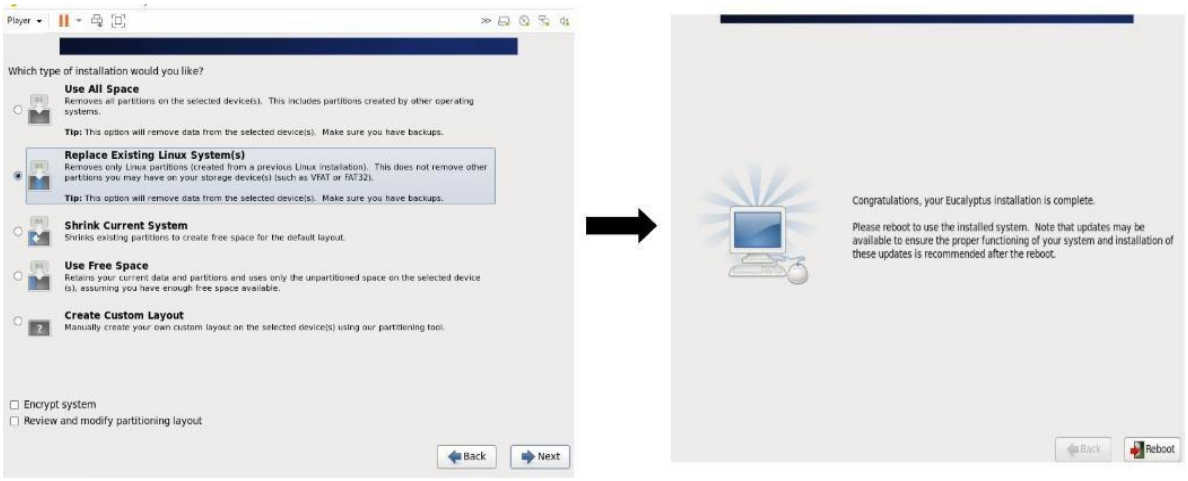
**11.** Enter root password : 123456 and enter it again to confirm. Press Next, and click Use anyway.



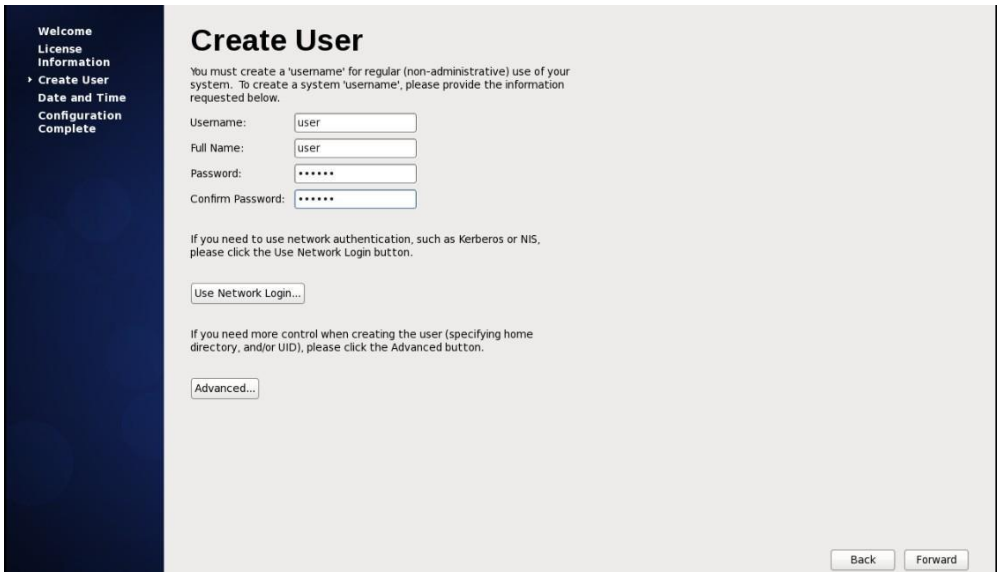

**12.** Set public IP range/list: 192.168.18.36-192.168.18.41 (Your system number- your system number+5).



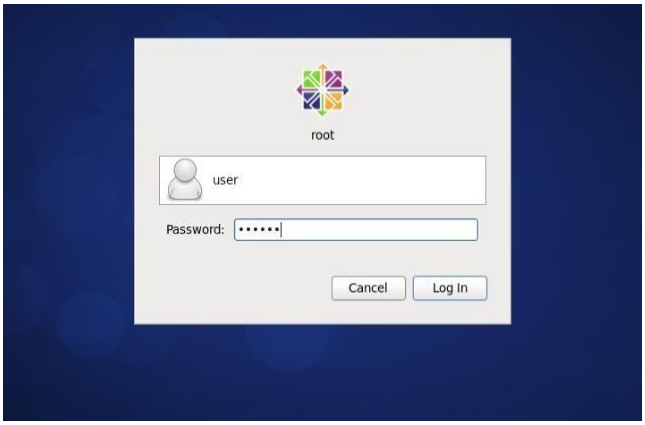

**13.** Click Replace existing Linux system and press Write changes; press reboot- it will take a few minutes.




**14.** Click Forward twice, then enter username and Full name as "user" and password as : 123456. Then click forward




**15.** Click finish. Virtual Machine is created with Linux system. And Enter password: 123456 and log in.



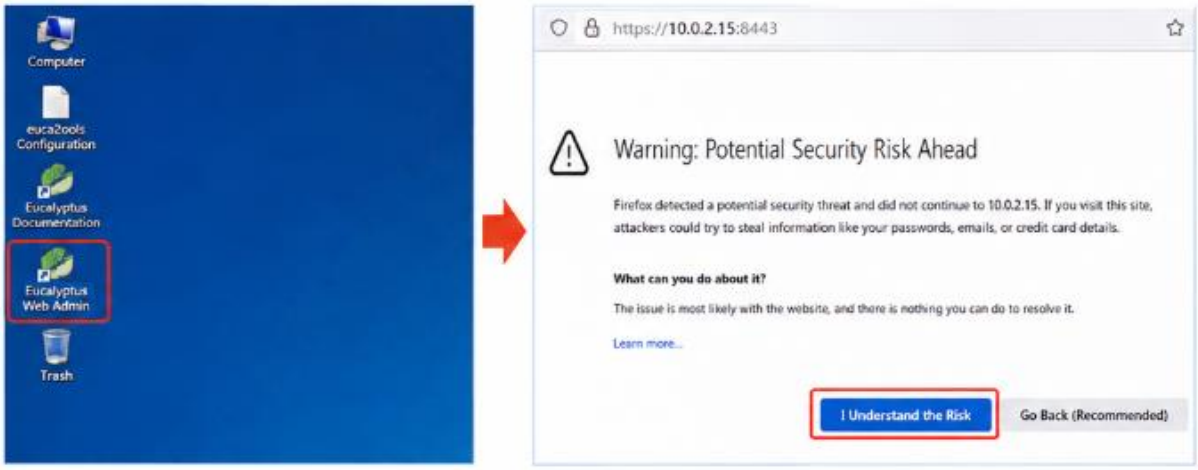

**RESULT:** Thus, setting up cloud using Eucalyptus has been completed successfully.

**EXP NO: 4 RUNNING VIRTUAL MACHINE OF DIFFERENT CONFIGURATIONS**
**DATE:**

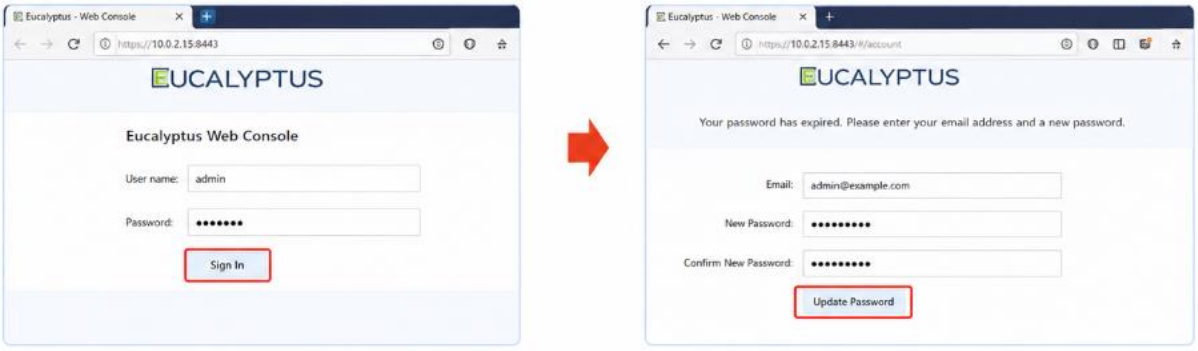
**AIM:** To run Virtual Machines of different configurations.

**PROCEDURE:**

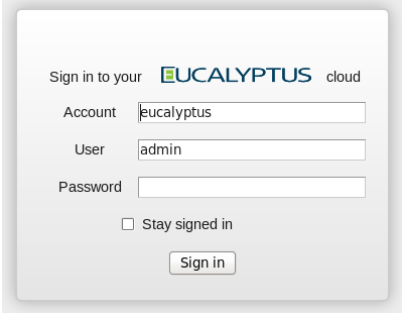
1. Open the Virtual Machine (VM) and click on Eucalyptus Web Admin from the desktop. In the browser, click "I Understand the Risk", then click "Add Exception" and "Confirm Security Exception."



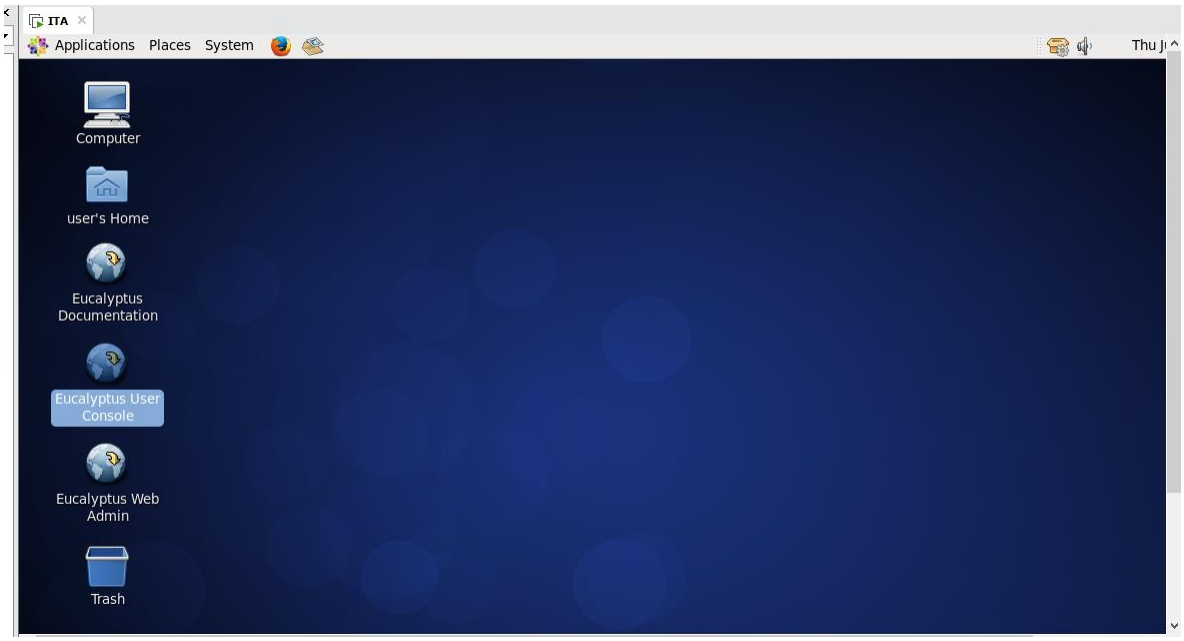

2. Log in as the administrator. Enter the username admin and password admin, then click Sign In. When prompted, enter your email ID and reset/set the new password as admin1

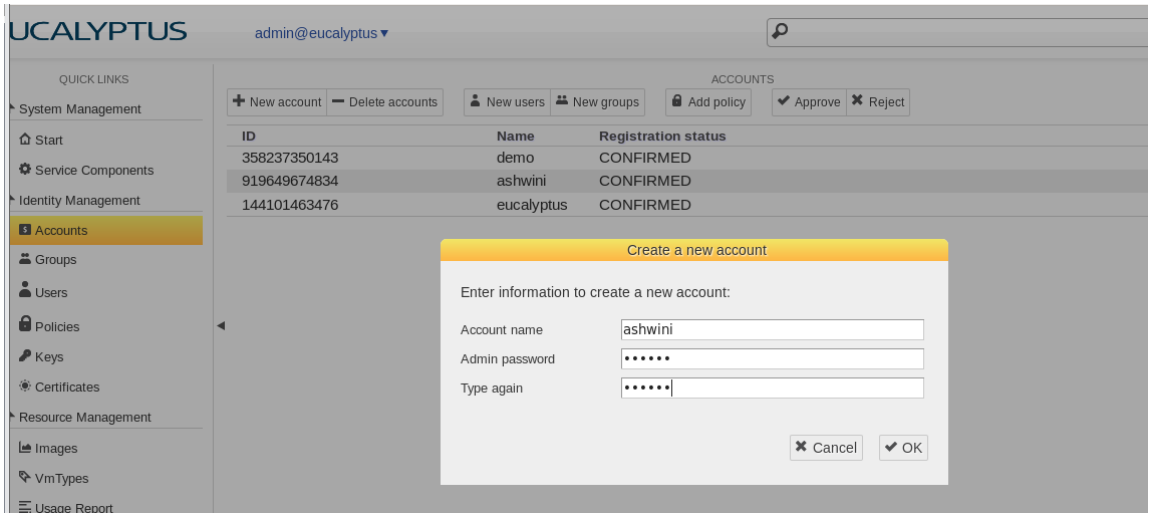


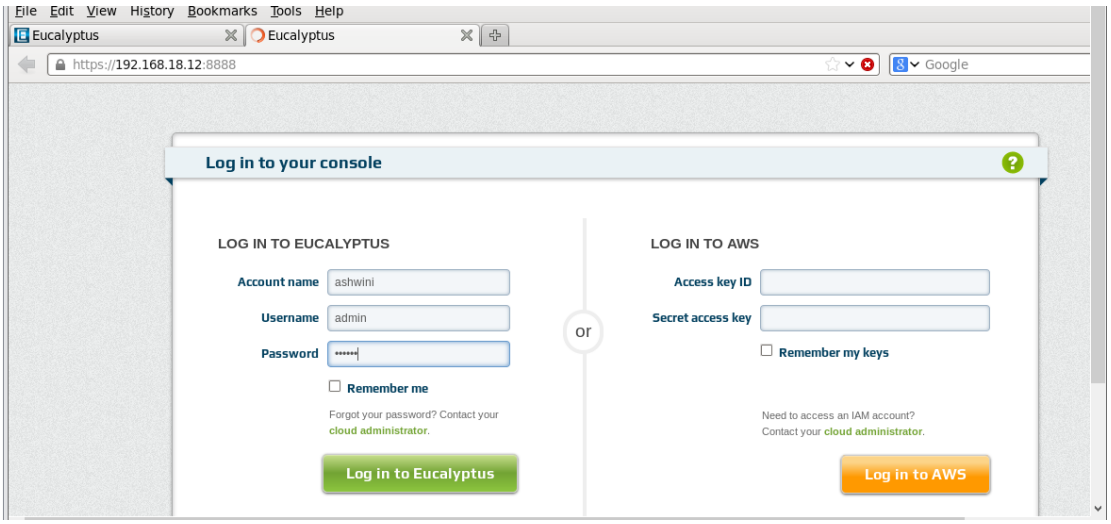

3. Log in again using the username admin and password admin1, then click OK.



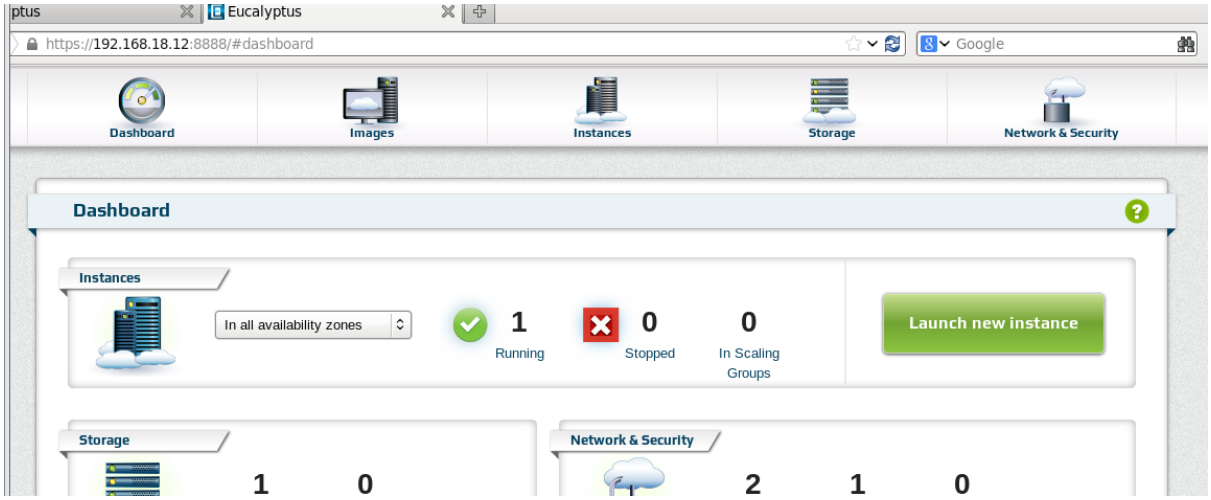
4. Open the Eucalyptus User Console. Enter the Account Name as your username, Username as admin, and Password as admin1. Click Login.

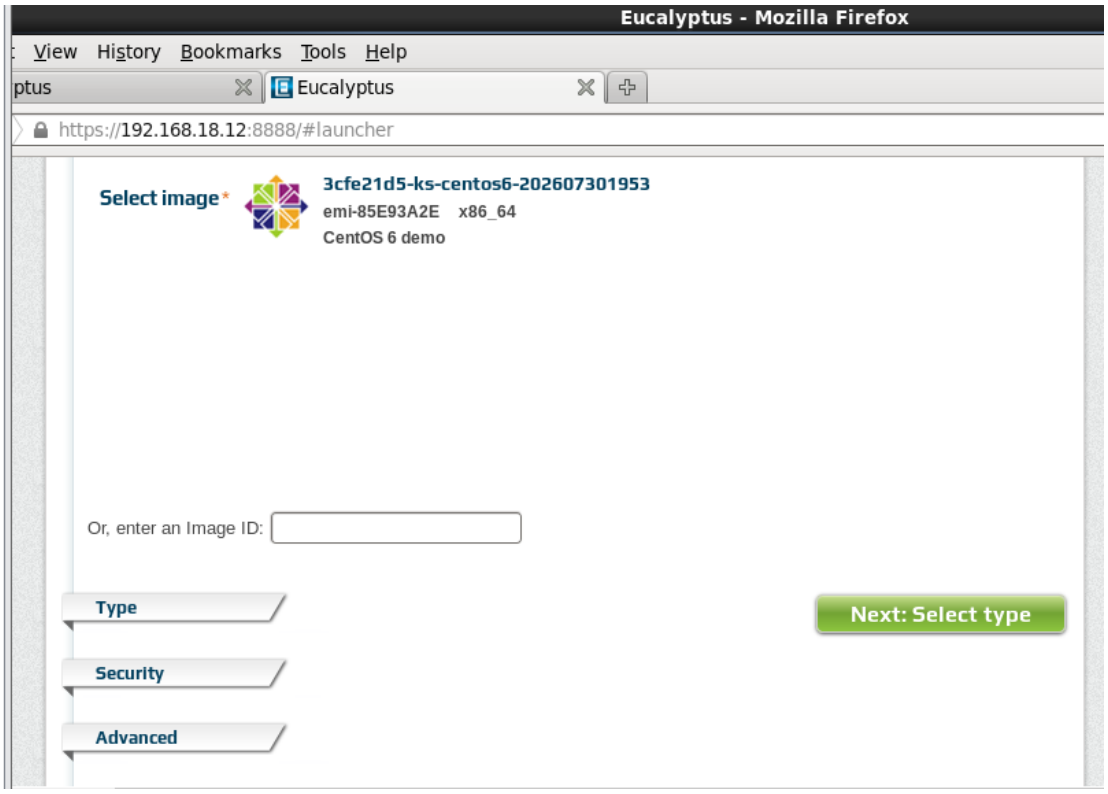


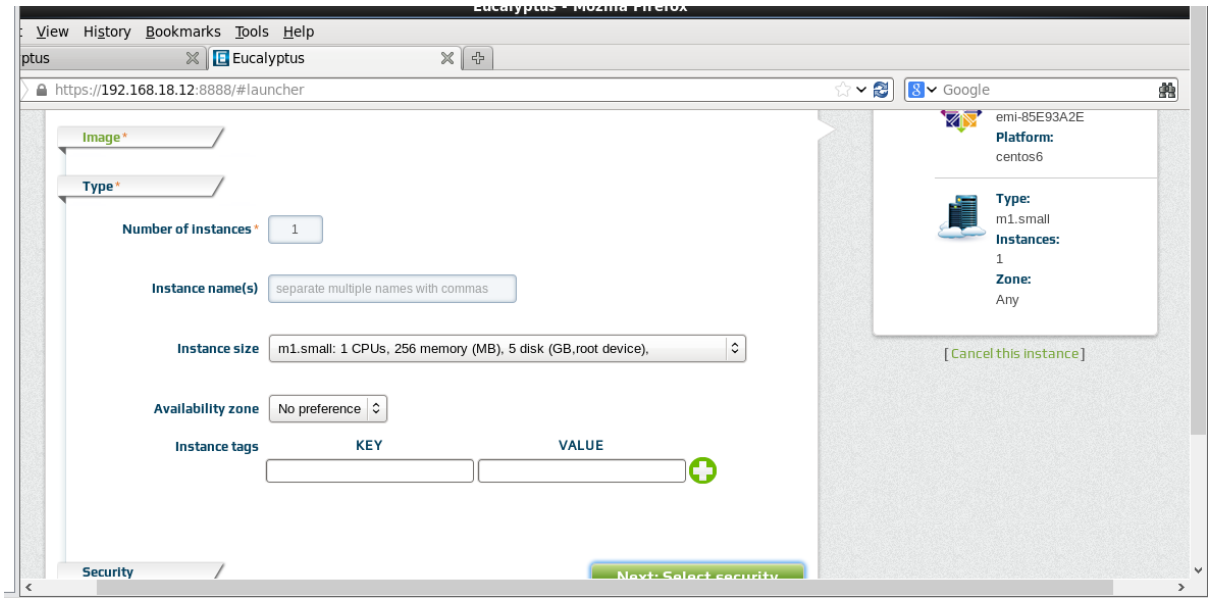



5. Click Launch New Instance. In the Select Image page, choose the required image and select the instance type.

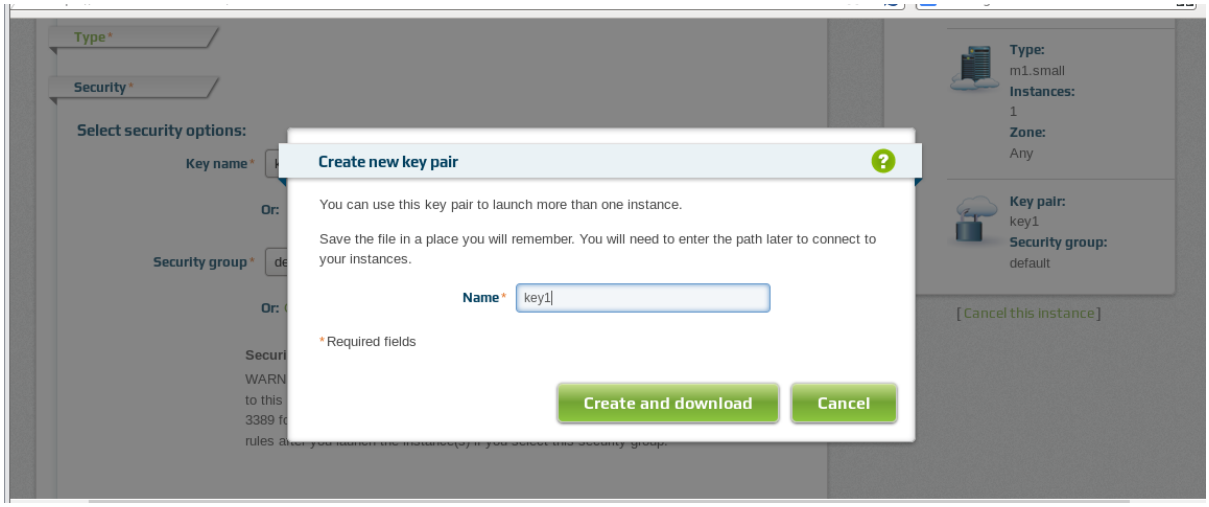


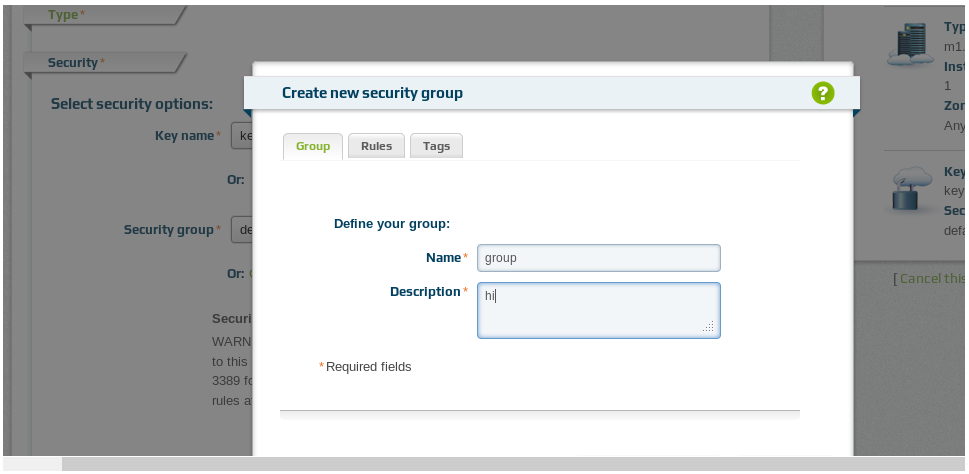

6. Set the Number of Instances as 1. Enter the Instance Name as your username and click Select Security.



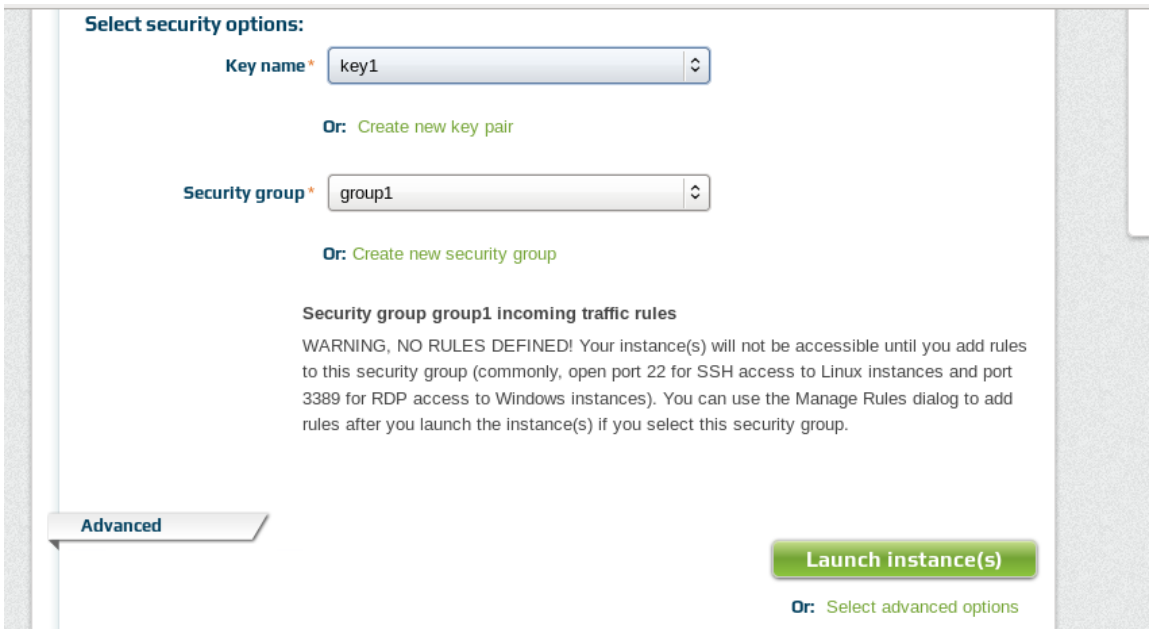

7. Click Create New Key Pair, enter the key pair name as key1, click Create, then Download the key pair and click OK.



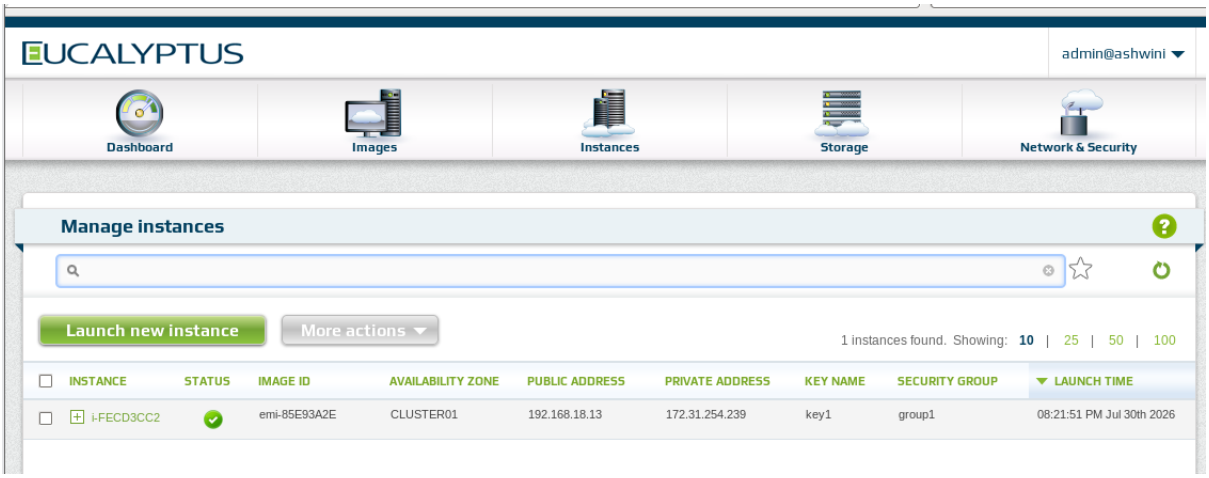

8. Click Create New Security Group, enter the group name as group1, provide a description (e.g., "Hi User"), and click Create Group.



9. Click Launch Instance.




10. Wait until the instance status changes to Running. This indicates that the virtual machine instance has been successfully created and launched.



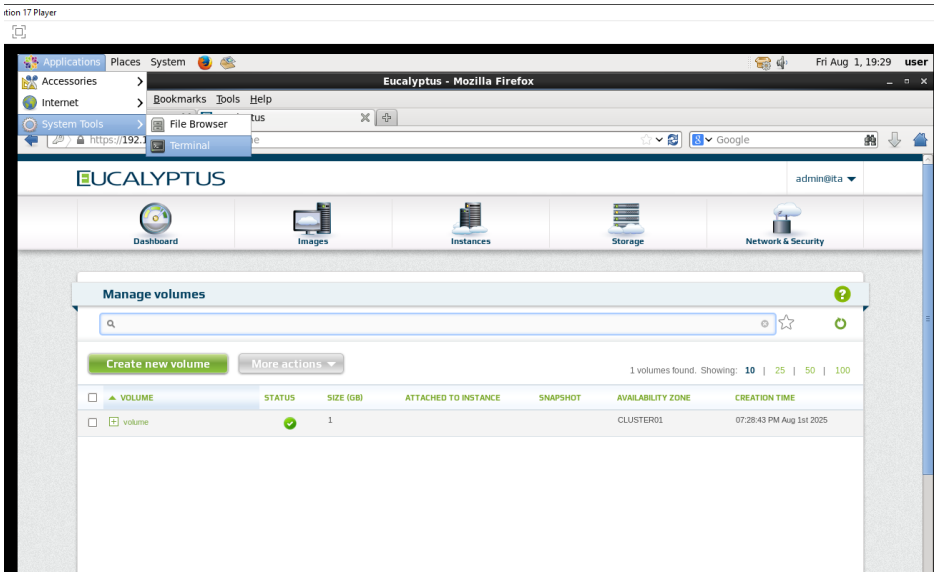

**RESULT:** Therefore, running the VM of different configurations is done successfully.

**EXP NO: 5**
**DATE:**

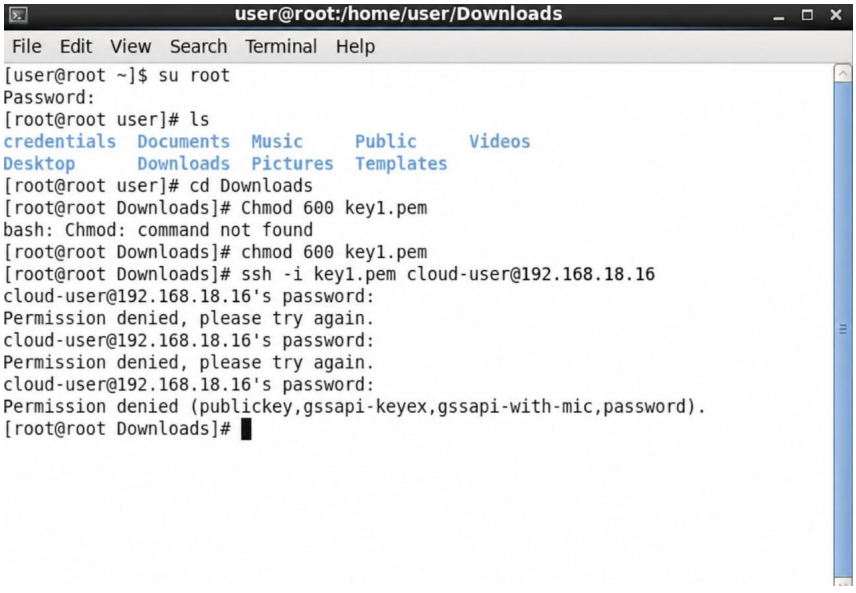
**AIM:**

**PROCEDURE:**

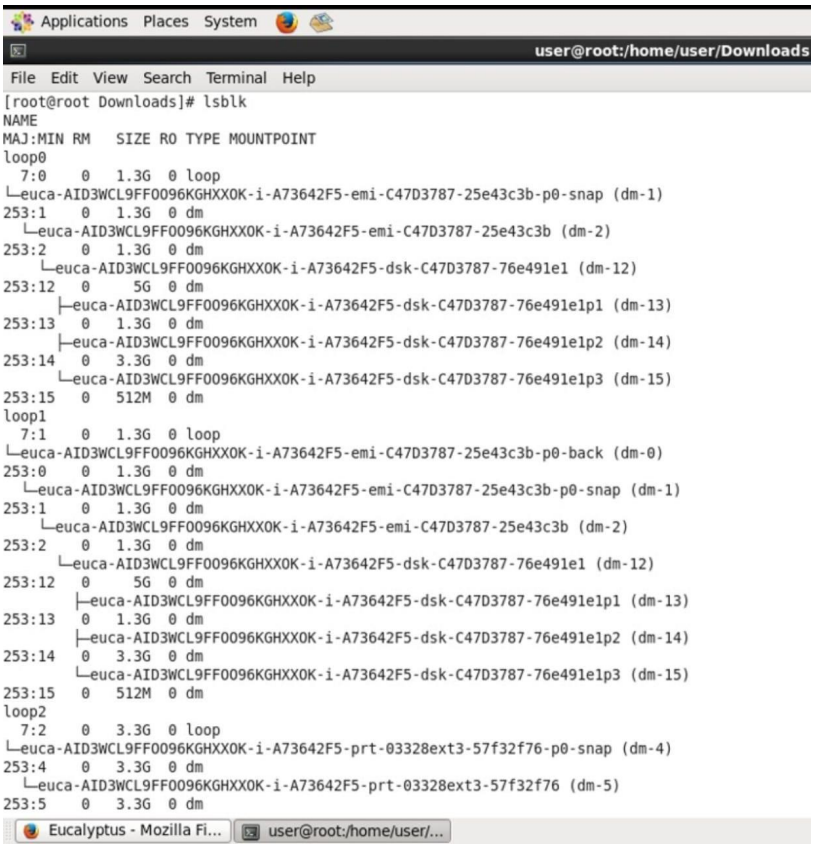
1. Create an instance in the Eucalyptus User Console.
2. In the desktop window, click on Applications, select system tool, then select Terminal.



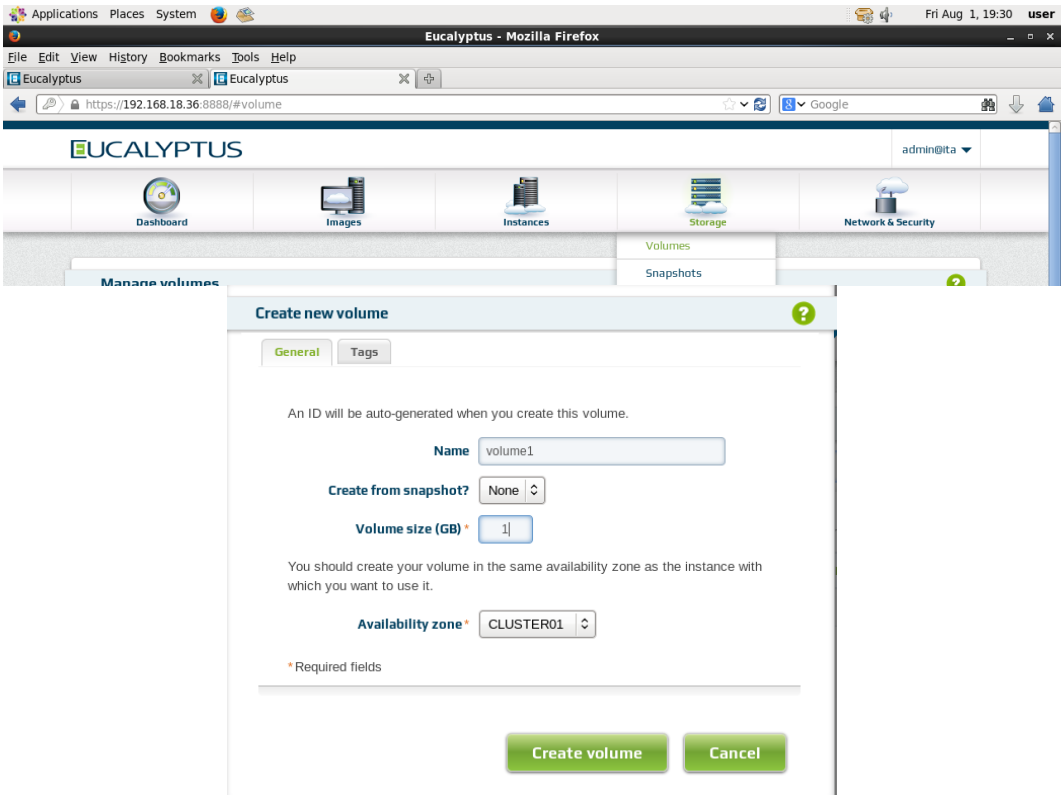

3. Type the su root command in the terminal, enter 123456 as password, then click enter.
4. List out the files and directories in the root using ls command.
5. Change the directory to Downloads using the cd Downloads command.
6. Type the command chmod 6000 key1.pem to change mode to 6000.
7. Then type the ssh-I key1.pem cloud-user@192.168.18.16 command.
8. Then enter password as 123456.



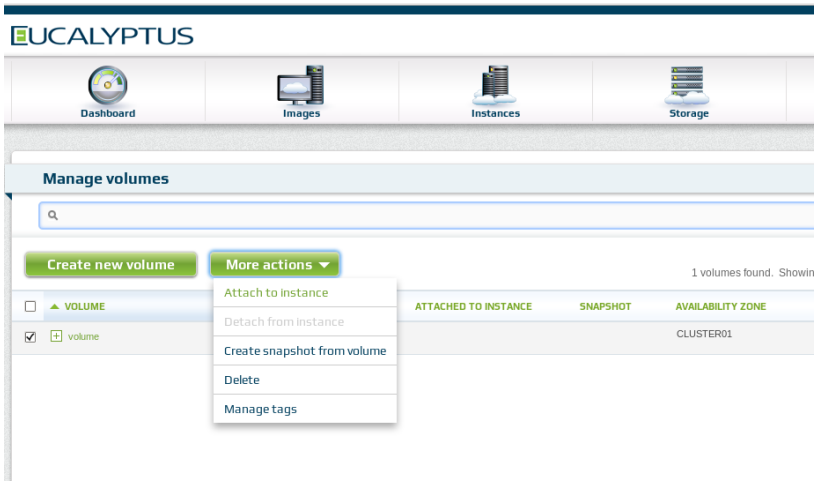
9. Then list out the blocks using lsblk.



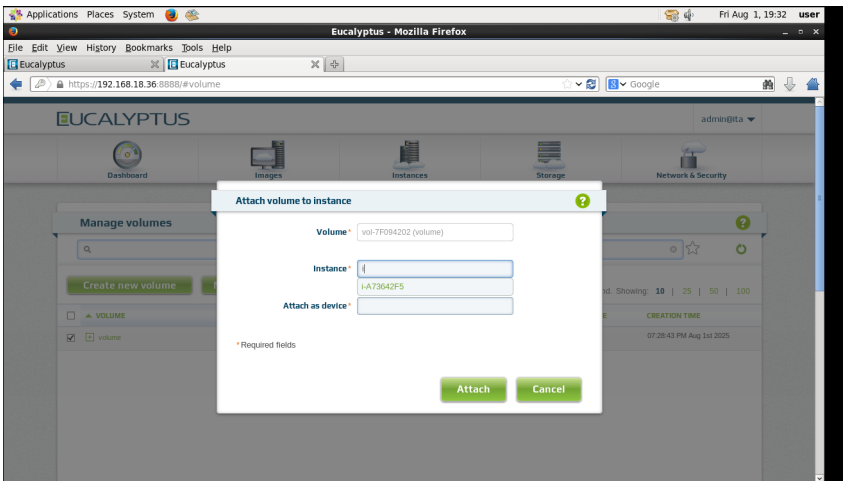

10. Now minimise the terminal, in the eucalyptus console, click on Storage, select volume, give name as volume1 and size as 1 GB and click on create volume.



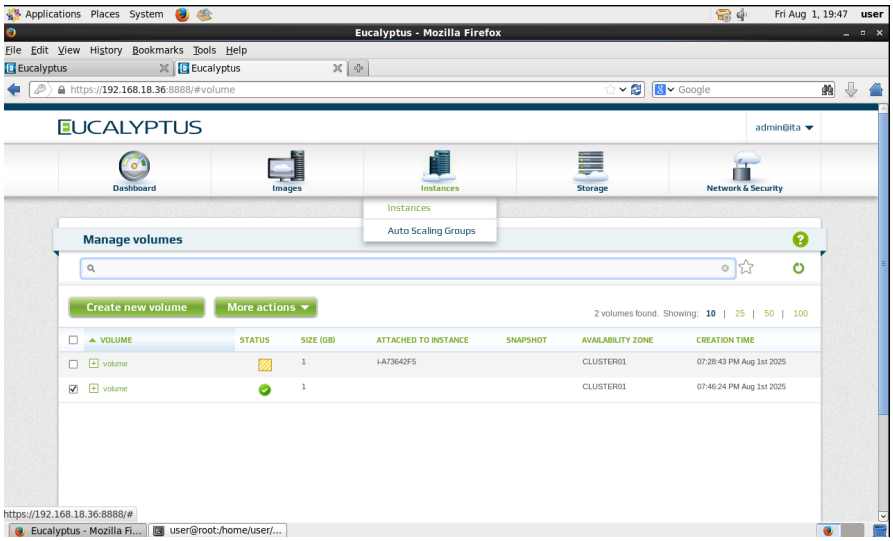
11. Select the volume, then click on more actions, then select attach instances.

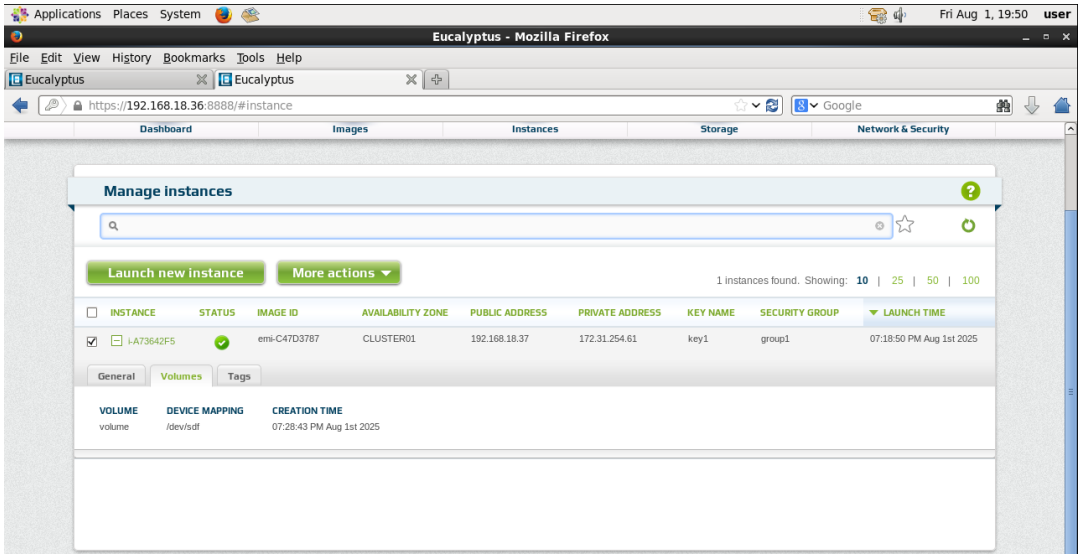



12. A screen with Default volume appears, then type the instance name, choose the instance number and then click attach option.




13. Now click on instances, expand your instance, click volume and check if the volume we created is attached to the instance we created.






14. To check if the volume is attached, enter lsblk command in the terminal again and see if the size is 1 GB.

```
└euca--ebs--storage--vg--vol--7F094202-euca--vol--7F094202 (dm-16)
253:16   0     1G  0 lvm
sr0
 11:0    1  1024M  0 rom
sda
  8:0    0   200G  0 disk
├─sda1
  8:1    0   500M  0 part /boot
├─sda2
  8:2    0 196.6G  0 part /
├─sda3
  8:3    0     3G  0 part [SWAP]
├─sda4
  8:4    0     1K  0 part
└─sda5
  8:5    0    18M  0 part
euca-zero (dm-9)
253:9    0     1P  0 dm
└euca-AID3WCL9FFOO96KGHXXOK-i-A73642F5-dsk-C47D3787-76e491e1-p0-snap (dm-11)
253:11   0  31.5K  0 dm
  └euca-AID3WCL9FFOO96KGHXXOK-i-A73642F5-dsk-C47D3787-76e491e1 (dm-12)
253:12   0     5G  0 dm
    ├euca-AID3WCL9FFOO96KGHXXOK-i-A73642F5-dsk-C47D3787-76e491e1p1 (dm-13)
253:13   0   1.3G  0 dm
    ├euca-AID3WCL9FFOO96KGHXXOK-i-A73642F5-dsk-C47D3787-76e491e1p2 (dm-14)
253:14   0   3.3G  0 dm
    └euca-AID3WCL9FFOO96KGHXXOK-i-A73642F5-dsk-C47D3787-76e491e1p3 (dm-15)
253:15   0   512M  0 dm
sdb
  8:16   0     1G  0 disk
[root@root Downloads]#
```

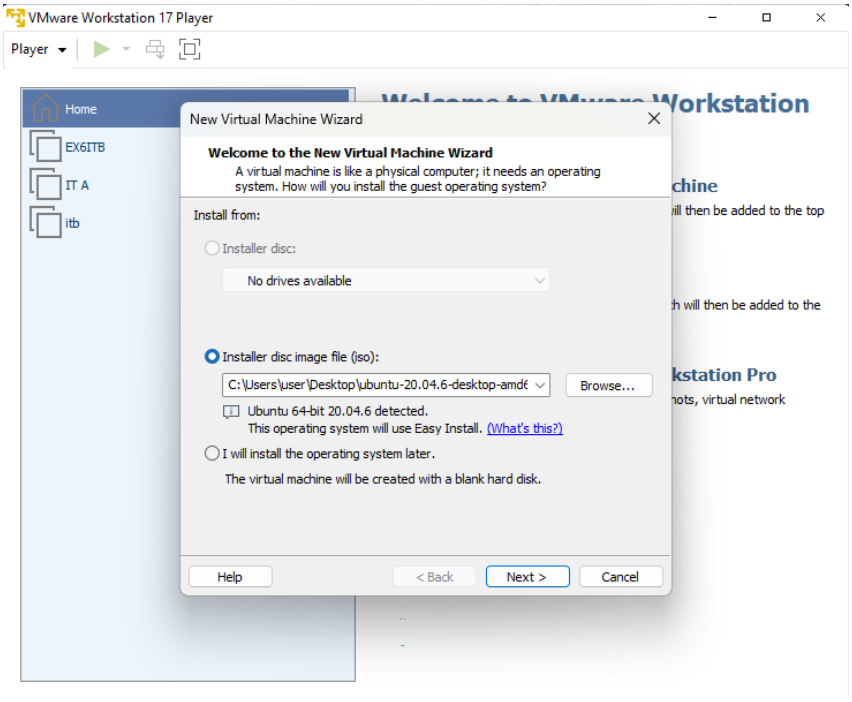
**RESULT:**

**EXP NO: 6** **INSTALL A C COMPILER IN THE VIRTUAL MACHINE AND**
**DATE:** **EXECUTE A SAMPLE C PROGRAM**

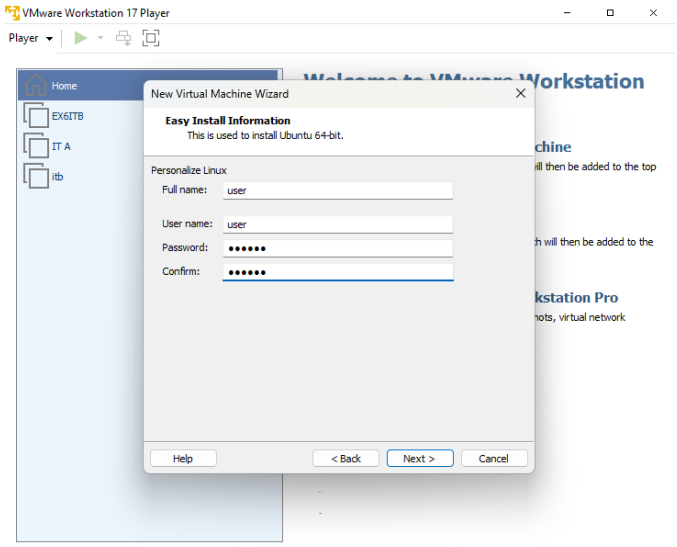
**AIM:** To install a C compiler in the virtual machine and execute a sample program.

**PROCEDURE:**

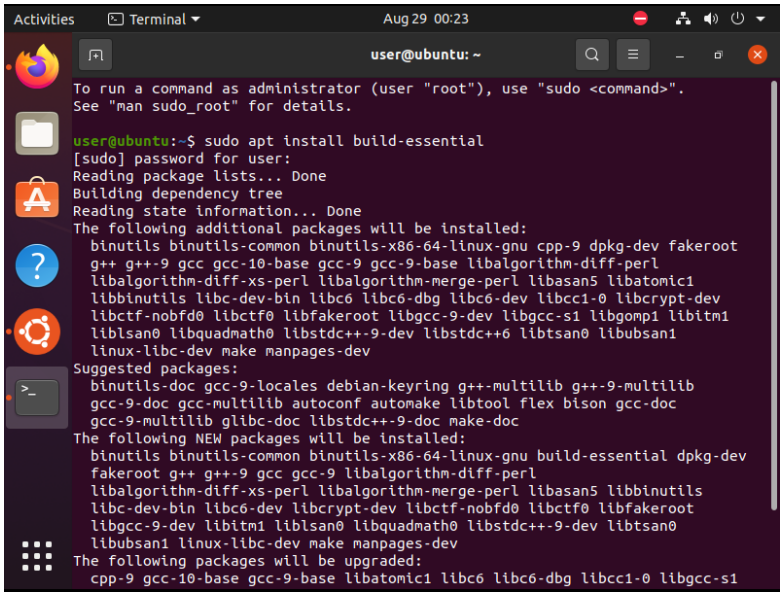
1. Open VMWare workstation and isntall UNIX OS by choosing unix AMD64 image file from the desktop.



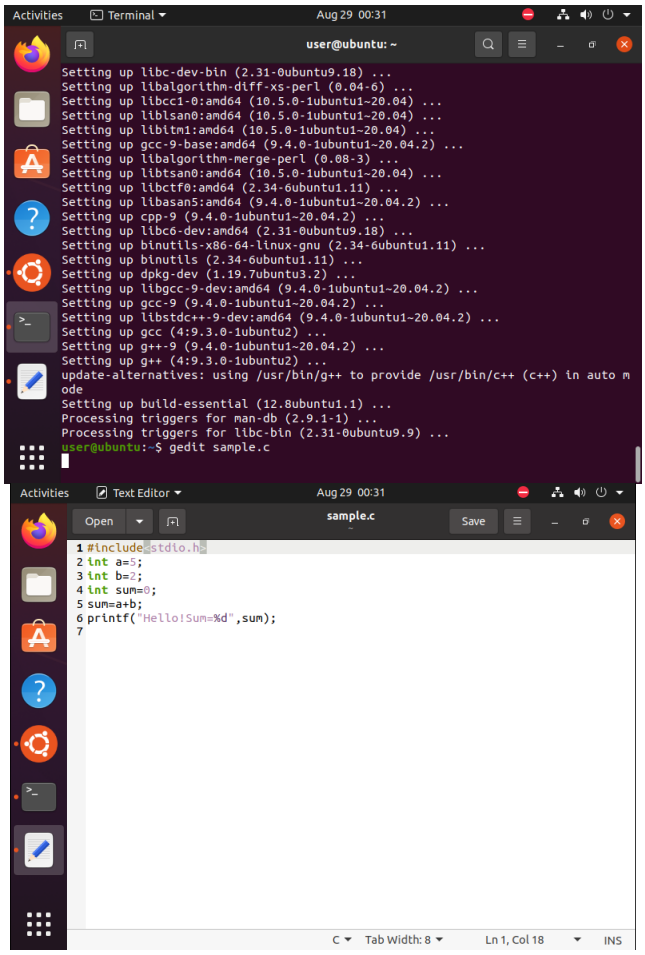

2. Enter Full name as user, user name as user and set password as 123456, confirm the password and click next.



3. Name your VM, follow the same steps as exercise 5. Finish once created and open the hotspot login.
4. Open the terminal in Ubuntu OS, install the C compiler using the command: sudo apt install build-essential and enter password as 1233456.




5. Create a sample C program using the command gedit sample.c



6. Compile C program using the command gcc, and then run the program using ./a.out command.

```
user@ubuntu:~$ gedit sample.c
^Z
[1]+  Stopped                 gedit sample.c
user@ubuntu:~$ gcc sample.c
user@ubuntu:~$ ./a.out
Hello!Sum=7user@ubuntu:~$
```

**RESULT:** Thus, the installation of C compiler in the virtual machine and execution of a sample program has been implemented.

**EXP NO: 7** **SHOW THE VIRTUAL MIGRATION BASED ON CERTAIN**
**DATE:** **CONDITION FROM ONE NODE TO THE OTHER**

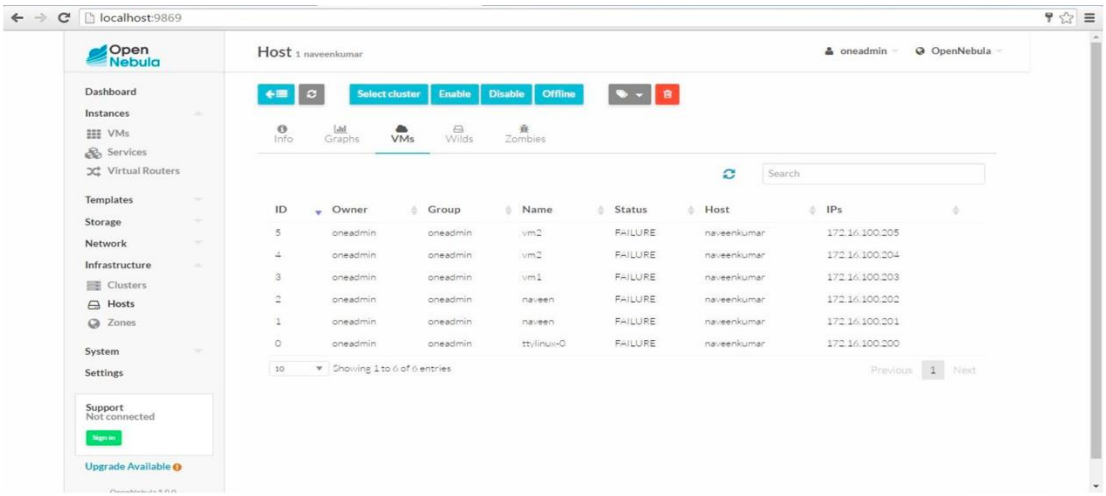
**AIM:** To show the virtual machine migration based on certain condition from one node to the other.

**PROCEDURE:**

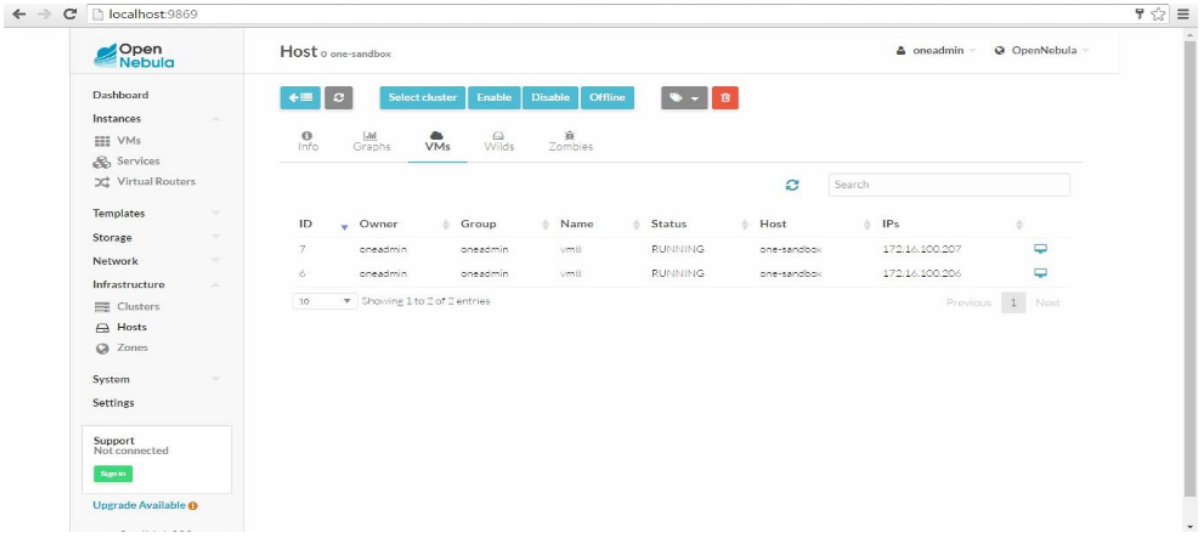
1. Open Browser, type localhost:9869
2. Login using username: oneadmin, password: opennebula
3. Then follow the steps to migrate VMs
    - Click on infrastructure
    - Select clusters and enter the cluster name
    - Then select host tab, and select all host
    - Then select Vnets tab, and select all vnet
    - Then select datastores tab, and select all datastores
    - And then choose host under infrastructure tab
    - Click on + symbol to add new host, name the host then click on create.
4. On instances, select VMs to migrate then follow the steps
    - Click on 8th icon, then the drop-down list will display
    - Select migrate, the popup window will display
    - Select the target host to migrate, then click on migrate

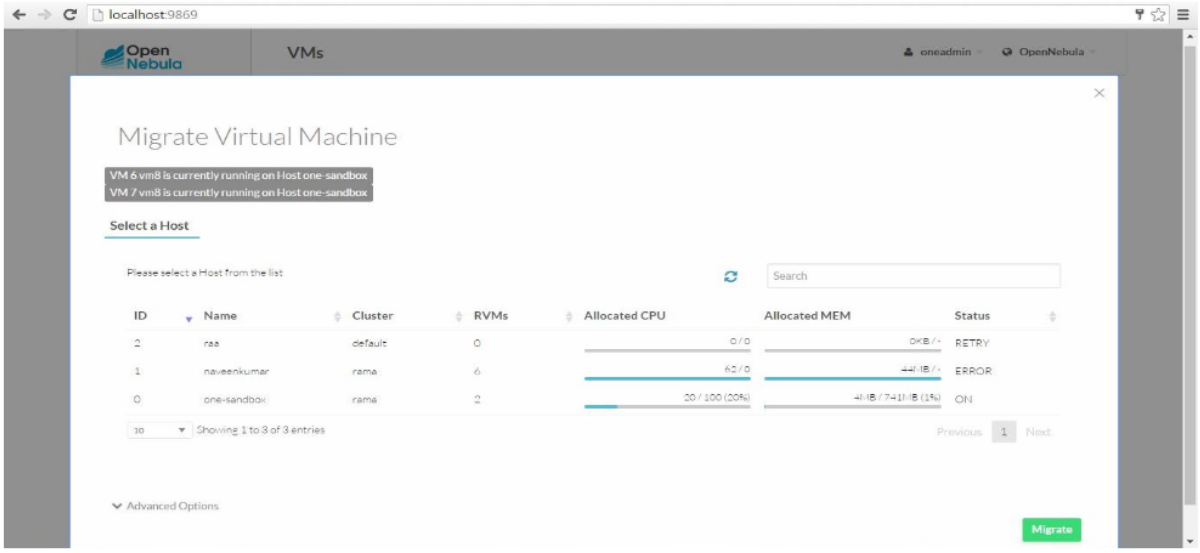
**BEFORE MIGRATION:**

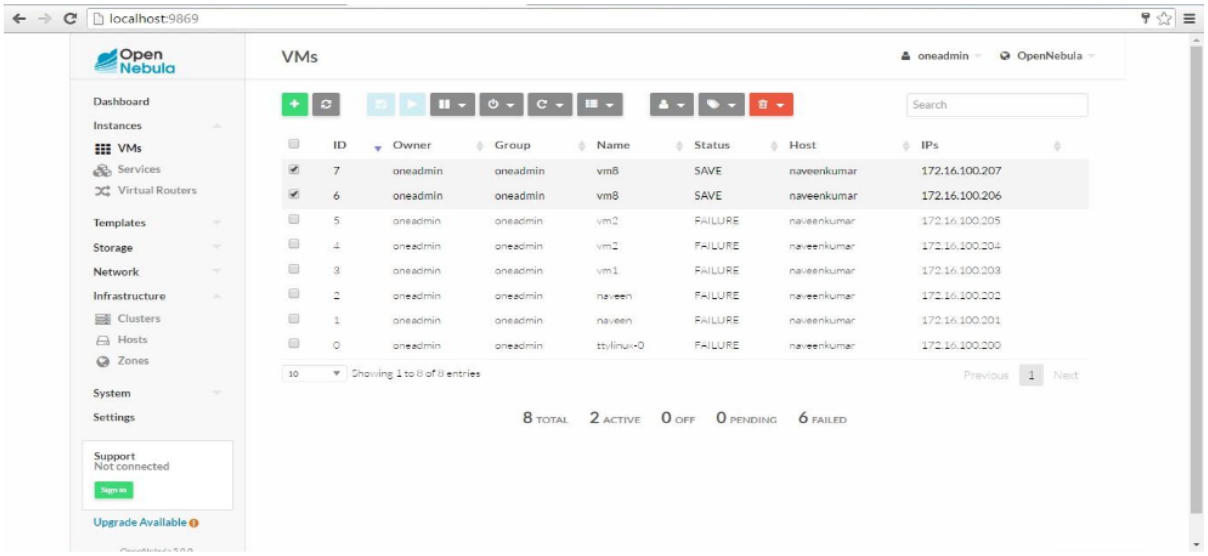
**Host: SVCE**



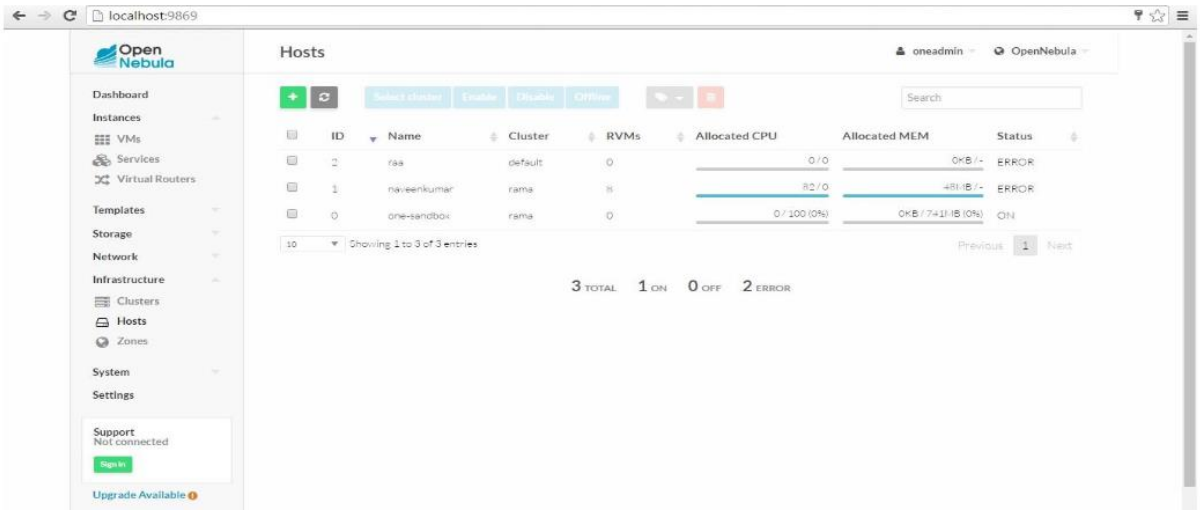
## Host: one-sandbox



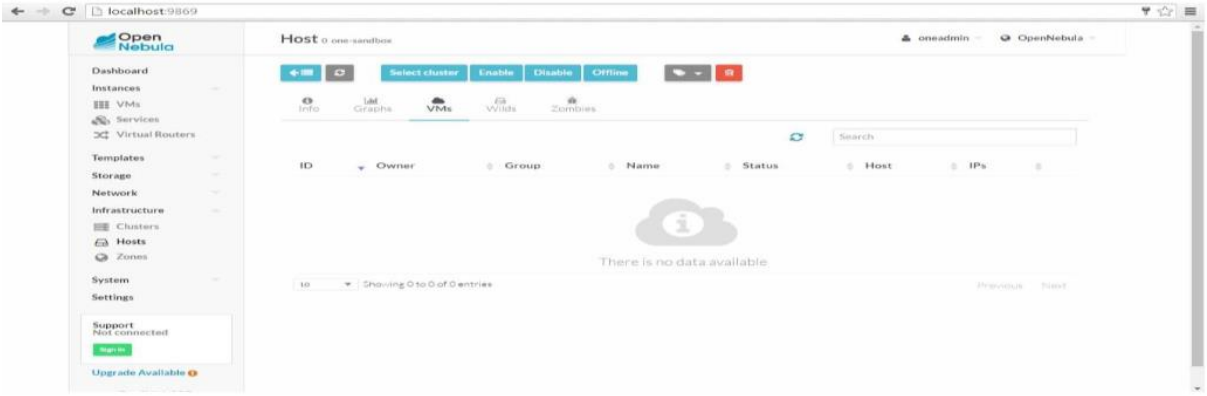



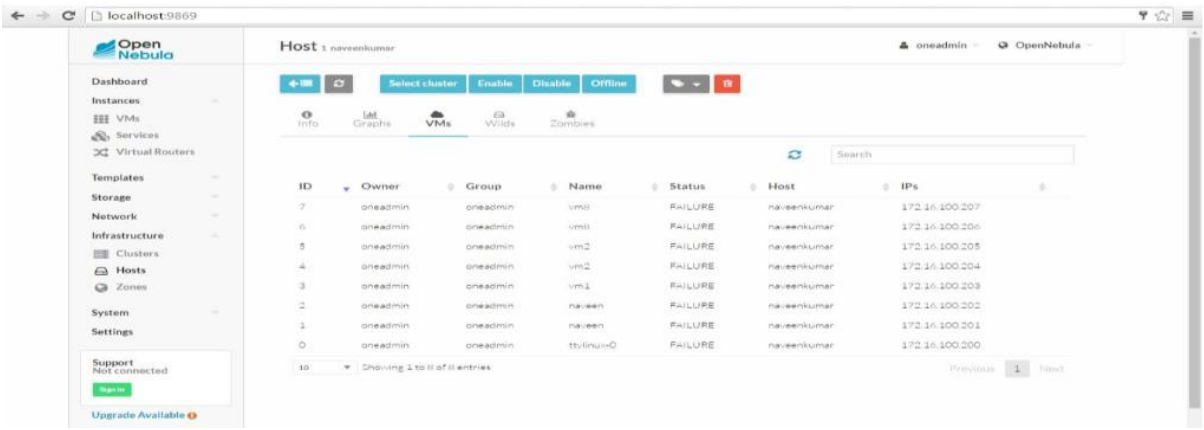
**AFTER MIGRATION:**




**Host: one-sandbox**




**Host: SVCE**




**RESULT:** Thus, the virtual machine migration based on the certain condition from one node to the other has been executed successfully.